Expense Management Tool for an IT Consultancy Company

Bachelor's Degree in Informatics Engineering
Specialization in Software Engineering

Bachelor's Thesis

Natalia Yike Wang Yang

Thesis Supervisor
Sergio Alejandro Lobig Machado (FSP Consulting Services Limited)
Thesis Director
Marc Oriol Hilari (Department of Software Engineering)

Universitat Politècnica de Catalunya
Facultat Informàtica de Barcelona
Computer Science Department

June 18, 2026

Contents

List of Figures

List of Tables

1 Context

This is a Bachelor's Thesis within the Software Engineering specialization of the Computer Engineering Degree, provided by the Barcelona School of Informatics at the Polytechnic University of Catalonia.

The project corresponds to modality B, as it is developed in a professional context in collaboration with FSP Consulting Services Limited. Consequently, this thesis is supervised under a dual-mentorship structure involving Marc Oriol Hilari, a lecturer in the specialization, and Sergio Alejandro Lobig Machado, a Senior Cloud Engineer at FSP.

1.1 Introduction

Organizations often hold company events that employees are required to attend. Travel to these events inevitably results in employees incurring various expenditures, such as flights, accommodation, and meals. As long as these expenses are business-related and not for personal use, companies typically offer reimbursement.

This is a common scenario within FSP's Barcelona team. FSP is a UK-founded private technology consultancy that has established Barcelona as one of its several office locations. As expected, the Barcelona-based team frequently travels to the UK for corporate events, meetings, and collaborative work, often struggling with high expenditures along the way.

The current method for managing these expense claims relies on a manual and document-based workflow. To do that, Barcelona's team must gather all tickets, record them in Excel spreadsheets, and email them to the Finance Department for processing and approval.

1.2 Key Concepts

The goal of this thesis is to apply and demonstrate the knowledge and competencies acquired throughout the author's Computer Engineering Degree. As a result, this document may contain jargon that could be challenging for readers without a background in this field. To facilitate reading of this thesis, the description of technical terms in this documentation is listed below and can be consulted as needed.

Expense Report Workflow

The full process of reporting an expense includes capturing the receipt, uploading the file, extracting information, submitting, and approving or rejecting the report.

Full-stack Application

An application that is composed of two parts, namely the frontend (visual user interface) and the backend (application's internal logic).

Pipeline

A set of ordered automated tasks executed sequentially.

Bottlenecks

Network traffic congestion.

Cloud Computing Platform

A virtual service that provides software development resources, replacing traditional physical resources.

RESTful API

API is the communication system between the frontend and backend, while REST (Representational State Transfer) is a widely recognized architectural pattern for designing APIs. REST standardizes API usage through uniform conventions, making it intuitive and straightforward for developers to understand and manage.

1.3 Problem Statement

The existing hand-operated expense reporting system at FSP introduces inefficiencies for both employees and the Finance department. Specifically, employees spend excessive time entering detailed receipt information in Excel spreadsheets. Concurrently, the Finance Department undertakes labour-intensive work in verifying outdated spreadsheets.

Moreover, from FSP's perspective, this manual data entry is prone to human error. According to the company's historical data, approximately 30% of submitted reports require manual correction due to transcription errors or missing information. Consequently, this not only increases the risk of inaccuracy but also results in expenses being delivered to the wrong person.

Quantitatively, FSP's Barcelona team of 25 employees departs for the UK 2-4 times annually, generating an average of 8 receipts per person per trip. Each employee spends about 20 minutes reporting a single expense, while the Finance Department (4 people) takes 20 minutes to review the full set of expenses for each employee per trip. Collectively, this unproductive task accounts for roughly 225 lost hours per year.

Therefore, the time consumed in this burdensome activity could be considerably reduced, enabling highly skilled employees to focus on activities that deliver higher value to the company. Hence, there is a need to optimize FSP's personnel investment.

1.4 Proposed Solution

FSP clearly lacks a more intelligent, automated expense management strategy to prevent employee frustration and guarantee accurate reimbursement of business expenses. This project aims to develop a web application that allows employees to manage, report, and submit expenses seamlessly on a single platform.

This application streamlines the entire workflow by offering effortless receipt uploads, AI-automated data extraction, and efficient expense report verification and approval. The primary goal is to simplify the previous expenses management workflow, optimizing time and effort for all parties, reducing human error, and ensuring information is transmitted securely and efficiently to the appropriate approver.

1.5 Stakeholders

With the problem and solution established, all parties involved in this project (also known as stakeholders) are identified below.

1.5.1 Product Owner

The Product Owner is responsible for understanding the client's interests and communicating them effectively to the application development team. Their primary role is to maximize the value of the final product from the client's perspective. They define the product's vision and requirements, ensuring that the application's features remain aligned with client needs throughout the development lifecycle

In this case, Sergio Alejandro Lobig Machado serves as Product Owner, as he is the only FSP representative on this project.

1.5.2 Thesis Director

The Bachelor's Thesis Director oversees the project from an academic perspective. Marc Oriol Hilari fulfills this role, continuously monitoring progress to ensure that all required technical and transversal competencies are achieved by the end of the project.

1.5.3 Development Team

The development team is responsible for the full implementation of the application, from initial pipeline setup and database design to final testing and product deployment. For now, this role is carried out solely by the thesis author. However, this does not rule out the possibility of extending and upgrading the project in the future, potentially adding new developers to the team.

1.5.4 End-users

This application addresses FSP's Barcelona Team needs; as a consequence, they become the primary beneficiary of the product and are part of he application's end-users.

Another subgroup of end-users is the Finance Department, which is responsible for processing the employee's reimbursement once the expense report is approved.

1.5.5 IT Administrator Team

FSP's IT department serves as the application's IT administrator team. IT administrators review and verify users' logs for security purposes and solve any technical problems.

2 Justification

Given the project's focus on developing an expense management tool for FSP (Section 1.4), it is necessary to justify the project's execution. This section helps define the project's scope, objectives, and requirements, which are discussed in Section 3.

Although the current market offers numerous off-the-shelf solutions, they impose significant architectural limitations. Adopting a third-party platform implies conforming to its predefined AI model, functional workflow, and limited integration points. Since FSP is a technology consultancy, it is not seeking a solution that simply mirrors market offerings, but rather a more agile, customized approach.

Consequently, an in-house, scalable expense management application is preferred at FSP because it provides three key architectural advantages over commercial products:

- **Horizontal Scalability**

  The application is built on a modern, cloud-native architecture based on decoupled microservices. This design allows each component to scale independently as the company grows. Scalability is often a major weak point of external providers, since it is beyond their control.

- **Maintainability and Extensibility**

  The application codebase is owned by FSP and follows Clean Architecture principles by separating technical and business logic. This facilitates extension with new features and adaptation to changing business needs. As a result, long-term maintenance becomes painless, while FSP gains full control over the application lifecycle.

- **Easy Integration**

  By implementing a decoupled, multi-layered Clean Architecture, the logic for interacting with any external system is encapsulated behind abstractions. The concrete implementations of these abstractions reside in a completely separate layer. Additionally, external systems are registered via abstract interfaces to adhere to the Dependency Inversion Principle. As a result, external services can be easily integrated without requiring critical changes in the backend API.

At the same time, this project represents the author's first independent application development. Consequently, it provides a valuable opportunity to demonstrate key Software Engineering competencies in a real-world context. In short, it is an ideal case that bridges the author's academic background and professional career.

3 Scope

As stated in the previous section, the main goal of this project is to **develop an in-house expense management web application that is scalable, maintainable, and integration-friendly**. To better achieve this objective, this section provides a more detailed breakdown of the project goal, specifies the application's functional and non-functional requirements, and concretizes the potential obstacles and risks.

3.1 Objectives

For a clearer understanding of the project scope and easier evaluation of its completion, the project's objective is broken down into smaller, more manageable sub-objectives:

SO1. Design the application's technology stack and the database structure.

SO2. Develop a full-stack web application.

SO3. Design a scalable pipeline to automate AI-driven data extraction from expense uploads.

SO4. Use a cloud infrastructure platform to manage external application resources.

SO5. Maintain a clean, reusable, maintainable, integration-friendly, and high-quality application codebase.

3.2 Requirements

This section introduces the application's requirements, which are defined based on the project proposal document drafted by the Product Owner.

Following a Software Engineering approach, requirements are classified into two main groups: functional and non-functional. The former defines what features the application provides, while the latter describes how those features are delivered [1].

3.2.1 Functional Requirements

Due to the constrained thesis timeline, the application's functional requirements are prioritized by applying the MoSCoW methodology. MoSCoW categorizes requirements into four priorities: **Must have**, **Should have**, **Could have**, and **Won't have**.

Throughout the full classification process, the Product Owner serves as the primary reference point for determining the relevance of each requirement. The resulting MoSCoW-prioritized requirements list appears below. Each requirement is also linked to a specific development task, which is further detailed in Section 5.1.

By following the MoSCoW strategy, high-priority requirements are addressed first, while lower-priority ones are deferred in case of setbacks. Consequently, this enables early delivery of a high-value product; only requirements with negligible outcome impact can be omitted if time allocation falls short.

Before delving into the specific requirements, notice that a report is a set of AI-processed and user-reviewed expenses submitted collectively to the reviewer for approval or rejection of the corresponding total reimbursement amount.

Must have: requirements that are critical for the core application functionality.

FR1. The system must centralize expense data on a single platform. (All ER category tasks)

FR2. The system must provide effortless receipt uploads and drag-and-drop functionality. (ER1, BD1, BD2, BD3)

FR3. The system must automate data extraction from receipts using an AI model. (BD4, BD5)

FR4. The system must display the current AI processing status of uploaded expenses. (ER2, BD6)

FR5. The system must allow users to review and correct the AI-extracted expense data. (ER3, BD2)

FR6. The system must allow users to delete single or multiple uploaded expenses that are not attached to a submitted report. (ER4, BD2)

FR7. The system must allow users to create, edit or delete reports. (ER5, BD2)

FR8. The system must allow users to attach and detach single or multiple expenses to a report pending submission. (ER5, BD2)

FR9. The system must enable simple report submissions to a predefined approver. (ER5, BD2)

FR10. The system must require users to log in through their company's account. (UA1, BD7)

FR11. The system must implement role-based access control with two roles: employees, who have access to basic features, and admins, who have access to additional features. (UA2)

FR12. The system must allow admin users to approve or reject submitted reports. (UA3, BD2)

FR13. The system must send email notifications to the report creator when a report is approved or rejected, and to the creator's supervisor when a report is submitted. (UA6)

FR14. The system should display the report's current status, categorized as Not Sent, Pending, Approved, or Rejected. (ER5)

Should have: requirements that are relevant for the application functionality, but permit a certain delay in complete achievement.

FR15. The system should provide search bar and filters for uploaded expenses. (FD1)

FR16. The system should display to employee users a history of reports in each of the following statuses: Not Sent and Submitted. (UA5)

FR17.The system should display to admin users a history of reports in each of the following statuses: Pending Review and Reviewed. (UA5)

Could have: requirements that enhance the application usability and are fulfilled if time

permits.

FR18. The system could provide a search bar to find specific reports. (FD2)

FR19. The system could allow admins to set expense budget limits for employees under their supervision. (UA4)

Won't have: requirements that are irrelevant and are not be included in the application. As they fall outside the application's scope, these requirements are not explicitly defined.

3.2.2 Non-Functional Requirements

The subsequent requirements fall under the non-functional category. A brief description and acceptance criteria are provided in each requirement for final project validation.

NFR1. The application must provide real-time updates on expense processing status.

The UI must update the expense statuses in less than **1 second** once the AI-driven data extraction is completed.

NFR2. The application must offer an intuitive UI.

The UI must be intuitive, allowing new users to learn how to use the application within **3 minutes**, establishing a low learning curve.

NFR3. The application must ensure high performance and availability.

Upon user login, the application should load the previously uploaded expenses within **3 seconds**. Other basic user interactions (e.g., saving expense information, deleting expenses) should be completed under **500ms**. Additionally, the application should be available **24 hours** a day, **365 days** a year.

NFR4. The application must support scalability as the company grows.

The application must implement a hybrid architecture that separates the initial user request from any subsequent intensive processing workload. Additionally, its resources must be deployed on a cloud computing platform (or Platform-as-a-Service) that supports automatic horizontal scaling.

NFR5. The application must guarantee compliance with GDPR.

The acceptance criteria of this specific requirement are justified in Section 14.1.

NFR6. The application must enable a more efficient expense management process and a streamlined approval workflow.

The application must optimize the expense workflow from the current 20 minutes to **5 minutes** for both employees and finance staff, achieving a **75%** efficiency gain.

NFR7. The application must support easy integration with external systems.

The application must define abstract interfaces for interactions with external systems, with their concrete implementations placed in a separate layer. The relationship between these abstractions and their implementations is established by adopting the Dependency Inversion Principle. This architectural approach ensures that future integrations of ex-

ternal components can be achieved with minimal code changes, thereby improving maintainability and extensibility.

NFR8. The project must support maintainability and extensibility.

The application must separate the technical implementation from the business logic, facilitating the extension of technical features or adaptation to a changing business structure.

3.3 Obstacles and Risks

Identifying obstacles and risks is crucial for more precise initial timeline estimation (further established in Section 5.4). Hence, this section outlines obstacles and risks that may hinder project execution. The analysis of their probability, impact and mitigation strategies is covered in detail in Section 5.5.

3.3.1 Coding Bugs

As the application development progresses, certain coding bugs will inevitably emerge, requiring additional time to fix. Given the high probability of encountering these risks, minor delays in the project are expected.

3.3.2 Tight Project Deadlines

The project timeline is constrained by the thesis deadline. Significant delays may affect the final product from fully meeting all defined requirements, representing a major challenge for the author.

3.3.3 Lack of Experience in Certain Technologies

The two previous situations are quite common in real-world engineering practice, and this one is no exception. Exploring and applying new technologies is a constant part of a software engineer's daily work.

3.3.4 Company's Licenses and Security Permissions

This project is executed within FSP's environment. The company's strategies to prevent cyberattacks may restrict the installation of new tools and resources. This supposes some constraints for the project, as certain installations are required throughout the development.

4 Methodology

ScrumBan is the methodology adopted trhoughout the project execution. Hence, the selection of monitoring tools have been selected according to this methodology. The following subsections provide a detailed description of the development methodology and the monitoring tools used to track the project's progress.

4.1 Development Methodology

This project applies the Agile methodology based on the ScrumBan framework. ScrumBan is a hybrid approach of Scrum and Kanban. Scrum features role assignments, focuses on team coordination, and aims to increment the product's value through predefined iterations (known as sprints). Kanban emphasizes flexibility with continuous workflows, incremental delivery, and strict work-in-progress (WIP) limits [2].

ScrumBan offers a wide range of options combining Scrum and Kanban methodologies, facilitating its adaptation to the project's context. The main roles defined for this project are a subset of Scrum roles: Product Owner and Development Team (their role descriptions and assignments are found in Section 1.5).

By contrast, the full development cycle reflects the continuous nature of Kanban. Since the development team consists of a single developer, traditional Scrum ceremonies such as sprint planning, sprint review, and sprint retrospective are excessive and therefore omitted. Instead, regular meetings with the Product Owner are scheduled every 2 days throughout the project to monitor progress.

Before starting the practical implementation, a product backlog (from Scrum) with all the project's tasks is defined to keep the project on track. Simultaneously, it is complemented by Kanban's pull principle, setting a **Work In Progress (WIP)** limit of **3**. WIP restricts the number of "in progress" tasks, preventing work overload and blocking next-steps tasks until any active task is completed.

4.2 Monitoring Tools

The upcoming monitoring tools in this section are used to facilitate the application of Agile methodology.

Git

A cloud version control system that allows multiple developers to work on the same project within a shared repository, and enables a single user to develop several features at the same time. These capabilities make Git essential for the Kanban's continuous flow principle, allowing developers to proceed with new tasks when the current one is blocked.

In Git, a repository is the location where the application's source code is stored. It serves as a project space where developers submit their code, allowing changes to be tracked throughout the development process. Using a Git repository for source code version control provides high availability since it is cloud-based.

Azure DevOps

Azure is a cloud platform that offers multiple tools; among them, two relevant monitoring tools are used for this project: the Git repository (established earlier) and the backlog dashboard (for product backlog management).

In the backlog, four statuses are assigned to tasks:

- **New**: A task that is defined but is pending development, or a bug that is pending resolution.
- **Active**: A task that is currently in progress. As clarified in Section 4.1, there are up to 3 tasks in Active status, according to the WIP limit.
- **Resolved**: A bug that has been fixed.
- **Closed**: A task that has been developed, integrated, and tested.

Microsoft Teams

A communication tool used for conducting regular meetings with the Product Owner, which is mentioned earlier in Section 4.1.

5 Temporal Planning

This section introduces the initial **122-day** project plan, scheduled from February 18, 2026, to June 19, 2026. The total estimated effort for all defined tasks is **513 hours**.

According to Polytechnic University of Catalonia, the bachelor's thesis carries a workload of 18 ECTS credits, equivalent to 540 effort hours. The discrepancy between the total estimated effort of 513 hours and the expected 540 hours results in a built-in buffer of **27 hours** to accommodate potential setbacks.

5.1 Task Description

The key to achieving effective time planning lies in a thorough analysis of all tasks, their priorities, and durations. This step ensures proper task sequencing and resource allocation.

Before this analysis, all tasks must be identified and categorized. For this reason, this section introduces the project tasks and groups them into: project starting, project management, final project delivery, design, development, and frontend and/or backend development. Notably, each task in the *frontend and/or backend development* category corresponds to a single functional requirement, as referenced in Section 3.2.1.

5.1.1 Project Starting

A solid project start requires a preliminary study and initial project definition. This aligns the project vision among all involved members and establishes a unified understanding of the project's scope and objectives. To achieve this, the subsequent tasks are defined:

PS1. Context and Scope

This task is divided into smaller sub-tasks:

- Analyze the current problem and identify all stakeholders.
- Define and propose a technical solution.
- Justify the project execution.
- Define the project scope in terms of objectives, requirements, obstacles, and risks.
- Select a methodology to guide project execution.
- Select the monitoring and validation tools to be used in the project.

PS2. Temporal Planning

Identify all tasks and resources, design an initial Gantt diagram timeline, and define a risk management strategy.

PS3. Budget and Sustainability

Define a project budget and analyze the sustainability viability of the proposed solution.

PS4. Initial Documentation

Consolidate the previous documentation into a definitive initial documentation.

5.1.2 Project Management

During this stage, the project's progress is monitored to keep it on track.

PM1. Product Owner Meetings

Run regular meetings with the Product Owner to discuss the development progress, implementation priorities, and any issues.

PM2. Thesis Director Review Meetings

Meet with the thesis director every two weeks to ensure the project is progressing correctly.

PM3. Product Backlog Update

Refine, update, and maintain the product backlog.

5.1.3 Final Project Delivery

Tasks for completing product delivery are listed below.

FPD1. User Manual Documentation

Write a user manual for the application's end-users as a reference guide.

FPD2. Final Thesis Documentation

Write the bachelor's thesis final documentation intended for an academic audience.

FPD3. Final Demonstration Video

Record a final demonstration of the application once it is ready for use.

FPD4. Bachelor's Thesis Defense

Plan and prepare a final exposition to defend the bachelor's thesis.

5.1.4 Design

This preliminary design phase establishes a clear application architecture to guide the subsequent development phase.

DT1. Use Case Diagram

Analyze all users' interactions with the system and design a Use Case Diagram that illustrates all actors, use cases, and relationships between use cases.

DT2. Conceptual Model Diagram

Design the UML database diagram, clarifying all database entities, their attributes, and the relationships between them.

DT3. Technology Stack Decision

Select the technology to be used for each application layer and provide a clear technology selection documentation that serves as a guide for the application development phase.

DT4. Logical and Physical Architecture Design

Design the logical and physical application architecture. The former describes how the implementation code is structured, including the purpose and role of each application layer. The latter explains how the application connects to external services and resources.

DT5. UI mockups Design

Deliver the mockup designs for each application page.

DT6. Security Design

Provide a security design for user authentication, access control, and information submission and storage.

5.1.5 Development

Essential tasks for the application development are specified in this task group.

Dev0. Self-learning and study of unfamiliar technologies

Study and learn new application technologies in advance.

Dev1. Environment Configuration

Set up a ready-to-use environment before starting application development such as installing Visual Studio Code and essential extensions, creating a Git repository, and performing other initial configuration tasks.

Dev2. Product Deployment

Release the final product for end-user availability.

Dev3. CI/CD Pipeline Implementation

Implement Continuous Integration and Continuous Delivery pipeline to automate compilation, unit and integration testing, and building of every updated code version.

Dev4. Logging and Monitoring

Set up logging and monitoring tools to track application performance and review logs.

Dev5. Git Repository Documentation

Maintain clear repository documentation for a technical audience, including an application setup guide, API contracts documentation, and meaningful code comments.

5.1.6 Frontend and Backend Development

As a full-stack web application, the implementation is divided into two main areas. The frontend is responsible for the user interface (UI) and user experience (UX). The backend implements the complex internal logic hidden from the frontend via a RESTful API and integrates with external services.

This development stage requires both frontend and backend development. Since it covers a wide range of functionalities, it is divided into two subgroups: Expense and Report Development, and User Access Development.

Expense and Report Development

This development subgroup includes the application's core functionalities, namely, the expense and report management.

ER1. Expense Uploads

Develop a file submission feature where the frontend handles the expense drag-and-drop and upload interface, and the backend registers the uploaded expense in the database.

ER2. Uploaded Expenses Real-time Dashboard

Develop a visual dashboard to display all uploaded expenses with their real-time processing status, which can be:

- **Processing:** The expense is pending processing by the AI model.
- **Pending Review:** The AI model has successfully analyzed the expense; the uploader can now review the AI-extracted information.
- **Reviewed:** The user has reviewed and completed all required information. The expense is ready to be included in a new or existing report.
- **Failed:** The AI couldn't extract data from the receipt (due to unavailability or unrecognized format). The uploader should manually fill in all required expense details.

ER3. AI-Extracted Expense Data Review and Correction

Develop a form page for users to review, edit, and save AI-extracted expense information, while updating the database with the corrected data.

ER4. Uploaded Expenses Deletion

Develop the expense deletion feature to delete one or multiple expenses at once, regardless of their current status. After user confirmation, they are marked as deleted in the database.

ER5. Expense Reports View, Creation, Edit, Submission, and Deletion

Develop report management functionalities (create, edit, submit, and delete reports) and the required RESTful API for synchronizing the database with the frontend changes.

User Access Development

The tasks described in this subgroup are contingent upon user authentication implementation before their execution can begin. It is important to note that administrators can set spending limits and manage and view submitted reports solely from employees under their supervision.

UA1. Company Email User Authentication

Develop the login feature that allows users to authenticate in the application through the company's email account, including:

1. User access verification.
2. (a) New account creation on first login.
   (b) Existing user data recovery for returning users.

UA2. Role-based User Access

Develop the user role assignment (based on two main roles: regular employee or administrator) and expose to the user only their corresponding role's functionalities.

UA3. Submitted Report Approval and Rejection

Develop a feature to allow administrators to review, approve, or reject submitted reports, and update the database accordingly.

UA4. Employee Spending Limit Configuration

Develop an admin-only interface to set employees' spending limits and record the changes in the database.

UA5. Submitted and Reviewed Expenses Records Visualization

Develop a dashboard that displays records of users' submitted reports and administrators' reviewed reports.

UA6. Email Notification Service Integration

Develop the email notification service to notify users when their submitted reports are reviewed, and to administrators when new reports are pending for their review.

5.1.7 Backend Development

The following tasks are implemented exclusively in the application backend logic.

BD1. RESTful API Backend Logic Implementation

Build a RESTful API that follows the technology stack defined in the DT3 task to manage backend responsibilities via frontend requests.

BD2. Azure SQL Database Integration

Integrate with Azure SQL Database using Entity Framework Core migrations to manage the application's data storage and configure the initial database according to the diagram designed in the DT2 task.

BD3. Azure Blob Storage Integration

Store original receipt files using Azure Blob Storage.

BD4. Microsoft Document Intelligence AI Model Integration

Extract the relevant information from expenses using the Microsoft Document Intelligence AI model.

BD5. Automated AI-driven Data Extraction Pipeline

Integrate Event Grid, Service Bus, and Azure Function to implement an automated data extraction workflow that prevents bottlenecks during multiple simultaneous uploads:

1. Save the uploaded receipt file in Blob Storage and associated data in SQL Database.
2. Detect the event using Azure Event Grid and route the file to the Service Bus queue.
3. Once the new expense is in the queue, trigger the Azure Function to process the receipt file using the integrated AI model (BD4 Task).

BD6. Azure SignalR Real-time Notification Service Integration

Integrate SignalR to configure a SignalR Hub that sends real-time expense status updates, eliminating manual page refreshing whenever the AI processes a new receipt file.

BD7. Microsoft Entra ID User Authentication Integration

Handle employee authentication through the company email account via Microsoft Entra ID.

5.1.8 Frontend Development

This category covers all tasks that require only frontend logic implementation.

FD1. Expenses Search and Filter

Develop visual tools that enable users to search and filter uploaded expenses.

FD2. Reports Search

Provide a search bar to search specific reports by name.

5.2 Resources

Following the analysis of all required tasks for successful project completion, all necessary resources must be identified. Resources are divided into two main groups: human resources and physical resources. All human resources are listed in Table 5.2.1, specifying the responsible individual for each role. The physical resources are further categorized into software and hardware resources, identified and described in Tables 5.2.2 and 5.2.3.

5.2.1 Human Resources

Resource ID	Role Name	Responsible Individual
PO	Product Owner	Sergio Alejandro Lobig Machado
TD	Thesis Director	Marc Oriol Hilari
GT	GEP Tutor	Paola Pinto Del Monte
DT	Development Team	Thesis author

Table 5.2.1: Human resources. [Own creation]

5.2.2 Physical Resources

Hardware Resources

Resource ID	Resource Name	Components
OS	Office Supplies	A table, a chair, electricity, internet, and office rental.
HW	Hardware Resources	A computer, two monitors, a mouse, and a keyboard.

Table 5.2.2: Hardware resources. [Own creation]

Software Resources

Resource ID	Resource Name	Description
GIT	Git	A version control tool for managing the source code repository.
AZR	Azure Resources	Cloud services and resources: SQL Database, Blob Storage, AI model, Event Grid, Azure Function, Service Bus, SignalR, EntraID, and Azure DevOps.
MCS	Microsoft Tools	Communication and coordination platforms: Microsoft Teams, Calendar, Outlook, and Word.
VSC	Visual Studio Code	Source code editor with extensions and debugging tools.
FIB	FIB Platforms	Platforms from Barcelona School of Informatics(FIB): Racó, Atenea, FIB secretary, bachelor's thesis official website guide.
GM	Gmail	Email communication with university entities.
WEB	Web Browsers	Research Browsers: Mozilla Firefox, Google Chrome, Microsoft Edge.
OVL	Overleaf LaTeX Editor	Thesis formatting editor for final thesis documentation.
OT	Online Tools	Diagram design tools: Visual Paradigm Online and Gantt Project.

Table 5.2.3: Software resources. [Own creation]

5.3 Summary of Tasks and Resources

See Table 5.3.1 for a summary of all defined tasks, including their identifiers, estimated effort in hours, any Finish-Start (FS) dependencies with other tasks, and the required resources. The *Dependencies* column indicates a finish-to-start dependency, meaning that the listed task must be fully completed before the task in that row can begin.

ID	Name	Time (h)	Dependencies	Resources
PS	**Project Starting**	**50**		
PS1	Context and Scope	20	-	PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PS2	Temporal Planning	10	PS1	PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL, OT
PS3	Budget and Sustainability	10	PS2	PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PS4	Initial Documentation	10	PS3	PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PM	**Project Management**	**46.5**		
PM1	Product Owner Meetings	30.5	PS2	PO, DT, OS, HW, AZR, MCS, VSC
PM2	Thesis Director Review Meetings	8	-	DT, TD, OS, HW, AZR, VSC, FIB, GM, WEB
PM3	Product Backlog Update	8	PS2	DT, PO, OS, HW, AZR, WEB
FD	**Final Product Delivery**	**56.5**		
FPD1	User Manual Documentation	2	PM5	DT, OS, HW, VSC, WEB
FPD2	Final Thesis Documentation	30.5	-	DT, TD, OS, HW, FIB, GM, WEB, OVL, OT
FPD3	Final Demonstration Video	4	FPD2	DT, OS, HW, WEB
FPD4	Bachelor's Thesis Defense	20	FPD2	DT, TD, OS, HW, FIB, GM, WEB
DT	**Design**	**16**		
DT1	Use Case Diagram	2	PS1	DT, OS, HW, OT, WEB, MCS
DT2	Conceptual Model Diagram	4	PS1	DT, OS, HW, OT, WEB
DT3	Technology Stack Decision	2	PS1	DT, OS, HW, WEB, MCS
DT4	Logical and Physical Architecture Design	4	PS1	DT, OS, HW, WEB, MCS
DT5	UI mockups Design	2	DT1	DT, OS, HW, OT, WEB

(Continued on next page)

(Continued from previous page)

ID	Name	Time (h)	Dependencies	Resources
DT6	Security Design	2	DT3, DT4	DT, OS, HW, WEB, MCS
Dev	**Development**	**86**		
Dev0	Self-learning and study of unfamiliar technologies	8	DT3	DT, OS, HW, WEB
Dev1	Environment Configuration	2	DT2, DT5, DT6	DT, OS, HW, GIT, AZR, VSC, WEB
Dev2	Product Deployment	8	UA4, UA5, UA6	DT, OS, HW, GIT, AZR, VSC, WEB
Dev3	CI/CD Pipeline Implementation	20	BD7	DT, OS, HW, GIT, AZR, VSC, WEB
Dev4	Logging and Monitoring	8	Dev2	DT, OS, HW, GIT, AZR, VSC, WEB
Dev5	Git Repository Documentation	40	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
Frontend and Backend Development		**128**		
ER	**Expense and Report Development**	**64**		
ER1	Expense Uploads	16	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
ER2	Uploaded Expenses Real-time Dashboard	4	ER1, BD6	DT, OS, HW, GIT, AZR, VSC, WEB
ER3	AI-Extracted Expense Data Review and Correction	20	ER2	DT, OS, HW, GIT, AZR, VSC, WEB
ER4	Uploaded Expenses Deletion	4	ER2	DT, OS, HW, GIT, AZR, VSC, WEB
ER5	Expense Reports View, Creation, Edit, Submission, and Deletion	20	ER3, ER4	DT, OS, HW, GIT, AZR, VSC, WEB
UA	**User Access Development**	**64**		
UA1	Company Email User Authentication	8	BD7	DT, OS, HW, GIT, AZR, VSC, WEB
UA2	Role-based User Access	16	UA1	DT, OS, HW, GIT, AZR, VSC, WEB
UA3	Submitted Report Approval and Rejection	8	ER5, UA2	DT, OS, HW, GIT, AZR, VSC, WEB
UA4	Employee Spending Limit Configuration	8	UA2	DT, OS, HW, GIT, AZR, VSC, WEB
UA5	Submitted and Reviewed Expenses Record Visualization	4	UA3	DT, OS, HW, GIT, AZR, VSC, WEB
UA6	Email Notification Service Integration	20	UA3	DT, OS, HW, GIT, AZR, VSC, WEB
BD	**Backend Development**	**120**		
BD1	RESTful API Backend Logic Implementation	20	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
BD2	Azure SQL Database Integration and Manipulation	20	BD1	DT, OS, HW, GIT, AZR, VSC, WEB
BD3	Azure Blob Storage Integration	8	BD2	DT, OS, HW, GIT, AZR, VSC, WEB
BD4	Microsoft Document Intelligence AI Model Integration	20	BD2, BD3	DT, OS, HW, GIT, AZR, VSC, WEB
BD5	Automated AI-driven Data Extraction Pipeline	20	BD4	DT, OS, HW, GIT, AZR, VSC, WEB
BD6	Azure SignalR Real-time Notification Service Integration	16	BD5	DT, OS, HW, GIT, AZR, VSC, WEB
BD7	Microsoft Entra ID User Authentication Integration	16	BD1	DT, OS, HW, GIT, AZR, VSC, WEB
FD	**Frontend Development**	**10**		
FD1	Expenses Search and Filter	8	ER2	DT, OS, HW, GIT, AZR, VSC, WEB
FD2	Reports Search	2	ER5	DT, OS, HW, GIT, AZR, VSC, WEB
Total		**513**		

Table 5.3.1: Summary of project tasks with effort, dependencies, and resources. [Own creation]

5.4 Gantt Diagram

A Gantt diagram is attached at the end of this section. Notice that the execution order aligns with the prioritized requirements defined in Section 3.2.1 based on MoSCoW criteria. Continuous arrows represent Finish-Start (FS) dependencies between tasks, while dashed arrows represent Finish-Finish (FF) dependencies. FS dependencies indicate that successor tasks cannot start until their predecessor tasks are completed. FF dependencies show that successor tasks cannot finish until their predecessor tasks are completed.

Referring to Diagram 5.4.1 , the project timeline spans **122 days** with a **one-week built-in buffer**. Considering the 513 total estimated hours, this equates to approximately **4 hours of work per day**. Consequently, the weekly workload is 28 hours, matching the weekly effort allocated in the Gantt diagram. Hence, the task effort estimations from

Table 5.3.1 are translated into the Gantt diagram using this pattern (e.g., the ER1 task, which costs 20 hours, represents 4 days of work in the diagram).

According to Section 4.1, regular meetings with the Product Owner are arranged every two days. Considering each meeting takes around 30 minutes, a total of 30.5 hours is dedicated to the **PM1** task. These meetings should be represented iteratively throughout the project timeline. However, since the diagram tool does not support this behavior, it is represented as a single concurrent task.

Excluding PM1, several other tasks are represented as concurrent and run concurrently:

- **PM2** remains undetermined due to undefined thesis director meetings, representing iterative appointments throughout the project lifetime.
- **PM3** starts after **PS2** (temporal planning) is completed and overlaps with full application development, as the product backlog requires regular updates based on the project's real-time progress.
- **FPD2** runs throughout the project execution for final delivery consistency.
- **Dev5**, which depends on **Dev1** (beginning once the repository is created), spans most of the project duration to satisfy objective SO5 from Section 3.1, as clean code requires ongoing, extensive documentation.

In addition, **ER1**, **ER3**, **UA2**, and **UA6** tasks have an initial effort estimation of 8 hours. However, the additional time allocated serves as a buffer for likely challenges. **ER1** is the first coding task; **ER3** requires extensive coordination with frontend and backend; **UA6** marks the author's first experience implementing email notifications; **UA2** requires a prior database record of the employee hierarchy. Finally, **Backend Development** tasks similarly incur additional time costs.

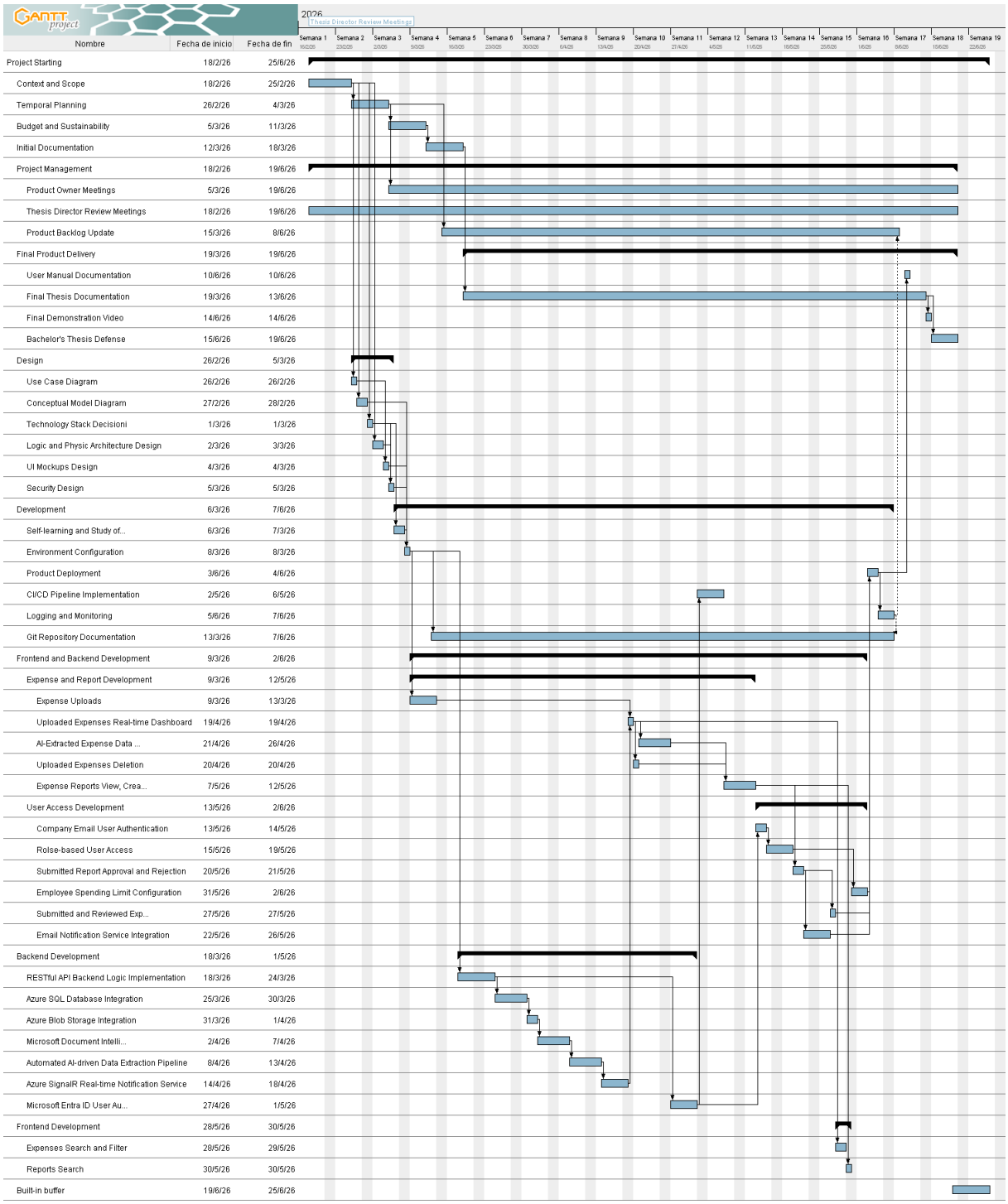

Figure 5.4.1: Initial project Gantt diagram. [Own creation]

5.5 Risk Management

Possible risks affecting project progress are outlined in Section 3.3. With the timeline now established, contingency plans are defined to mitigate potential obstacles. This section presents each identified risk along with its probability, impact, and corresponding mitigation strategy. Moreover, all identified risks, along with the corresponding affected tasks and the contingency resources required for mitigation, are detailed in Table 5.5.1.

Coding Bugs **Probability:** Medium

Impact: High

Coding bugs are inevitable, making thorough testing essential. For this reason, static code analysis is performed using dedicated extensions. During the Dev1 task, the ESLint and SonarQube extensions for Visual Studio Code are installed to automate bug detection from the very beginning of the implementation.

If bugs persist, additional debugging time is allocated, using up to **10 hours** of the buffer time. It is important to fix critical bugs before proceeding with new feature development to prioritize functional application delivery.

However, thanks to the installed extensions, the number of bugs is minimized, reducing the likelihood of significant setbacks. Furthermore, complex tasks already include built-in buffer time, as mentioned in section 5.4, and the reserve hours are available to cover any delays that may arise.

Tight Project Deadlines **Probability:** High

Impact: Medium

Software development projects often struggle with tight deadlines. Due to the author's limited professional experience, deviation between the actual task effort and the initial time estimation is anticipated. However, since this project follows Agile methodology, iterative meetings help to detect and manage overruns at an early stage.

To overcome this challenge, **9 hours** of overhead serve as a lifesaver. This is an intentional allocation, as the project is scheduled to finish several days before the thesis defense period. Therefore, a one-week delay is permitted without affecting the final submission.

In the worst-case scenario, if the project misses the expected deadline, a selection of non-essential features, based on the MoSCoW principles defined in Section 3.2.1, will be completed only if time permits. Since the Gantt diagram's (Section 5.4) tasks follow MoSCoW prioritization, this is a viable solution. This alternative ensures a high-value solution is delivered to FSP but may be adopted solely with the Product Owner's approval.

Lack of Experience in Certain Technologies **Probability:** High

Impact: Medium

Given that several technologies used in application implementation lie beyond the author's expertise, mitigation plans have been established to address this challenge. Time for learning and becoming familiar with these technologies is factored into the project timeline (Task Dev0), thereby minimizing potential setbacks.

Since a prior study cannot fully mitigate this risk, a proposed solution is to set aside up to **8 hours** from the available buffer. If delays are prolonged, assistance from the FSP team serves as a fallback. Seeking timely assistance is an essential professional skill, as it helps prevent unnecessary deadline extensions.

Company's Licenses and Security Permissions **Probability:** Medium

Impact: Low

Another risk involves FSP's security system. Cyberattacks are taken more seriously every day, often requiring developers to request and wait for permission to install resources and extensions. Consequently, these restrictions may limit initial resource availability.

To prevent this blocker, Dev1 has been created to install all the previously identified tools and extensions in advance. However, this prior setup does not remove the probability of installing forthcoming tools.

As a contingency, a reordering of the project backlog can be considered using the Gantt chart (Figure 5.4.1) as a reference. Tasks with no dependencies, such as BD7 and the subsequent UA category tasks, may be prioritized and initialized earlier. Likewise, other non-dependent tasks (e.g., Dev5 task) may also be advanced to maintain project progress.

Risk	Probability	Time (h)	Affected Tasks	Resources
Coding Bugs	15%	10	Dev3, ER1-5, UA1-6, BD1-7, FD1-2	DT, HW
Tight Project Deadlines	30%	9	Dev2-4, ER1-5, UA1-6, BD1-7, FD1-2	DT, HW
Lack of Experience in Certain Technologies	30%	8	Dev2, Dev3, ER1-5, UA1-6, BD1-7,	DT, HW
Company's Licenses and Security Permissions	15%	0	Dev2, BD1-7	DT, HW

Table 5.5.1: Summary of identified risks, their probabilities, affected tasks, and contingency duration and resources. [Own creation]

6 Budget

This section provides an analysis of the project's economic viability. The overall budget is calculated based on the tasks and resources defined in Section 5, and is structured into four categories: Personnel Cost (PC), Generic Cost (GC), Contingency Cost (CC), and Incidental Cost (IC). Each cost is justified and calculated in the following subsections, after which the total project cost is summarized and a management control plan is proposed.

6.1 Personnel Costs

To obtain a realistic personnel cost estimate, the average hourly wage for each personnel role was calculated in advance using market salary data [3], assuming 260 working days per year and an average of 7.5 working hours per day.

Table 6.1.1 specifies the hourly wage calculation for each role, including the Social Security (SS) cost. Each human resource identified in Section 5.2.1 is assigned to a specific role in the table, resulting in 5 different roles:

- **Project Manager (PM)**: Responsible for planning and leading the entire project.
- **Junior Full-Stack Developer (D)**: Responsible for the full application implementation
- **QA Tester (T)**: Responsible for verifying the application functionalities.
- **Software Architect (SA)**: Responsible for designing the application's technical architecture while considering the project requirements and scope.
- **Technical Writer (TW)**: Responsible for drafting clear, accurate project documentation.

Role	Annual Salary (€)	Total including SS (€)	Cost / Hour (€)	Assigned HR
Project Manager (PM)	43,300	56,290	28.87	PO, TD, GT
Full-stack Junior Developer (D)	29,800	38,740	19.87	DT
QA Tester (T)	26,888	34,954.4	17.93	DT
Software Architect (SA)	41,800	54,340	27.87	DT
Technical Writer (TW)	31,300	40,690	20.87	DT

Table 6.1.1: Annual and hourly salaries for different project roles. [Own creation]

Given each personnel member's hourly wage and their corresponding effort hours in each task (extracted from Table 5.3.1), the total personnel cost is calculated as shown in Table 6.1.2. Each task cost results from multiplying both values.

			Hours per role					
ID	**Name**	**Total hours**	PM	D	T	SA	TW	**Cost (€)**
PS	**Project Starting**	**50**	**80**	**0**	**0**	**0**	**0**	**1,433.33**
PS1	Context and Scope	20	20	0	0	0	0	577.33
PS2	Temporal Planning	10	10	0	0	0	0	288.67
PS3	Budget and Sustainability	10	10	0	0	0	0	288.67
PS4	Initial Documentation	10	10	0	0	0	0	288.67
PM	**Project Management**	**46.5**	**38.5**	**46.5**	**0**	**0**	**0**	**2,035.17**
PM1	Product Owner Meetings	30.5	30.5	30.5	0	0	0	1486.37
PM2	Thesis Director Review Meetings	8	8	8	0	0	0	389.87
PM3	Product Backlog Update	8	0	8	0	0	0	158.93
DT	**Design**	**16**	**0**	**0**	**0**	**16**	**0**	**445.87**
DT1	Use Case Diagram	2	0	0	0	2	0	55.73
DT2	Conceptual Model Diagram	4	0	0	0	4	0	111.47
DT3	Technology Stack Decision	2	0	0	0	2	0	55.73
DT4	Logical and Physical Architecture Design	4	0	0	0	4	0	111.47
DT5	UI mockups Design	2	0	0	0	2	0	55.73

(Continued on next page)

(Continued from previous page)

ID	Name	Total hours	PM	D	T	SA	TW	Cost (€)
DT6	Security Design	2	0	0	0	2	0	55.73
FPD	**Final Product Delivery**	**56.5**	**0**	**5**	**0**	**0**	**51.5**	**1173.97**
FPD1	User Manual Documentation	2	0	1	0	0	1	40.73
FPD2	Final Thesis Documentation	30.5	0	0	0	0	30.5	636.43
FPD3	Final Demonstration Video	4	0	4	0	0	0	79.47
FPD4	Bachelor's Thesis Defense	20	0	0	0	0	20	417.33
Dev	**Development**	**86**	**0**	**76**	**10**	**0**	**0**	**1689.12**
Dev0	Self-learning and study of unfamiliar technologies	8	0	8	0	0	0	158.93
Dev1	Environment Configuration	2	0	2	0	0	0	39.73
Dev2	Product Deployment	8	0	8	0	0	0	158.93
Dev3	CI/CD Pipeline Implementation	20	0	10	10	0	0	377.92
Dev4	Logging and Monitoring	8	0	8	0	0	0	158.93
Dev5	Git Repository Documentation	40	0	40	0	0	0	794.67
Frontend and Backend Development		**128**	**0**	**128**	**0**	**0**	**0**	**2,542.93**
ER	**Expense and Report Development**	**64**	**0**	**64**	**0**	**0**	**0**	**1,271.47**
ER1	Expense Uploads	16	0	16	0	0	0	317.87
ER2	Uploaded Expenses Real-time Dashboard	4	0	4	0	0	0	79.47
ER3	AI-Extracted Expense Data Review and Correction	20	0	20	0	0	0	397.33
ER4	Uploaded Expenses Deletion	4	0	4	0	0	0	79.47
ER5	Expense Reports View, Creation, Edit, Submission, and Deletion	20	0	20	0	0	0	397.33
UA	**User Access Development**	**64**	**0**	**64**	**0**	**0**	**0**	**1,271.47**
UA1	Company Email User Authentication	8	0	8	0	0	0	158.93
UA2	Role-based User Access	16	0	16	0	0	0	317.87
UA3	Submitted Report Approval and Rejection	8	0	8	0	0	0	158.93
UA4	Employee Spending Limit Configuration	8	0	8	0	0	0	158.93
UA5	Submitted and Reviewed Expenses Record Visualization	4	0	4	0	0	0	79.47
UA6	Email Notification Service Integration	20	0	20	0	0	0	397.33
BD	**Backend Development**	**120**	**0**	**120**	**0**	**0**	**0**	**2,384.00**
BD1	RESTful API Backend Logic Implementation	20	0	20	0	0	0	397.33
BD2	Azure SQL Database Integration and Manipulation	20	0	20	0	0	0	397.33
BD3	Azure Blob Storage Integration	8	0	8	0	0	0	158.93
BD4	Microsoft Document Intelligence AI Model Integration	20	0	20	0	0	0	397.33
BD5	Automated AI-driven Data Extraction Pipeline	20	0	20	0	0	0	397.33
BD6	Azure SignalR Real-time Notification Service Integration	16	0	16	0	0	0	317.87
BD7	Microsoft Entra ID User Authentication Integration	16	0	16	0	0	0	317.87
FD	**Frontend Development**	**10**	**0**	**10**	**0**	**0**	**0**	**198.67**
FD1	Expenses Search and Filter	8	0	8	0	0	0	158.93
FD2	Reports Search	2	0	2	0	0	0	39.73
Total		**513**	**88.5**	**385.5**	**10**	**16**	**51.5**	**11,913.05**

Table 6.1.2: Personnel costs breakdown by task, role, and total personnel cost. [Own creation]

6.2 General Costs

This category covers the project operational costs and is based on the resources identified in Section 5.2.2. The project duration, calculated in advance in Section 5 (513 hours over 122 days), is used in each cost calculation.

Amortization

The amortization cost corresponds to the proportion of hardware resource usage time relative to their total usable lifespan. Hence, this cost is formulated as:

$$\text{Amortization (€)} = \frac{\text{Resource Price}}{\text{Total Depreciation Hours}} \times \text{Project Usage Hours}$$

The usage hours equal the project execution time, since hardware resources are used throughout the project. Assuming a four-year depreciation period and the same annual working hours from PC (260 working days * 7.5 working hours/day), which is a total of 1950 hours, the following equation is applied for each hardware resource in Table 6.2.1:

$$\text{Amortization (€)} = \text{Resource price} \times \frac{1}{4 \text{ years}} \times \frac{1 \text{ year}}{1950 \text{ working hours}} \times \text{hours used}$$

Hardware Resources	Price (€)	Quantity	Total Price (€)	Usage Time (h)	Amortization (€)
PC Dell 16 Pro [4]	1,193.53	1	1,193.53	513	78.50
Wireless Mouse [5]	11.99	1	16.99	513	1.12
Dell Monitor [6]	171.82	2	343.64	513	22.60
Keyboard [7]	19.99	1	19.99	513	1.31
Total					**102.22**

Table 6.2.1: Amortization cost. [Own creation]

Electric cost

Only hardware resources require electricity and are the only resources considered in this cost. Electricity costs are calculated by multiplying the current electricity cost of **0.1748 €/kWh** [8] by the energy consumed and the resource usage time. Table 6.2.2 details the calculations and the resulting total cost:

Hardware Resources	Power (W)	Usage Time (h)	Energy Consumed (kWh)	Electricity Cost (€)
PC Dell 16 Pro [4]	65.00	513	33.35	5.83
Wireless Mouse [5]	0.05	513	0.03	0.00
Dell Monitor [6]	24.00	513	12.31	2.15
Keyboard [7]	0.10	513	0.05	0.01
Total			**45.73**	**7.99**

Table 6.2.2: Electricity cost. [Own creation]

Internet cost

The basic internet plan costs €21.95 per month [9]; hence, this cost is calculated as follows:

$$\text{Internet cost} = 21.95 \text{ €/month} \times \frac{1 \text{ month}}{30 \text{ days}} \times 122 \text{ days} = \mathbf{€89.26}$$

Office rental cost

The project development always takes place in the FSP office. The FSP office is located in Plaça Francesc Macià, Barcelona, and demands a monthly rental fee of €6,000. Given that there is enough space to accommodate up to 25 employees (which is the number of employees at FSP's Barcelona team, as defined in Section 1.3), this is the cost calculation:

$$\text{Office rental cost} = 6,000 \text{ €/month} \times \frac{1 \text{ month}}{30 \text{ days}} \times \frac{1 \text{ office}}{25 \text{ employees}} \times 122 \text{ days} = \mathbf{€976.00}$$

Software resources cost

Most software resources are open-source and are available for free. The primary expenses in this cost are Azure and Microsoft resources.

Azure charges a pay-as-you-go fee and offers a customized price for each resource usage [10]. Since the application target user group is small, the most basic/free tier for each resource is considered. A monthly fee of €3.96 is estimated for this project, as shown in Table 6.2.3.

Azure resources	Cost (€/month)
SQL DataBase	3.90
Bandwidth	0.01
Blob Storage	0.01
Log Analytics	0.01
Event Grid	0.01
Service Bus	0.01
Functions	0.01
Total	**3.96**

Table 6.2.3: Azure resource cost. [Own creation]

Accordingly, the total Azure resources cost in the project is dervived:

$$\text{Azure resource cost} = 3.96\ €/\text{month} \times \frac{1\ \text{month}}{30\ \text{days}} \times 122\ \text{days} = \mathbf{€24.16}$$

Based on the Microsoft annual fee of €99 [11], the corresponding total cost is the following:

$$\text{Microsoft resource cost} = 99\ €/\text{year} \times \frac{1\ \text{year}}{365\ \text{days}} \times 122\ \text{days} = \mathbf{€33.09}$$

Thus, the total software resources cost equals the sum of both costs:

$$€24.16 + €33.09 = \mathbf{€57.25}$$

Total generic cost

By summing all previously calculated costs, the total generic cost is obtained, as shown in Table 6.2.4.

Concept	Cost (€)
Amortization	102.22
Electricity	7.99
Internet	89.26
Office Rental	976.00
Software	57.25
Generic Cost (GC)	**1,232.72**

Table 6.2.4: Total generic cost. [Own creation]

6.3 Other Costs

Contingencies

No project strategy can eliminate the possibility of encountering obstacles. Therefore, a time buffer (27 hours) has been included to cover potential setbacks. However, simply adding extra time is insufficient, as it also entails additional resource consumption.

For this reason, a portion of the project budget must be allocated for these potential scenarios. Therefore, the contingency cost amounts to **€1,314.57**, which is the 10% of the total general cost calculated so far:

$$PC\,(€) + GC\,(€) = 11{,}913.05 + 1{,}232.72 = \mathbf{13{,}145.77\ €}$$

Incidental Costs

Incidental costs encompass the project risks analysis outlined in Section 5.5. The cost of each risk is calculated by extracting its probability, time cost, and required resources from

Table 5.5.1. A breakdown calculation is presented in Table 6.3.1. Note that the resources required for each risk are identified as Rx (R1, R2, ...).

Incident	Risk (%)	Time (h)	R1	R1 Cost (€/h)	R2	R2 Cost (€/h)	Total Cost (€)	Risk Cost (€)
Coding Bugs	15	10	D	19.87	HW	1.99	200.66	30.10
Tight Project Deadlines	30	9	D	19.87	HW	1.79	180.59	54.18
Lack of Experience in Certain Technologies	30	8	D	19.87	HW	1.59	160.53	48.16
Company's Licenses and Security Permissions	15	0	D	19.87	HW	0.00	0.00	0.00
Total								**132.44**

Table 6.3.1: Incidental cost. [Own creation]

6.4 Total Costs

After summing the total costs from each section, the final project budget is determined, as shown in Table 6.4.1.

Concept	Cost (€)
Personnel Cost (PC)	11,913.05
Generic Cost (GC)	1,232.72
Contingency Cost (CC)	1314.57
Incidental Cost (IC)	132.44
Total Budget	**14,592.78**

Table 6.4.1: Total cost of the project. [Own creation]

6.5 Cost Management Control

Challenges, both expected and unexpected, are always present in any project. Even with a precisely estimated timeline and budget, it is very common for a project to fall short of the initial plan.

At the end of the project, the real execution cost is calculated using the same methodology as the initial budget estimation. Based on these results, the project budget deviation is determined across three areas:

- **Human resources deviation:** To contrast the actual effort with the previous estimated effort of human resources in each task.

$$\text{Human resources deviation} = \sum_{i \in pit} (\text{estimated\_cost\_per\_hour}_i - \text{real\_cost\_per\_hour}_i) \times \text{total\_real\_hours}_i$$

  Where i indexes each individual in pit, the set of all personnel involved in that task.

- **Amortization deviation:** To compare the previously estimated use of hardware resources with the actual usage time of those resources.

$$\text{Amortization deviation} = \sum_{i \in hr} (\text{estimated\_usage\_hours}_i - \text{real\_usage\_hours}_i) \times \text{price\_per\_hour}_i$$

  Where i indexes each resource in hr, the set of all hardware resources used in the project.

- **Total cost deviation:** To determine each task's real cost deviation from the estimated cost. The general cost described in the formula represents the sum of the personnel and generic costs, excluding contingency and incident costs. Hence, the following formula is applied to each task:

$$\text{Total cost deviation} = (\text{estimated\_general\_cost} - \text{real\_general\_cost})$$

Note that the resulting total cost deviation value can be either positive or negative. A positive value indicates an overestimation of the initial budget for that resource or task. In this case, the additional budget can be reallocated to address issues that arise during the project. Conversely, a negative value indicates an underestimation, requiring contingency and incident costs to cover the budget shortfall.

Furthermore, some costs are not included in the deviation calculation. For example, Azure offers a fixed monthly fee; therefore, overusage will not affect the overall cost deviation. The same applies to the remaining omitted costs.

7 System Specifications

This section provides a detailed technical specification of the application's requirements. Building upon the requirements defined in Section 3.2, this section translates them into a complex system design.

7.1 Functional Areas

Before describing how the application's functional requirements are structured within the system architecture, it is necessary to clarify the distinction between an application and a system. An application is a software program that performs specific tasks, whereas a system is a collection of interacting components, among which the application is considered one element.

Within this system, functional requirements are organized into functional areas based on shared purpose or behavior. Each functional area groups related use cases performed by a specific actor. To illustrate the structure of these areas, a dedicated UML Use Case diagram is provided for each one.

In this context, actors represent the application's end users. Two types of actors are identified: Users and Administrators (labeled as "Admin" in the diagrams). The user category includes both ordinary employees and administrators. Administrators, in addition to general user capabilities, possess further permissions to review and manage the expense reports submitted by employees under their supervision.

To illustrate how these use cases are implemented, this section provides a detailed technical description for three critical and representative use cases: user authentication, expense registration via file upload, and report approval. These specific examples have been carefully selected because they best demonstrate the system's core functionalities, security mechanisms, asynchronous processing architecture, and multi-role interactions.

The complete and exhaustive technical description to all use cases diagrammed in this section, including their respective actors, triggers, pre/postconditions, and detailed event flows, is available for comprehensive review in Appendix A: Detailed Use Case Specifications.

7.1.1 Account Management

This functional area covers all use cases related to user accounts. Since the application's core functionality is expense management, user login is the only required action. The resulting use case diagram is presented in Figure 7.1.1.

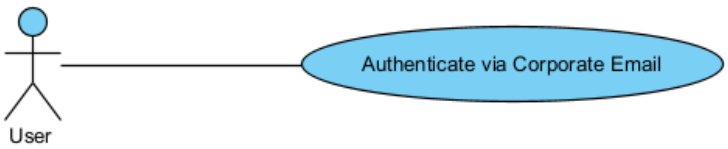

Figure 7.1.1: Account management use case diagram. [Own creation]

Use Case: Authenticate via Corporate Email

Primary Actor: User

Trigger: The user attempts to access the application URL.

Preconditions: None.

Postconditions: The user is authenticated and redirected to the home page with role-specific access.

Main Success Scenario:

1. The user navigates to the web application and initiates the login process.
2. The system displays the login interface.
3. The user enters their account credentials.
4. The system authenticates the credentials successfully.
5. The system verifies that the email belongs to the authorized corporate domain.
6. The system assigns the corresponding role to the user and redirects them to the home page, displaying the available functionalities according to the user's assigned role.

Extensions:

2a. Server Connection Error:

2a1. The system cannot connect to the server and cannot display the login interface.

4a. Authentication Failed (Invalid Credentials):

4a1. The system displays an invalid credentials error message ("Invalid email" or "Invalid password").

4a2. The user remains on the login page and returns to step 3 until valid credentials are provided.

5a. Unauthorized Domain:

5a1. The system detects that the authenticated account does not belong to the authorized corporate domain.

5a2. The system denies access and notifies the user.

7.1.2 Expense Management

This functional area encompasses all features related to expense management. It includes functionalities such as uploading, editing, and deleting expenses, as well as obtaining AI-extracted data and assigning expenses to either a new or an existing report. All these actions are accessible within the Expenses list page view, as shown in Figure 7.1.2.

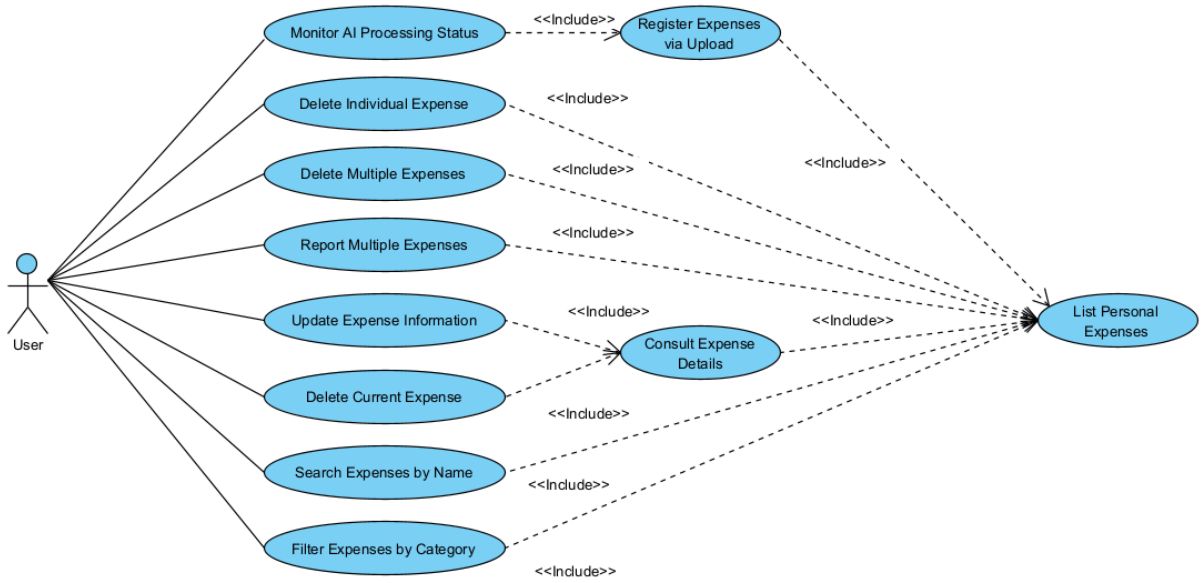

Figure 7.1.2: Expense management use case diagram. [Own creation]

Use Case: Register Expenses via Upload

Primary Actor: User

Trigger: The user uploads or drops one or more receipts.

Preconditions: The user is successfully authenticated and viewing the Expenses page.

Postconditions: The new expenses are successfully registered and added to the AI processing queue.

Main Success Scenario:

1. The user uploads or drops one or more receipts.
2. The system registers new expense records, associates them with the authenticated user, and queues them for automated data extraction.
3. The system updates the expense list to include the recently uploaded items.

Extensions:

2a. Invalid File Format:

2a1. The system detects that the uploaded file format is not supported (non-image or non-PDF).

2a2. The system rejects the upload and keeps the user on the current page without registering any expense.

2b. Database Connection Error:

2b1. The system is unable to register the expense records due to a database connection failure.

2b2. The system notifies the user of the error and no expenses are registered.

7.1.3 Report Management

This functional area groups all the operations related to report management. In this application, a report serves as a container for a set of expenses prepared for submission. Once a report has been reviewed and finalized, it is submitted to the appropriate reviewer.

Core functionalities within this area include creating, modifying, deleting, and submitting reports. Additionally, expenses can be easily attached or detached to a report, and any assigned expenses can be updated while preserving data consistency. The use case diagram for this functional area is shown in Figure 7.1.3.

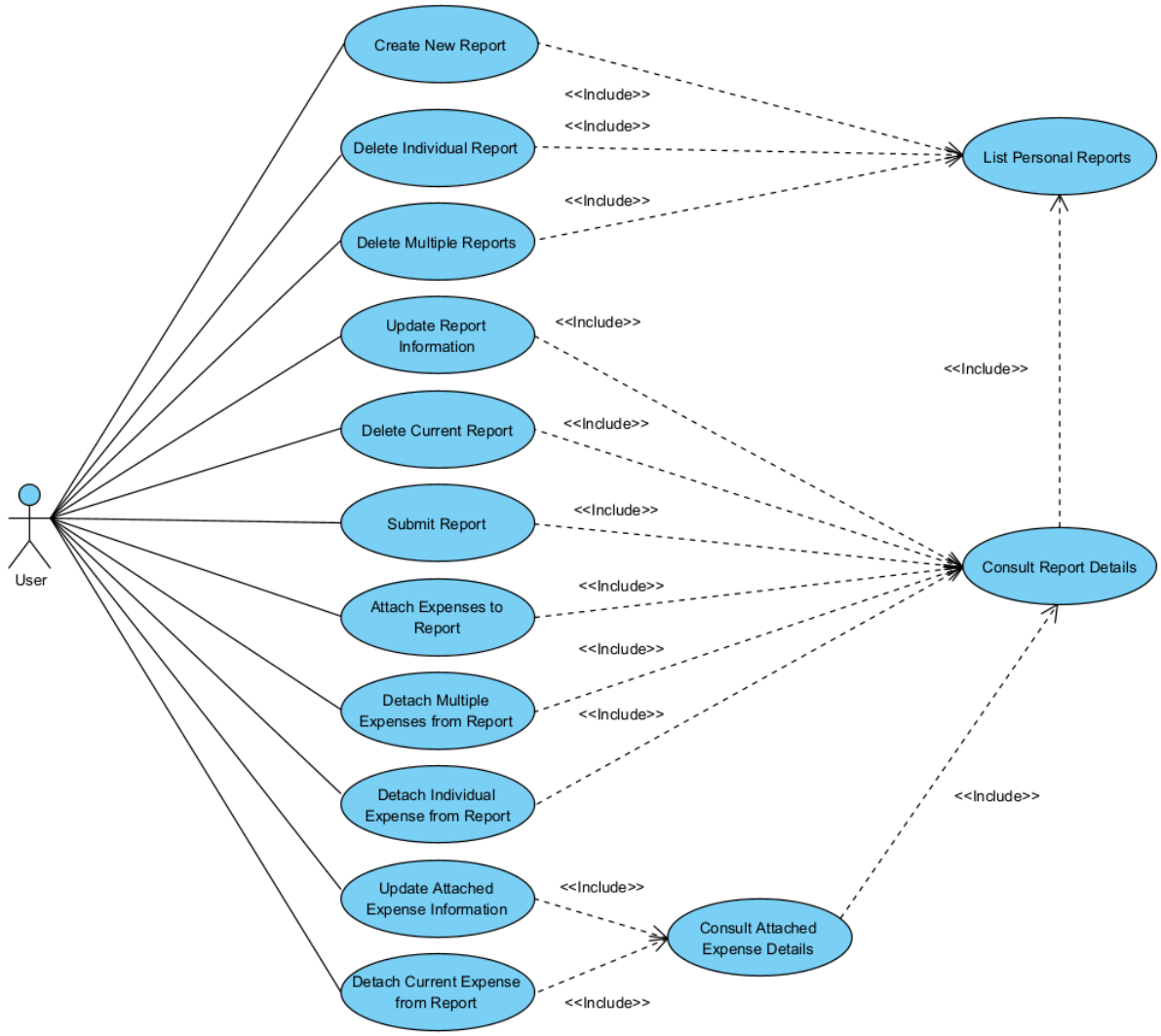

Figure 7.1.3: Report management use case diagram. [Own creation]

7.1.4 Submitted Report Management

This functional area allows users to monitor the real-time status of their reports. It enables them to determine whether submitted expenses have been reviewed and approved or rejected by a supervisor. As submitted reports are no longer editable, user interaction is limited to viewing report details and the associated expenses. This functional area is accessible to all users. Figure 7.1.4 depicts the corresponding use case diagram.

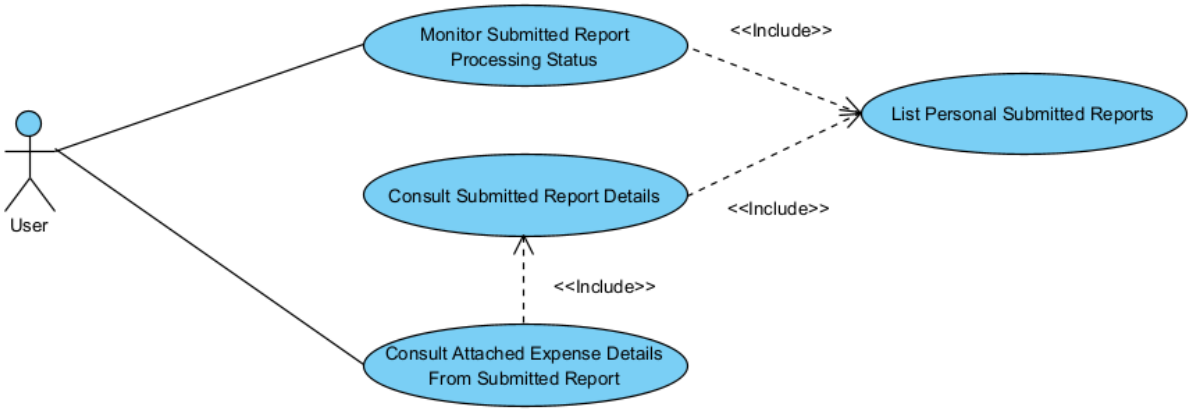

Figure 7.1.4: Submitted report management use case diagram. [Own creation]

7.1.5 Pending Review Report Management

Unlike the previously described functional areas, this area is accessible only to administrative users, who are responsible for supervising ordinary employees. Within this area, administrators can view all pending reports and their associated expenses, and decide whether to approve or reject them. All use cases are represented in the diagram shown in Figure 7.1.5.

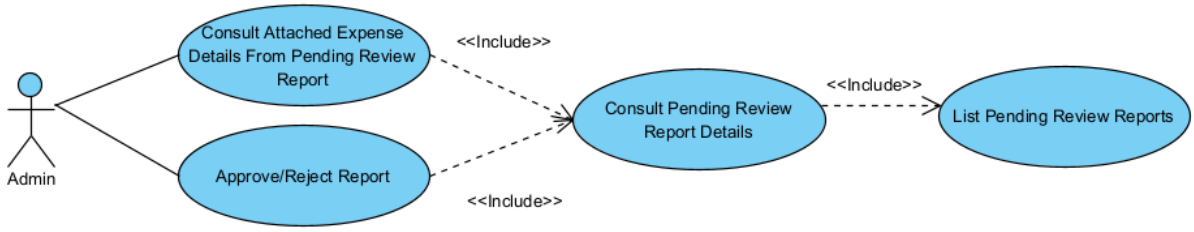

Figure 7.1.5: Pending review report management use case diagram. [Own creation]

Use Case: Approve/Reject Report

Primary Actor: Administrator

Trigger: The administrator approves or rejects a report pending review.

Preconditions: The administrator is authenticated, and the system displays the report metadata along with a list of associated expenses.

Postconditions: The system updates the report status according to the administrator's decision.

Main Success Scenario:

1. The administrator approves or rejects the currently displayed report.
2. The system updates the report status accordingly and notifies the report owner of the review.

Extensions:

2a. Report Review Error:

2a1. An error occurs during the report review due to a database connection error or an internal error.

2a2. The system notifies the administrator of the error and keeps them on the current report details view.

7.1.6 Reviewed Report Management

Finally, this functional area provides a record of all reports reviewed by the user and is therefore restricted to administrative users. Unlike the previous area, it only allows administrators to view reviewed reports and examine the details of each report and its associated expenses. Figure 7.1.6 presents the use case diagram of this functional area.

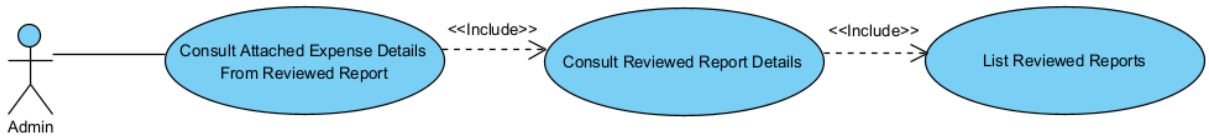

Figure 7.1.6: Reviewed report management use case diagram. [Own creation]

7.2 Conceptual Data Model

This section introduces the abstract structure of how system information is conceptually organized. A conceptual data model translates business logic and system requirements into logical, technology-independent entities, enabling seamless integration with any database or platform [12]. Accordingly, this section presents a conceptual representation of the application's data, including its validation rules and entity specifications.

7.2.1 Conceptual Data Schema

The system's conceptual data schema is presented in Figure 7.2.1. It includes the main domain entities (classes), their essential properties (attributes), and the relationships among them (associations, aggregations, compositions, and generalizations).

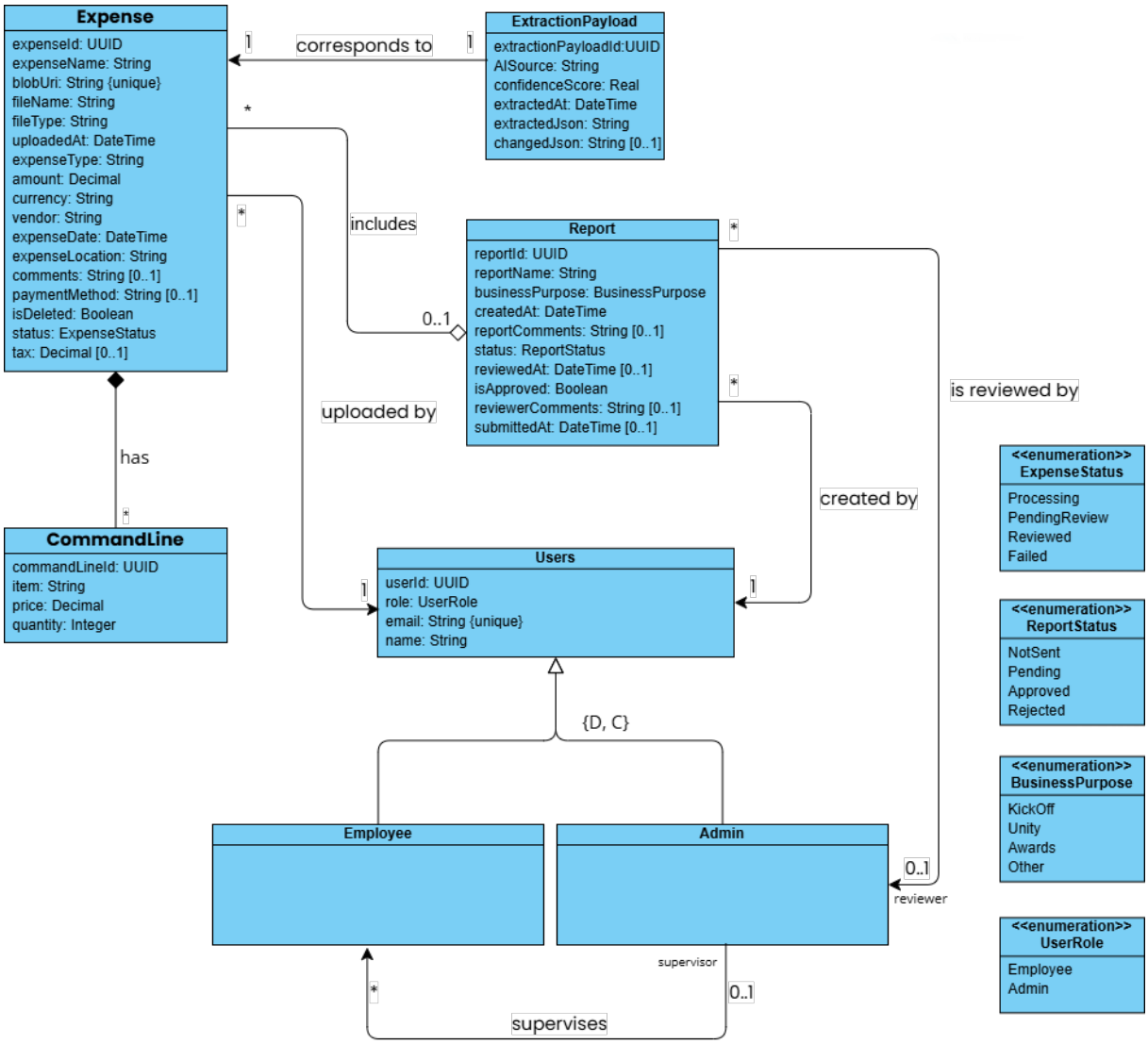

Figure 7.2.1: Conceptual data schema. [Own creation]

7.2.2 Integrity Constraints

To ensure a valid and consistent data conceptual schema, the following integrity constraints must be applied:

Primary keys: (User, userId), (Expense, expenseId), (CommandLine, commandLineId), (ExtractionPayload, extractionPayloadId), (Report, reportId).

RT2: A user is classified as either an Employee or an Admin, depending on their assigned role.

RT3: An administrator can only review reports submitted by employees under their supervision.

RT4: The expense amount and tax must be greater than 0.

RT5: The expense currency must be a valid international currency.

RT6: The user's email must be a valid email address.

RT7: The expense date must be earlier than the current date.

RT8: The expenseDate timestamp must be earlier than the uploadedAt timestamp of the same expense.

RT9: The expense payment method must be either "cash" or "credit card".

RT10: The command line price must be greater than or equal to 0.

RT11: The command line quantity must be greater than or equal to 1.

RT12: The extraction payload confidence score must be between 0 and 1.

RT13: The report submittedAt timestamp must not be null if the report status is "Pending", "Approved", or "Rejected".

RT14: The report reviewedAt timestamp must not be null if the report status is "Approved" or "Rejected".

RT15: The report's createdAt timestamp must be earlier than the submittedAt timestamp (if the latter is not null).

RT16: The report's submittedAt timestamp must be earlier than the reviewedAt timestamp (if both are not null).

RT17: An expense can be attached to a report only if the report is created by the same user who uploaded the expense.

RT18: A report status may transition from "NotSent" to "Submitted", and from "Submitted" to "Approved" or "Rejected".

RT19: A report without attached expenses is not allowed to transition to any status other than "NotSent".

7.2.3 Class Descriptions

Each class in the conceptual data schema is described with a brief overview and a short description of its attributes.

User

This class represents the application's end users. Any individual who can log in to the application is considered a user. Each user is classified as either an Employee or an Admin. Consequently, users cannot review their own reports; administrators are only permitted to review reports submitted by employees under their supervision.

- **userId:** Unique identifier of the user.
  - **type:** UUID.
  - **constraints:** mandatory, unique, non-null.
- **role:** Assigned role of the user.
  - **type:** UserRole Enum (Employee, Admin).
  - **constraints:** mandatory, non-null.
- **email:** Corporate email address of the user, used for authentication.
  - **type:** String.
  - **constraints:** mandatory, unique, must follow a valid email format.
- **name:** Full name of the user.
  - **type:** String.
  - **constraints:** mandatory, non-null

Expense

This class represents an expense submitted by a user. Each expense must be associated with the user who uploaded it.

- **expenseId:** Unique identifier of the expense.
  - **type:** UUID.
  - **constraints:** mandatory, unique, non-null.
- **expenseName:** Name assigned to the expense by the user.
  - **type:** String.
  - **constraints:** mandatory, non-null.
- **blobUri:** URI of the stored expense file.
  - **type:** String.
  - **constraints:** mandatory, unique, non-null.
- **fileName:** Name of the uploaded file.
  - **type:** String.
  - **constraints:** mandatory, non-null.
- **fileType:** Type of the uploaded file (e.g., PDF, JPEG).
  - **type:** String.
  - **constraints:** mandatory, non-null.
- **uploadedAt:** Timestamp indicating when the expense was uploaded.
  - **type:** DateTime.
  - **constraints:** mandatory, non-null.
- **expenseType:** Category of the expense (e.g., travel, meals).
  - **type:** String.
  - **constraints:** mandatory, non-null.
- **amount:** Monetary amount of the expense.

    - **type:** Decimal.
    - **constraints:** mandatory, non-null, must be ≥ 0.
- **currency:** Currency of the expense amount (e.g., EUR, USD).
    - **type:** String.
    - **constraints:** mandatory, non-null, must be a valid international currency code.
- **vendor:** Name of the vendor or service provider.
    - **type:** String.
    - **constraints:** mandatory, non-null.
- **expenseDate:** Date when the expense was incurred.
    - **type:** DateTime.
    - **constraints:** mandatory, non-null, must be $\leq$ current datetime and uploaded datetime.
- **expenseLocation:** Location where the expense was incurred.
    - **type:** String.
    - **constraints:** mandatory, non-null.
- **comments:** Additional notes or description provided by the user.
    - **type:** String.
    - **constraints:** optional.
- **paymentMethod:** Method used for the payment (e.g., cash, credit card).
    - **type:** String.
    - **constraints:** optional.
- **isDeleted:** Indicates whether the expense has been logically deleted.
    - **type:** Boolean.
    - **constraints:** mandatory, default = false.
- **status:** Current AI extraction process status of the expense.
    - **type:** ExpenseStatus Enum (Processing, PendingReview, Reviewed, Failed).
    - **constraints:** mandatory, non-null.
- **tax:** Tax amount associated with the expense.
    - **type:** Decimal.
    - **constraints:** optional, must be ≥ 0.

CommandLine

This class represents an individual line item within an expense, typically corresponding to a specific product or service listed on a receipt. If the associated expense is deleted, the command line must also be deleted.

- **commandLineId:** Unique identifier of the command line.
    - **type:** UUID.
    - **constraints:** mandatory, unique, non-null.
- **item:** Description or name of the purchased item or service.
    - **type:** String.
    - **constraints:** mandatory, non-null.
- **quantity:** Number of units of the item.
    - **type:** Integer.
    - **constraints:** mandatory, non-null, must be ≥ 1.

- **price:** Monetary value of the item.
  - **type:** Decimal.
  - **constraints:** mandatory, non-null, must be ≥ 0.

ExtractionPayload

This class represents the data extracted from an expense using an AI service. An extraction Payload must always correspond to a single, unique expense.

- **extractionPayloadId:** Unique identifier of the extraction payload.
  - **type:** UUID.
  - **constraints:** mandatory, unique, non-null.
- **aiSource:** Identifier of the AI service or model used for extraction.
  - **type:** String.
  - **constraints:** mandatory, non-null.
- **confidenceScore:** Confidence level assigned by the AI to the extracted data.
  - **type:** Real.
  - **constraints:** mandatory, non-null, must be between 0 and 1.
- **extractedAt:** Timestamp indicating when the data was extracted.
  - **type:** DateTime.
  - **constraints:** mandatory, non-null.
- **extractedJson:** Raw data extracted by the AI in JSON format.
  - **type:** String.
  - **constraints:** mandatory, non-null.
- **changedJson:** Modified version of the extracted data after user adjustments.
  - **type:** String.
  - **constraints:** optional.

Report

This class represents a collection of expenses grouped together for submission and review. A report must always belong to the user who created it, and it can only be reviewed by that user's supervisor.

- **reportId:** Unique identifier of the report.
  - **type:** UUID.
  - **constraints:** mandatory, unique, non-null.
- **reportName:** Name assigned to the report.
  - **type:** String.
  - **constraints:** mandatory, non-null.
- **businessPurpose:** Business purpose of the report.
  - **type:** BusinessPurpose Enum (KickOff, Unity, Awards, Other).
  - **constraints:** mandatory, non-null.
- **createdAt:** Timestamp indicating when the report was created.
  - **type:** DateTime.
  - **constraints:** mandatory, non-null.
- **reportComments:** Additional comments or notes provided by the user.
  - **type:** String.

    - **constraints:** optional.
- **status:** Current status of the report.
    - **type:** ReportStatus Enum (NotSent, Pending, Approved, Rejected).
    - **constraints:** mandatory, non-null.
- **submittedAt:** Timestamp indicating when the report was submitted.
    - **type:** DateTime.
    - **constraints:** optional (null if not yet submitted).
- **reviewedAt:** Timestamp indicating when the report was reviewed.
    - **type:** DateTime.
    - **constraints:** optional (null if not yet reviewed).
- **isApproved:** Indicates whether the report has been approved.
    - **type:** Boolean.
    - **constraints:** mandatory, default = false.
- **reviewerComments:** Comments provided by the reviewer during the review process.
    - **type:** String.
    - **constraints:** optional.

8 Alternative Analysis

During the application definition phase, a set of possible alternatives was evaluated. Next, an analysis of these options, including the selected alternative and its justification, is presented.

8.1 Platform Selection

Currently, there are three main application platforms: web applications, native mobile applications, and hybrid mobile applications. Web applications are accessible from any device through a browser by entering the URL. Native mobile applications are installed on a mobile phone and are specific to a single operating system, either Android or iOS. Hybrid mobile applications combine the characteristics of both web and native platforms.

A native application requires deep specialization for both operating systems, demanding higher development costs. Its adoption may also be slower, as users must install the application before using it. While mobile phones are convenient for uploading the expenses via the camera, performing the subsequent AI-extraction review and report creation is much more comfortable on a larger screen.

Conversely, a web application offers more practical advantages. A web application can be accessed from any device without installation. Development is much faster, a single codebase implementation is more than enough, because it is valid for all users. Ongoing maintenance and updates are instant after simply publishing the new version on the server.

Given the primary use case of creating expense reports at a desk and the need for universal accessibility without installation barriers, the **web platform** is selected. In terms of cost and development effort, this one is also the most suitable option.

8.2 Data Extraction Selection

After uploading the expenses, their relevant information can be extracted in several ways: manually entering the data, using OCR technology, or leveraging a pre-trained AI service. FSP's traditional expense management approach relies on manual data entry, which sometimes can be frustrating, error-prone, and time-consuming (as discussed in Section 1.3).

OCR is a technology that captures text from images and converts it into machine-readable text. However, this technology introduces certain drawbacks. It is sensitive to low-quality images, increasing the probability of errors. It also follows strict rules; even minor differences in keyword capitalization or data formatting may produce inconsistencies. Consequently, the solution must account for countless receipt formats, which raises the need for regular maintenance.

Unlike OCR, AI models not only read text but also understand context across a wide variety of formats without relying on rigid rules. With this capability, they can also recognize structured data such as line items and their corresponding amounts, going well beyond what conventional text-scanning technologies can achieve.

While traditional OCR with extensive rules is a possibility, it is overly brittle and requires intensive maintenance to handle the wide variety of real-world receipts. An **AI model** is a more effective choice, since it provides higher accuracy and significantly lower development and maintenance overhead. In addition, it fulfills FR3, one of the core project requirements outlined in Section 3.2.1.

8.3 Architectural Pattern Selection

The automated data extraction pipeline, defined in the BD5 task in Section 5.1, can be implemented using two distinct architectural patterns. The first is a synchronous monolithic architecture, in which the entire process, from the initial upload to the AI processing of the expense, is handled through a single API call. The second is an asynchronous microservices-based architecture, where each service is responsible for a specific function in the workflow.

An expense upload triggers the `POST /api/expenses` call. A simple `POST` request is quick and lightweight, whereas the AI-powered extraction introduces considerable processing delay. Consequently, the two architectural patterns exhibit opposite behaviours.

In the case of synchronous monolithic architecture, the `POST` call cannot complete until AI extraction is finished, so the server response time is considerably prolonged. This is particularly harmful in bulk-upload scenarios, where expenses are processed sequentially, degrading user experience and reducing server availability. At the same time, reliability is low: any error in the end-to-end workflow causes the entire request to fail, requiring users to retry the upload.

In an asynchronous microservices-based architecture, the `POST` call is fully decoupled from the AI extraction process. Single uploads complete instantly, and bulk uploads are handled concurrently, preventing server overload. After a successful expense upload, the expense is routed to a queue and processed by the AI when resources are available. In addition, a more robust error-handling strategy is applied: if the AI model fails to extract the data, the expense is preserved and sent back to the queue for a later retry.

Due to its negative impact on user experience and scalability, the synchronous monolithic approach is discarded, biasing the design toward the alternative solution. The **asynchronous architecture with microservices** delivers immediate feedback to the user, prevents API bottlenecks, and results in a more resilient, scalable system. By making this choice, the non-functional requirements NFR1, NFR3, and NFR4 (Section 3.2.2) are also fulfilled.

9 Technical Design

Building upon the decisions justified in the previous Alternative Selection section, this section provides a comprehensive overview of the application architecture. It aims to present a clear understanding of the system's technical structure before delving into the specific implementation details in the subsequent section.

9.1 Physical Architecture

A physical architecture diagram shows all the hardware and software resources used by the application. This visual representation provides a clearer understanding of how these components communicate across the network. The physical architecture diagram is shown in Figure 9.1.1.

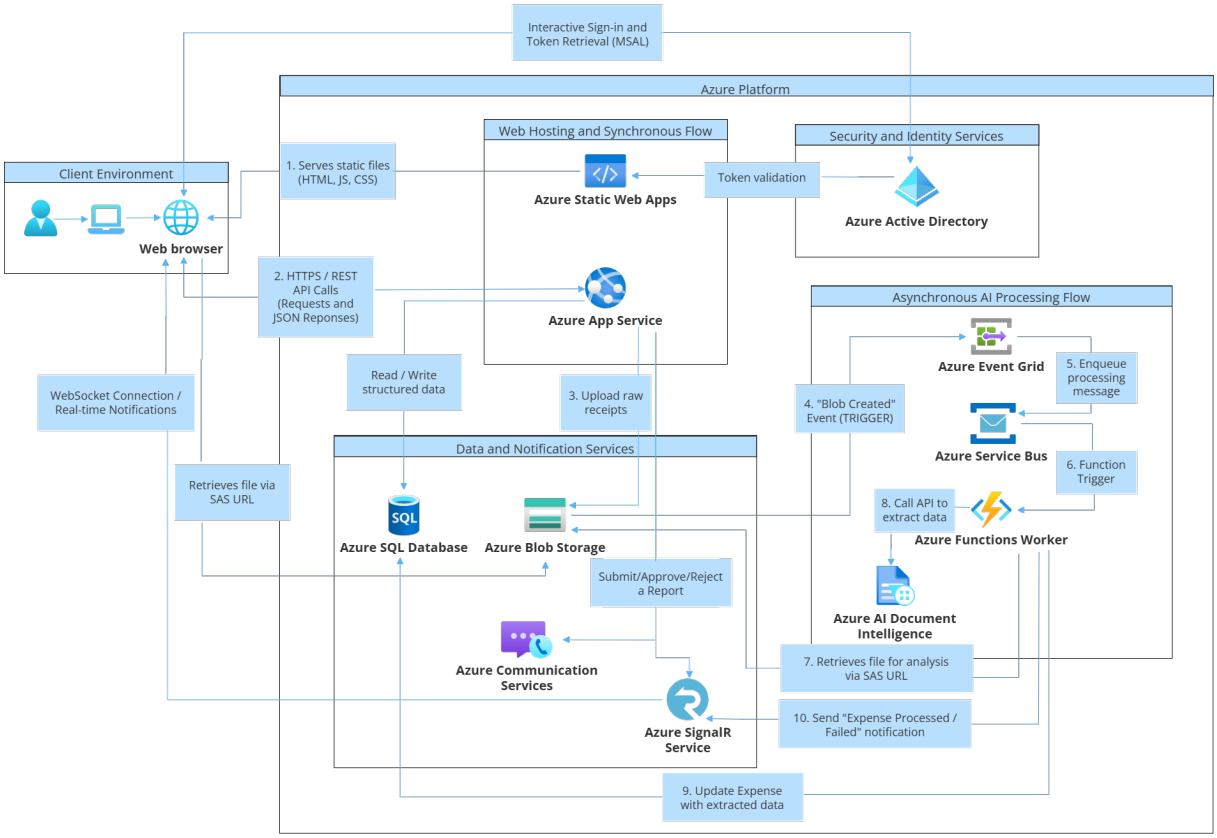

Figure 9.1.1: Physical architecture diagram. [Own creation]

The application's physical architecture follows a modern, cloud-native approach, leveraging Microsoft Azure Platform-as-a-Service (PaaS). It is structured around two main operational flows: the Synchronous Flow, which handles immediate user requests, and the Asynchronous AI Processing Flow, which executes complex tasks in the background. This design ensures scalability while maintaining a highly responsive user experience.

The Client Environment and Synchronous Flow

As defined in sub-objective SO2 in Section 3.1, this project aims to develop a full-stack web application; consequently, the application comprises a frontend and a backend. The frontend is implemented as a React application and deployed using **Azure Static Web Apps**, while the backend is a .NET solution hosted on **Azure App Service**.

User interaction begins in the **Client Environment**, where the **web browser** executes the React application. Unlike traditional frontend architectures, where the React application communicates directly with the backend API, Azure Static Web Apps does not act as an intermediary in this interaction. Instead, the browser communicates directly with the backend service.

The interaction flow is as follows:

1. The user opens a **web browser** and navigates to the application URL.
2. **Azure Static Web Apps** delivers the static assets (HTML, CSS, and JavaScript) with low latency via a Content Delivery Network (CDN).
3. The application is executed locally in the user's browser, not on Azure Static Web Apps.
4. The browser is responsible for making REST API requests to the backend.

In this setup, **Azure Static Web Apps** serves only as a static content host. The benefit of this approach is that fast global access is accomplished, as assets are distributed through a CDN, reducing load times for users regardless of location.

Consequently, as illustrated in Figure 9.1.1, the browser communicates directly with the backend hosted on **Azure App Service**. This communication follows a standard HTTPS request-response pattern, represented as a bidirectional interaction (arrow). Each time the browser sends a request (e.g., upload a new expense or retrieve expense data), the API returns a JSON-formatted response.

Azure App Service executes the .NET API and synchronously handles business logic, data validation, authentication, and authorization. Additionally, Azure App Service is also responsible for interacting with **data persistence and notification services**.

Data Persistence

The application uses two data storage services to support distinct storage approaches. **Azure SQL Database** is designed to persistently store structured, relational data, such as user information, expense metadata, and report details. In contrast, **Azure Blob Storage** is optimized for storing and retrieving large unstructured objects, specifically raw receipt files (binary files such as images and PDFs).

When the application needs to access previously uploaded receipt files, the browser interacts with **Azure Blob Storage**. A critical architectural pattern used here is the "Valet Key Pattern": each time the client requests a stored receipt, the backend API generates a temporary secure link, known as a Shared Access Signature (SAS) URL. Using this URL, the browser can securely download the file directly from **Azure Blob Storage**.

The Asynchronous AI Processing Flow

The Asynchronous Flow is an event-based solution designed to perform the heavy, complex task of extracting expense data using an AI model. At the same time, it avoids introducing significant delays that impact the user experience, preventing the user from having to wait until the expense is processed. This is achieved through an event-driven architecture, where the entire process is triggered whenever a new file is uploaded to the **Blob Storage**.

Once a file is uploaded, **Azure Event Grid**, a highly scalable service, immediately detects the "Blob Created" event and forwards the expense metadata. Instead of sending the expense directly to the AI for processing, it is queued to **Azure Service Bus**, a reliable container that acts as a durable buffer. In this way, if the processing service is temporarily unavailable, the request is not lost and will be handled once the service is recovered.

In response, the **Azure Function** is triggered when a new message is available in the Service Bus queue. The Function then orchestrates the analysis process: it first generates a temporary, secure access key (a SAS URL) for the newly uploaded receipt file stored in **Azure Blob Storage**. It then calls the **Azure AI Document Intelligence** service, providing this SAS URL.

Using the SAS URL, the Azure AI service directly and securely retrieves the receipt file from Blob Storage for processing and returns the extracted data in JSON format. The Azure Function receives the extracted data from the AI service and updates the corresponding expense record accordingly in Azure SQL Database.

After successfully updating the database, the Function's final responsibility is to notify the user in real time. It sends a message to **Azure SignalR Service**, which instantly pushes a notification to the user's web browser, allowing the UI to update automatically and reflect that the expense has been processed. This event-driven design fully decouples the initial user action from the complex background workflow.

Real-time Notifications

Receiving real-time feedback is a key positive aspect of the user experience. Once the AI has finished processing the receipt, a message is sent to the **Azure SignalR Service**, which maintains a persistent WebSocket connection with the user's browser.

Whenever the SinalR receives a "Expense Processed" message, it immediately pushes this notification to the client application. This approach allows the user to visualize a real-time update of the expense processing status without needing a manual page refresh. Moreover, **Azure Communication Services** dispatches email notifications to the interested user whenever a report is submitted or reviewed.

Security and Identity Management

The application's security is built upon **Azure Active Directory (Azure AD)**. Azure AD is an identity management system that acts as a central **Identity Provider (IdP)**. It not only handles user authentication but also serves as the authority for token issuance. The architecture implements a modern OAuth 2.0 and OpenID Connect flow.

OAuth 2.0 is used for authorization, allowing applications to access resources on behalf of a user without knowing their password, while OpenID Connect is used for authentication and provides information about the user's identity.

The client side handles user interaction for signing in to the application and uses the Microsoft Authentication Library (MSAL) for orchestrating login popups and redirects against Azure AD. After the user successfully logs in, MSAL obtains a JWT access token and stores it securely in session storage. For this reason, the diagram illustrates a two-way communication between the **browser** and **Azure AD** during this process.

All further communication with the backend, including **REST API** calls and real-time **SignalR** connections, is secured using this JWT token. On the client side, the token is automatically added to the Authorization header as a Bearer token to communicate with the backend API. The backend API, hosted on **Azure App Service**, is protected using Microsoft.Identity.Web library. This middleware intercepts each incoming request, validates the JWT signature, claims against Azure AD's OpenID Connect metadata endpoints, and establishes the user identity.

This validation step is shown in the diagram as a one-way communication from the **App Service** to **Azure AD**. After successful validation, controllers and SignalR hubs enforce access control using the `[Authorize]` attribute. The application also maps the token's object identifier claim to a specific user, allowing it to perform authorized actions.

Note that in Task BD7, the resource is referred to by its former name, "Microsoft Entra ID". Since 2023, this service has been renamed to "Azure Active Directory (Azure AD)". This change does not affect the functionality or implementation of the resource, which continues to serve as the application's identity management system.

9.2 Logical Architecture

The logical architecture is a high-level view of the application. It describes the technical organization of the code into layers and the software-level data flow across the entire infrastructure.

The application's logical architecture is designed according to **Clean Architecture** principles. Its core objective is to separate the responsibilities of each layer. The result is a clean code structure in which business logic and domain rules are independent of external service implementations. As a consequence, the system is highly maintainable, testable, and extensible. Figure 9.2.1 depicts the application's logical architecture.

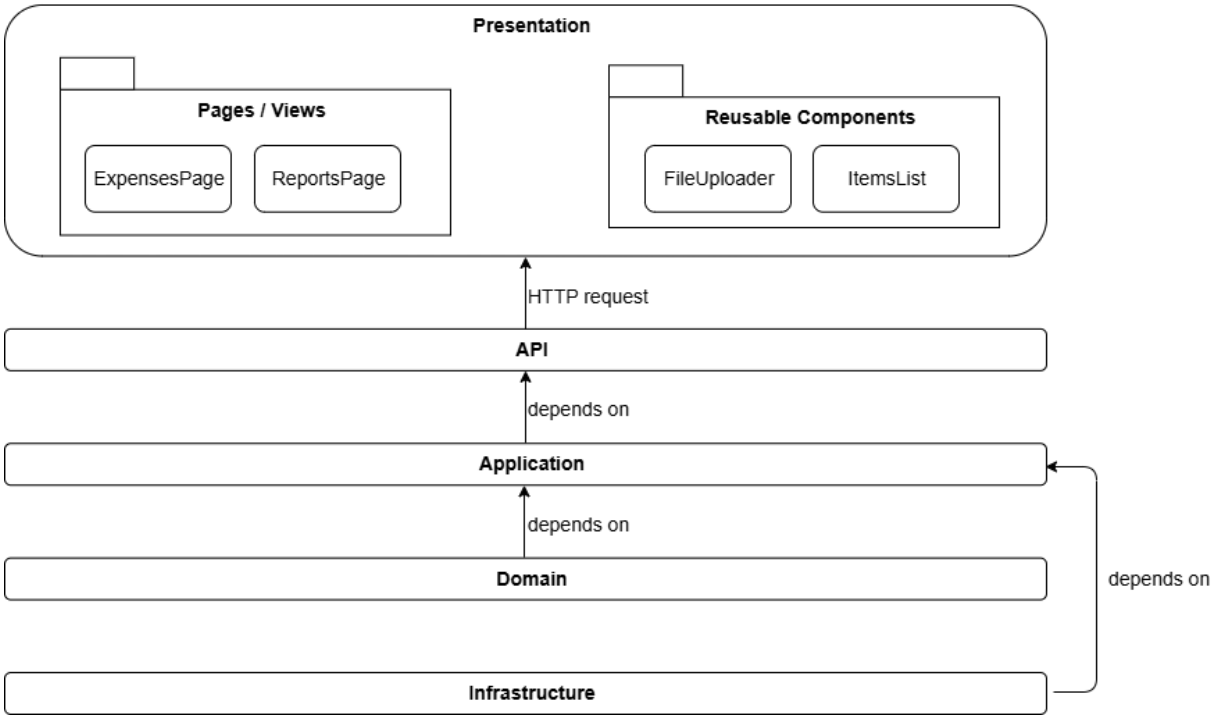

Figure 9.2.1: Logical architecture diagram. [Own creation]

9.2.1 Logical Layers

The application's logical architecture is visually divided into the following main layers:

Presentation layer

The Presentation layer corresponds to the frontend component and is responsible for handling the user interface, user experience, and API communication. Each navigation state or view in the application corresponds to a distinct page, and each reusable component can be used across multiple pages.

The user interface is built using React with TypeScript. This strategic choice allows developers to leverage React's component-based architecture, enabling the creation of reusable UI blocks (components) that compose larger screens (pages). Although this approach requires a more complex initial implementation effort, it speeds up development and improves code maintainability and extensibility over time. TypeScript also enforces contract definitions, ensuring consistency across the frontend data flow.

API layer

The API layer exposes the application's use cases to external parties via RESTful HTTP endpoints. Different controllers handle distinct functional areas (set of use cases), with the main controllers dedicated to Expenses and Reports. Controllers receive HTTP requests, validate them, translate them into commands or queries for the Application layer, and return an HTTP response. This layer, along with the subsequent ones, constitutes the backend component and uses the ASP.NET Core platform.

Application Layer

The Application layer contains the application logic and is responsible for handling all use cases. This layer orchestrates domain entities to execute business rules and interacts with repository abstractions to retrieve and persist data. The concrete implementations of these repositories are provided by the Infrastructure layer.

Additionally, this layer defines Data Transfer Objects (DTOs) and interfaces. DTOs standardize data exchange between components, while interfaces are abstractions that specify how external dependencies are accessed without exposing implementation details.

Domain layer

The Domain layer is the core of the application and represents the business model. It encapsulates entities (e.g., `Expense`, `User`), value objects, and the most critical and stable business rules. This is a pure layer, with no dependencies on other layers, and it remains independent of how data is stored or presented to the user. This independence ensures a stable and long-lasting business model.

Infrastructure layer

While the Application layer depends on abstractions to perform data access and communicate with external services, the Infrastructure layer provides the corresponding concrete implementations. It is responsible for handling communication with external systems and services. `ExpenseRepository`, `BlobStorageService`, and `DocumentIntelligenceService` are some of the implementations contained in this layer.

This layer is expected to evolve as external technologies and infrastructure requirements change. For example, switching from Blob Storage to another storage provider only requires changes within this layer, without affecting the upper layers, which depend on abstractions rather than concrete implementations.

Another key advantage of using the .NET platform is availability of Entity Framework Core (EF Core) and the built-in Dependency Injection (DI) container. EF Core enables higher-level interaction with the database, allowing data manipulation, relationship management, and migrations through clear, easy-to-understand C# code. DI is explained in more detail in the next section.

9.2.2 Guiding Principles and Design Patterns

The structure described above is not arbitrary; it is intentionally crafted based on a set of industry-standard principles and patterns that ensure high-quality, professional software design.

The SOLID principles

The SOLID principles are a set of five foundational guidelines that help developers create maintainable and flexible software solutions. This application's architecture adheres to them as follows:

1. **Single Responsibility Principle** Each class and layer serves a single purpose and, therefore, has only one reason to change. A clear example is the separation of expense-related responsibilities:
   - The `ExpensesController` in the API layer handles HTTP communication with the Presentation layer by receiving requests and returning responses; changes are required only when the API contract evolves.
   - The `ExpenseService` in the Application layer is responsible for expense-related business logic; modifications occur only when business rules change.
   - The `ExpenseRepository` in the Infrastructure layer manages data persistence; changes are required only when the database or storage service changes.
2. **Open/Closed Principle**

   The system is open for extension but closed for modification. Thanks to the interfaces defined in the Application layer, introducing new functionality requires only the implementation of new classes in the Infrastructure layer. The existing, tested code in the Application layer remains unchanged.

   Currently, the `ReportService` in the Application layer depends on the `INotificationService`. If, in the future, clients require SMS notifications in addition to email notifications, it is sufficient to introduce a new `SMSNotificationService` class in the Infrastructure layer that implements the same interface. The Application layer does not require any modification.
3. **Liskov Substitution Principle**

   Any implementation of an interface should be substitutable for another without breaking the application. A clear example is the use of repositories: any class at the Infrastructure layer that implements the `IExpenseRepository` interface can be used interchangeably by the Application layer. Components that depend on external interfaces are responsible only for knowing how to use them, not for understanding their implementation.
4. **Interface Segregation Principle**

   Components should not be forced to depend on unnecessary interfaces. Interfaces are defined at a granular level, such as `IExpenseRepository` and `IFileStorage`, rather than combining responsibilities into a single large interface. As a result, components depend only on the interfaces they truly require.
5. **Dependency Inversion Principle**

   This is the key principle that holds Clean Architecture together. High-level modules should not depend on low-level modules; instead, both should depend on abstractions. For example, the Application layer does not depend directly on the Infrastructure layer, even though it relies on services implemented there. Rather, both layers depend on interfaces that define the required behavior.

Key Design Patterns

Several well-known design patterns are used throughout the source code to support the application of these principles:

- **Programming to an Interface**

  Components do not depend on concrete classes; instead, they rely on abstract interfaces. These interfaces (e.g., `IExpenseRepository`) define clear contracts for interacting with external components without exposing services' internal logic. This approach is the primary mechanism for achieving a decoupled system.

- **Service Pattern**

  The logic in the Application layer is organized into services (e.g., `ExpenseService`). Each service groups related business operations, providing a clear and easy-to-understand structure.

- **DTO Pattern**

  Data Transfer Objects (DTOs) are simple data containers that protect internal domain models from external exposure. Their primary purpose is to facilitate data transfer between the API and Application layers, establishing a clear and consistent contract between them.

- **Domain-Driven Design (DDD)**

  The Domain layer is structured according to Domain-Driven Design principles. It encapsulates the core business logic within a pure, isolated layer. As illustrated in Figure 9.2.1, all dependencies point inward toward the Domain layer, ensuring that the core business logic remains independent of external technologies and implementation details.

- **Repository Pattern**

  Repositories implement interfaces such as `IExpenseRepository` and encapsulate the logic behind these abstractions. For example, the Application layer can request to save an expense via `IExpenseRepository` without needing to know which database or storage mechanism is used.

- **Dependency Injection (DI)**

  Dependency Injection enables components to depend on external sources (e.g., `ExpenseService` depends on a repository for data operations). It is the practical implementation of the Dependency Inversion Principle, allowing layers to remain decoupled in the code while being connected and functional at runtime. This approach clarifies component responsibilities, reduces the need for code changes, and simplifies testing by removing dependencies on internal implementations.

All these patterns collectively support and reinforce the adoption of Clean Architecture principles. For a complete overview of the system structure applying these principles and patterns, refer to the UML Class Diagram in Section 9.4.2.

9.3 Frontend Design

The frontend is the primary interface through which users interact with the application. The central goal of its design is to provide an intuitive, responsive, and efficient user experience.

9.3.1 Navigation Flow Diagram

The Navigation Flow Diagram illustrates the sequences through which users can access the application's different screens to complete their tasks. Below, the application's navigation flow diagram is shown in Figure 9.3.1.

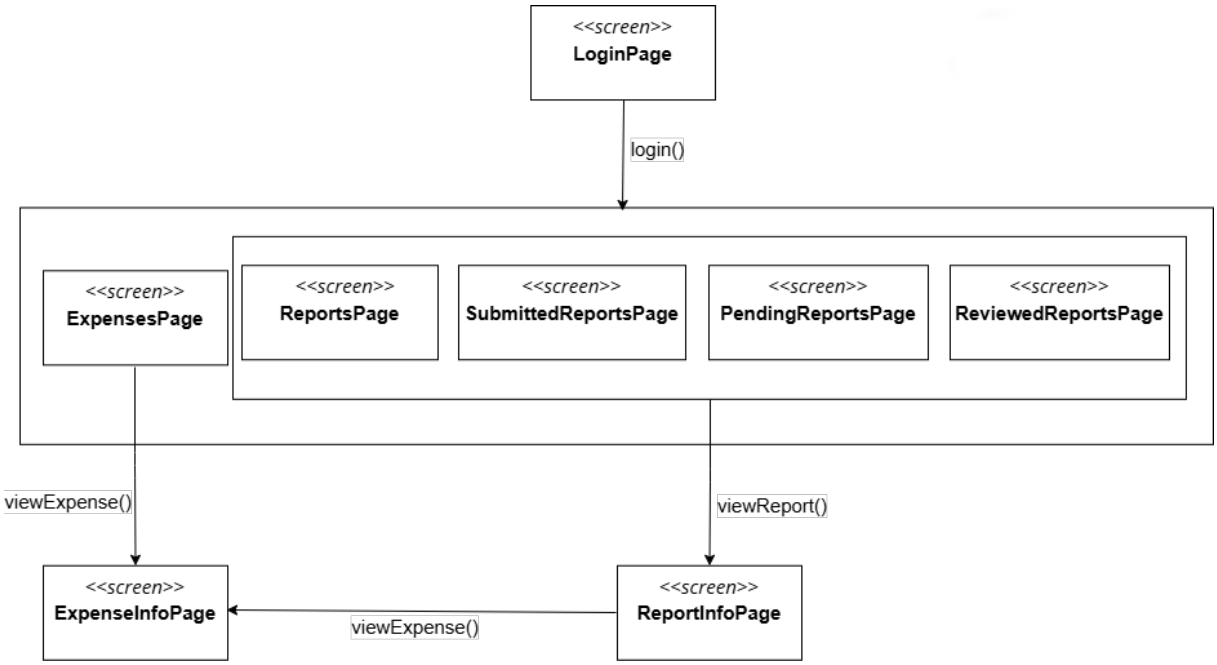

Figure 9.3.1: Navigation flow diagram. [Own creation]

The overall navigation flow is designed to be logical and task-oriented, guiding users through the application's functionalities. Each time a user accesses the application, the LoginPage is displayed, requiring authentication via a corporate email address. Upon successful authentication, the user is redirected to the default ExpensesPage. However, users may freely navigate to other main screens. Each main screen corresponds to a functional area described in Section 7.1, where users can perform the tasks outlined in the use case diagrams from the same section.

The application's main screens are defined as follows:

- **ExpensesPage**: Displays all expenses uploaded by the user that are not yet associated with any report.
- **ReportsPage**: Displays all reports created by the user that have not yet been submitted for review.
- **SubmittedReportsPage**: Displays all reports created by the user that have been submitted to a reviewer.

- **PendingReportsPage**: Accessible only to Admin users; displays all reports submitted to the Admin that are pending review.
- **ReviewerReportsPage**: Accessible only to Admin users; displays all reports that have been submitted to and reviewed by the Admin.

From any main screen, users can select an expense or report and navigate to the corresponding **ExpenseInfoPage or ReportInfoPage** to view detailed information. Within the ReportInfoPage, a list of associated expenses is presented, allowing users to further navigate to the respective **ExpenseInfoPage** entries.

Both **ReportInfoPage** and **ExpenseInfoPage** enable users to view detailed information and, when permitted, edit the displayed data. These pages remain editable only while the report has not been submitted. When accessed from **SubmittedReportsPage**, **PendingReportsPage**, or **ReviewerReportsPage**, both pages are presented in a read-only state.

As described in Section 9.2, the user interface follows a component-based architecture, where each component is constructed using reusable elements. This approach facilitates a consistent and simplified navigation flow. The **ExpensesPage** design is reused across other main report-related screens, which also share the same **ReportInfoPage**. Similarly, the **ExpenseInfoPage** is reused across both **ExpensesPage** and **ReportInfoPage**.

9.3.2 UI Design

Building on the navigation flow and functional description presented in the previous sections, this section provides a visual overview of the application's user interface. The following screenshots illustrate the main screens, their layout, and the interface states relevant to the user workflow. Additional interface variations and supplementary views are included in Appendix B for completeness.

LoginPage View

Figure 9.3.2 displays the UI corresponding to the application's secure access portal. This page is shown when the user accesses the application for the first time or when the session has expired.

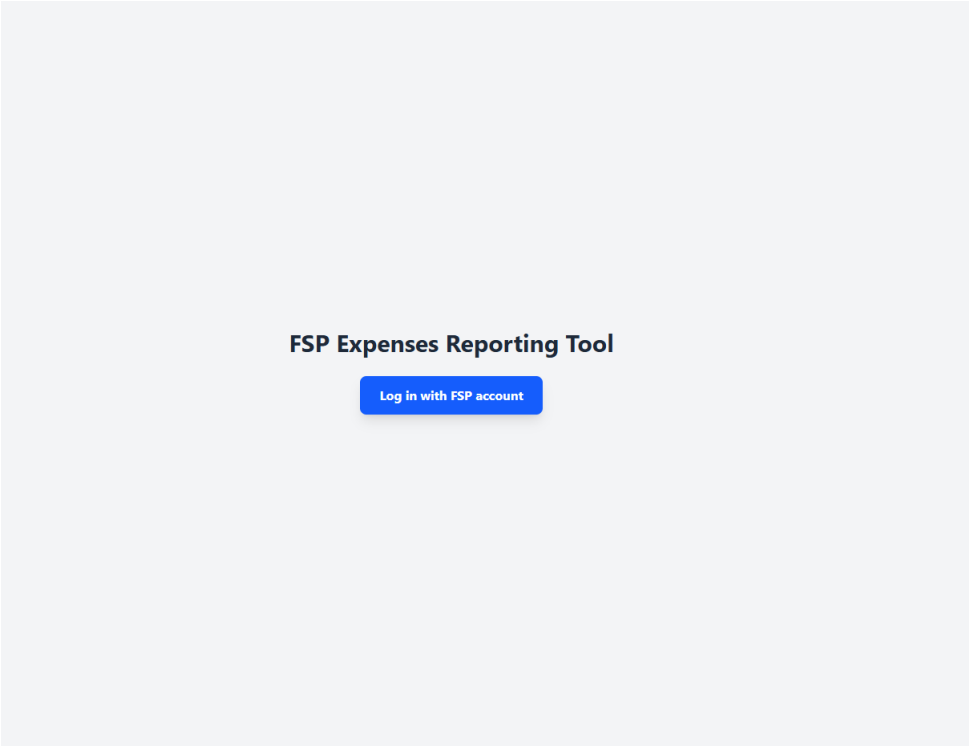

Figure 9.3.2: LoginPage user interface view. [Own creation]

ExpensesPage View

This is the ExpensesPage view, where users can perform all expense-related tasks. In the UI shown in Figure 9.3.3, users can upload multiple receipt files or images for processing and view the real-time AI processing status of an expense. Users can also search or filter uploaded expenses, as shown in Figure 9.3.4. Another feature of this page is the ability to attach expenses with a “Reviewed” status to a new or existing report. This last feature is illustrated in Figure B.2.3 from Appendix B.

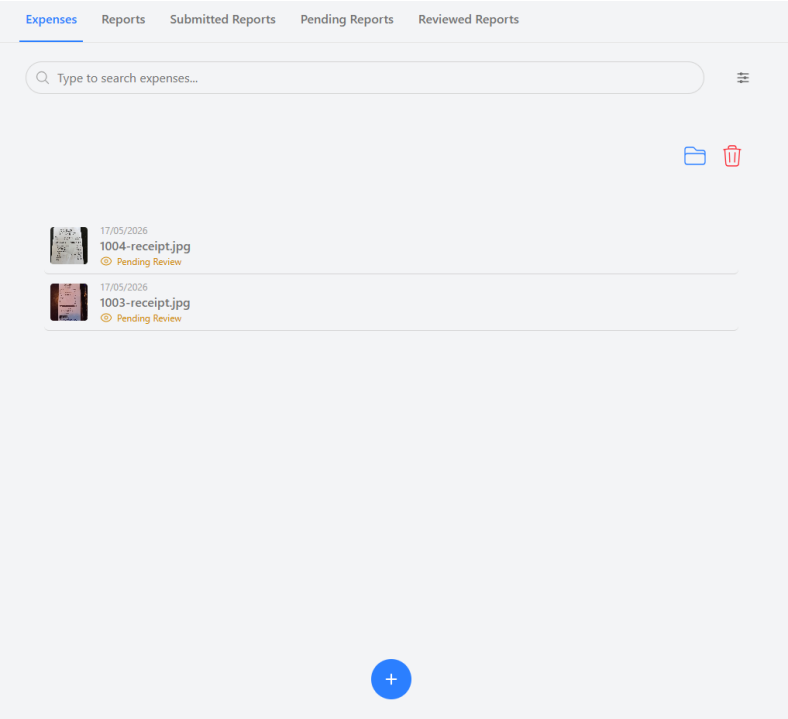

Figure 9.3.3: ExpensesPage user interface view. [Own creation]

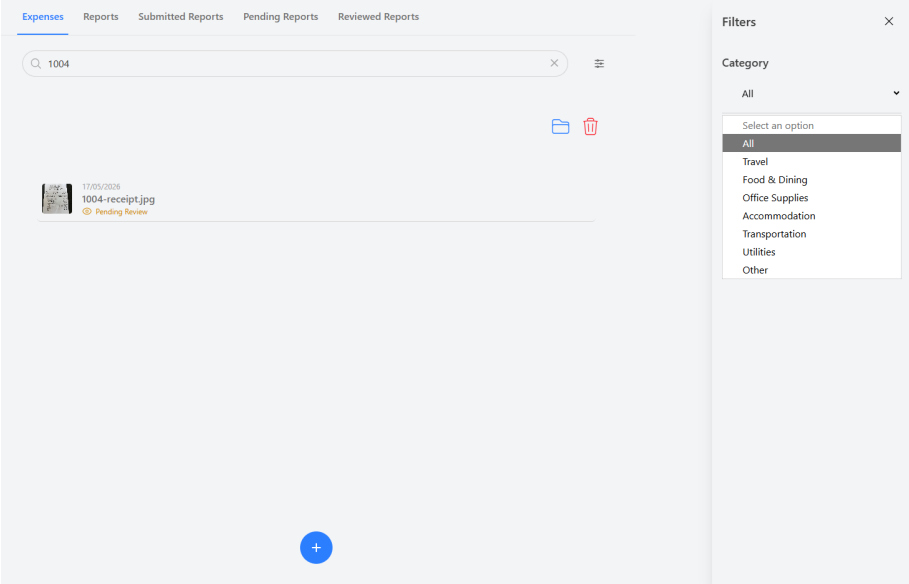

Figure 9.3.4: ExpensesPage user interface view for filtering by category and name. [Own creation]

ReportsPage View

In the ReportsPage view in Figure 9.3.5, users can view all reports they have created that are pending submission. The subsequent Figure 9.3.6 also illustrates the same page, which allows users to create new reports attaching expenses with a "Reviewed" status and delete existing reports.

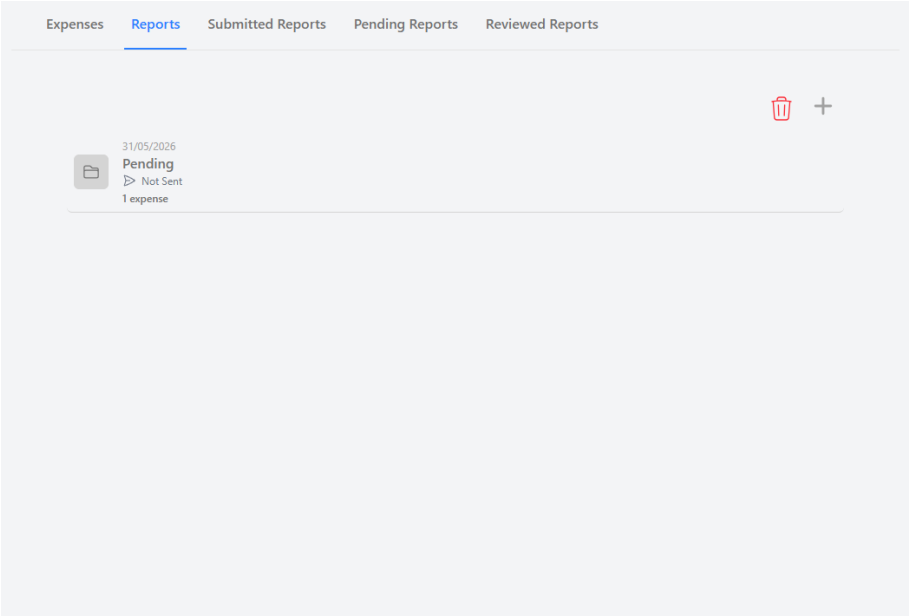

Figure 9.3.5: ReportsPage user interface view. [Own creation]

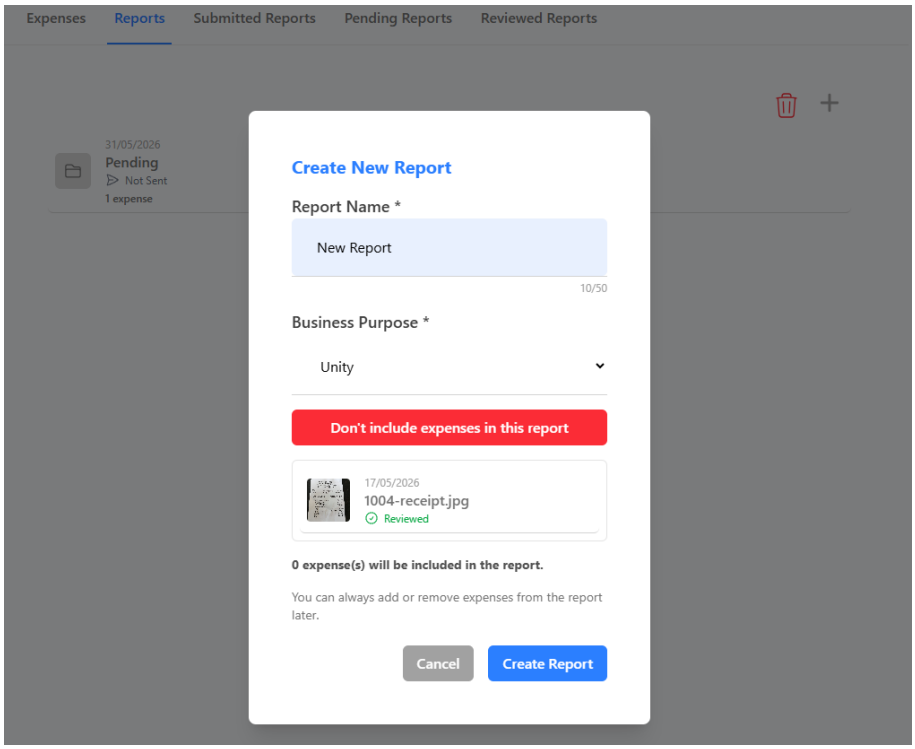

Figure 9.3.6: ReportsPage user interface view for creating a new report. [Own creation]

ExpenseInfoPage View

If an uploaded expense is successfully processed by the AI, the user can enter the corresponding ExpenseInfoPage view (Figure 9.3.7) to review and correct the AI-extracted information. If the AI extraction fails, the user can also enter the same page to manually enter the expense information.

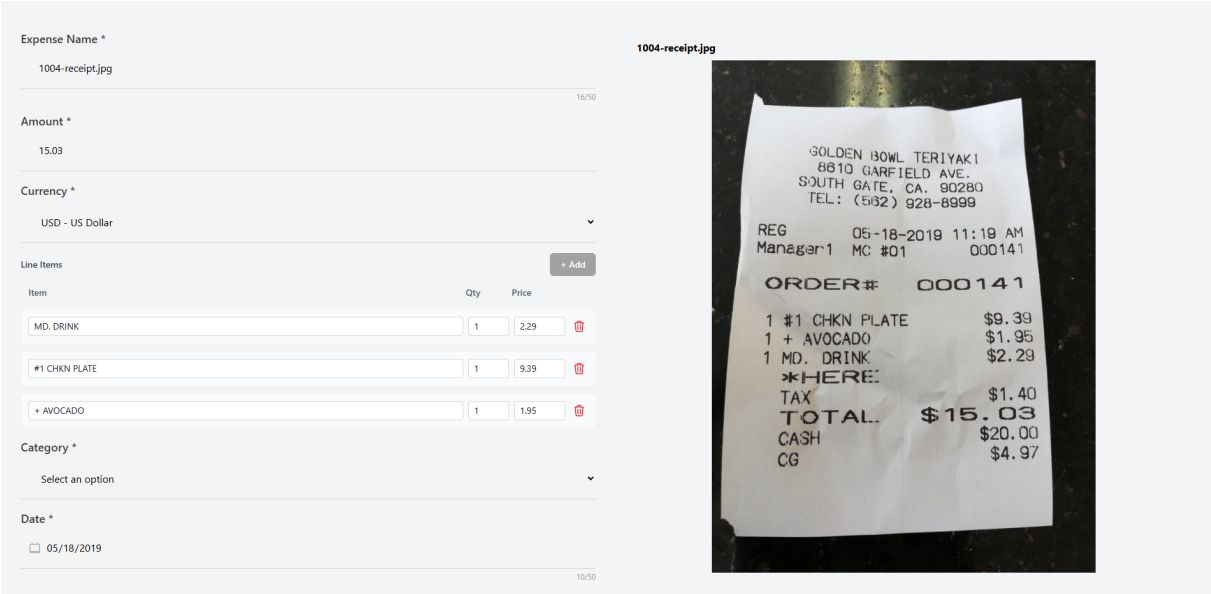

Figure 9.3.7: ExpenseInfoPage user interface view. [Own creation]

ReportInfoPage View

The last UI is the ReportInfoPage view displayed in Figure 9.3.8. Here, the user can review and modify the detailed information of each expense. On the right side, the user can attach or detach expenses from the current report and navigate to the attached expenses' ExpenseInfoPage views as well.

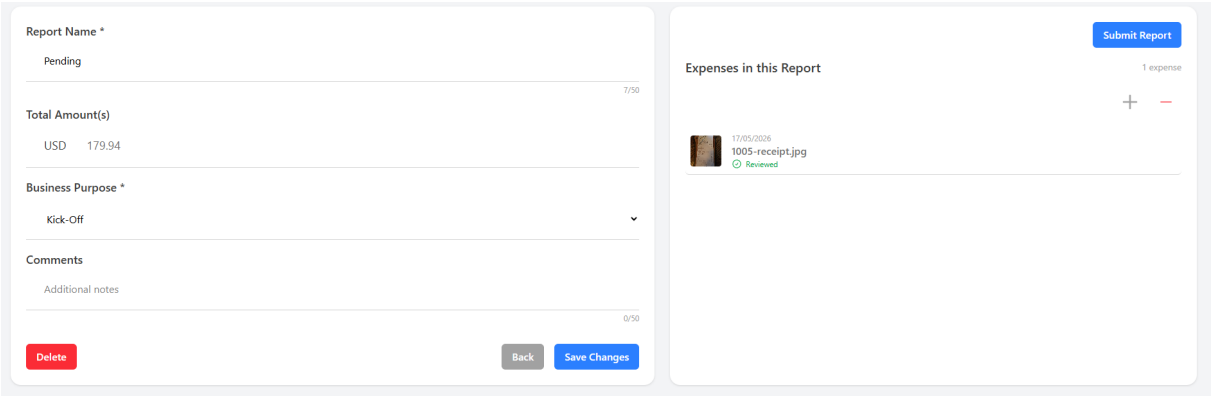

Figure 9.3.8: ReportInfoPage editable user interface view. [Own creation]

9.4 Backend Design

The backend is the engine of the application and is responsible for implementing all business logic, managing data persistence, and ensuring the system's security and integrity. The application's backend is developed on the ASP.NET platform, and its design is a direct, concrete implementation of the Clean Architecture principles outlined in the Logical Architecture section.

9.4.1 API Endpoints Specification

The API is the intermediary between the frontend and the backend. The application's API is designed to follow RESTful principles, using standard HTTP methods (`GET`, `POST`, `PUT`, and `DELETE`) for operations and communicating via the JSON data format. As a result, a clear, standardized interface is provided for any client application.

The following table summarizes the main functional capabilities exposed by the API, classified by their business resource. The core objective is to provide a high-level overview of what the system can do. A complete visual overview of all endpoints, along with detailed specifications for representative use cases, is available in Appendix C: API Endpoint Specification.

Resource	Endpoint Path	Description
Users	`GET /api/users/me`	Retrieves the profile and identity information of the currently authenticated user.
Expenses	`GET, POST, PUT, DELETE /api/expenses/{id}`	Provides full CRUD (Create, Read, Update, Delete) operations for individual expense records.
Reports	`GET, POST, PUT, DELETE /api/reports/{id}`	Provides full CRUD operations for expense reports, which act as containers for expenses.
Reports	`POST, DELETE /api/reports/{id}/expenses`	Manages the relationship between a report and its expenses (attaching or detaching an expense from a report).
Reports	`PATCH /api/reports/{id}/[action]`	Manages the report lifecycle workflow. Actions include submit, approve, and reject.
Blob	`GET /api/blob/sas-url`	Generates a secure, temporary URL (SAS) for direct and safe access to receipt files stored in the blob storage service.

Table 9.4.1: API endpoints specification. [Own creation]

9.4.2 UML Class Diagram

The UML class diagram displayed in Figure 9.4.1 provides a static, focused view of the main classes involved in the core Expense use case. This diagram is the concrete blueprint that visually represents how the abstract concepts of Clean Architecture are translated into code for a primary functional area.

Building on this blueprint, the diagram illustrates the relationships between the `ExpenseController`, the `IExpenseService`, and the `IExpenseRepository`, as well as their implementations, it also demonstrates the application of the main architectural principles, patterns, and design approaches explained in Section 9.2.2.

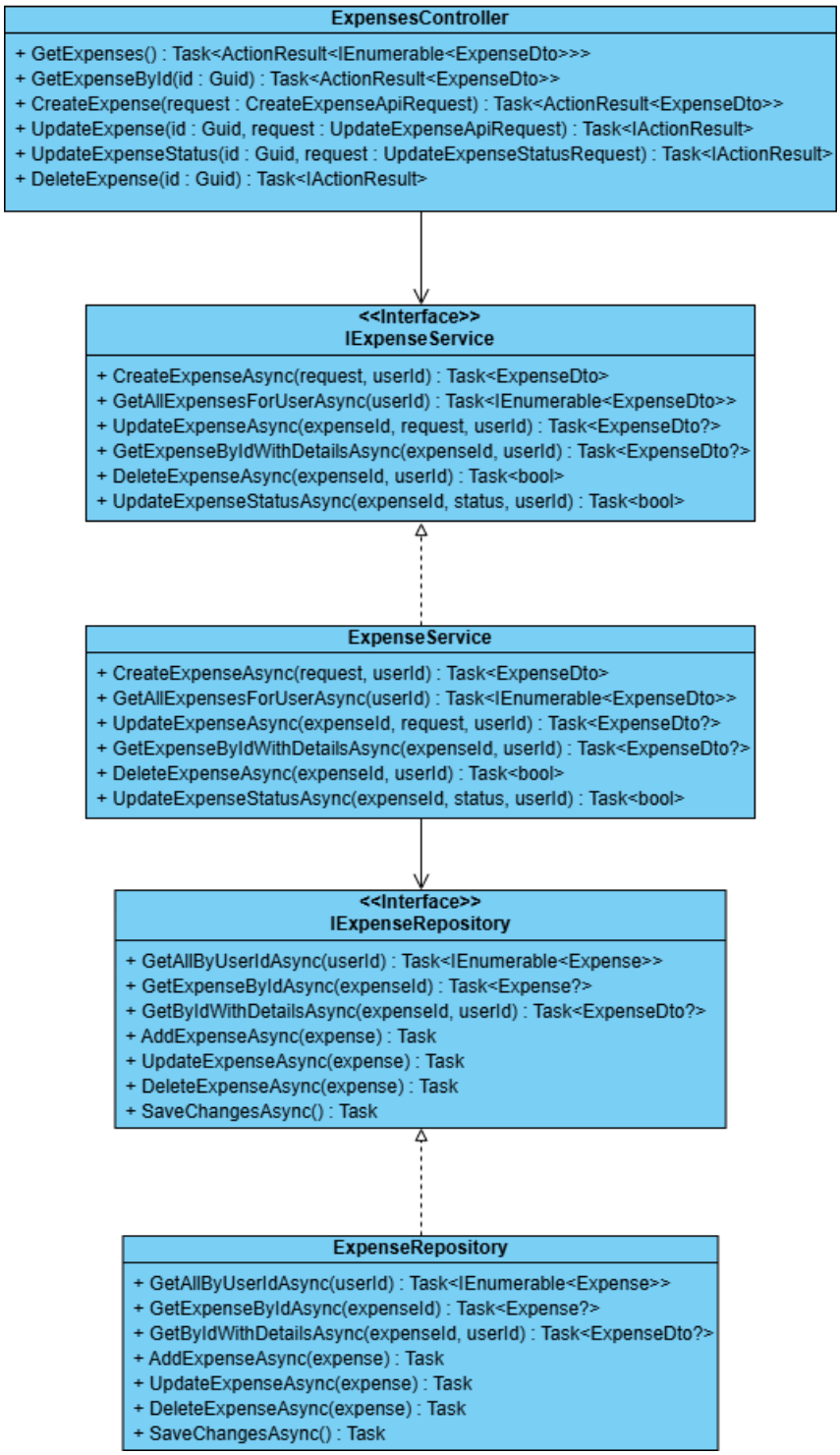

Figure 9.4.1: Backend UML class diagram focused on Expense functionality. [Own creation]

A detailed UML class diagram covering all classes within the backend's logical layers has also been created to ensure a complete system model. Due to its size and level of detail, it is provided in Appendix D. Given the diagram's complexity, the Domain and DTO classes are omitted, although they are implicitly included within each class's operations. For a more detailed domain diagram, please refer to Figure 9.6.1.

9.4.3 Main Use Cases Sequence Diagrams

While a class diagram shows the status structure, sequence diagrams illustrate the dynamic behaviour of the system over time. To demonstrate how the backend orchestrates the logical layers to fulfill user requests, this section presents the sequence of interactions for the two most critical use cases.

Before delving into the sequence diagrams, it is important to note that all API Controllers are protected using the ASP.NET Core authorization middleware through the `[Authorize]` attribute. This middleware enforces authentication before any request reaches a controller action.

Specifically, before a request reaches the controller, the middleware validates the Azure AD Bearer token included in the request. If the token is invalid or missing, the middleware immediately returns a `401 Unauthorized` response. If validation succeeds, the user's claims are extracted and made available through the `CurrentUserService`, and the request is forwarded to the corresponding controller action.

Use Case 1: Uploading a New Expense

This use case showcases the system's hybrid processing model. The initial file upload request is handled synchronously to return an immediate response to the user, while the high computational cost of AI-driven data extraction is isolated to an asynchronous background process.

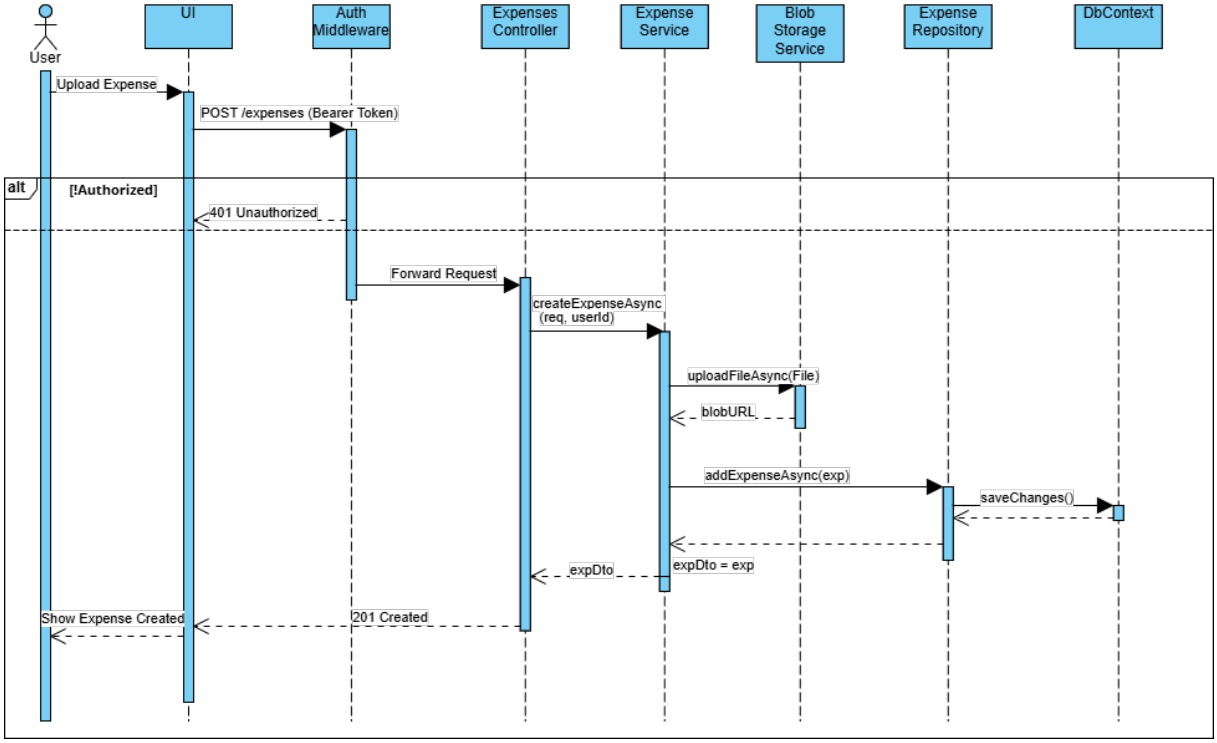

Figure 9.4.2: Sequence diagram for the synchronous Expense upload workflow. [Own creation]

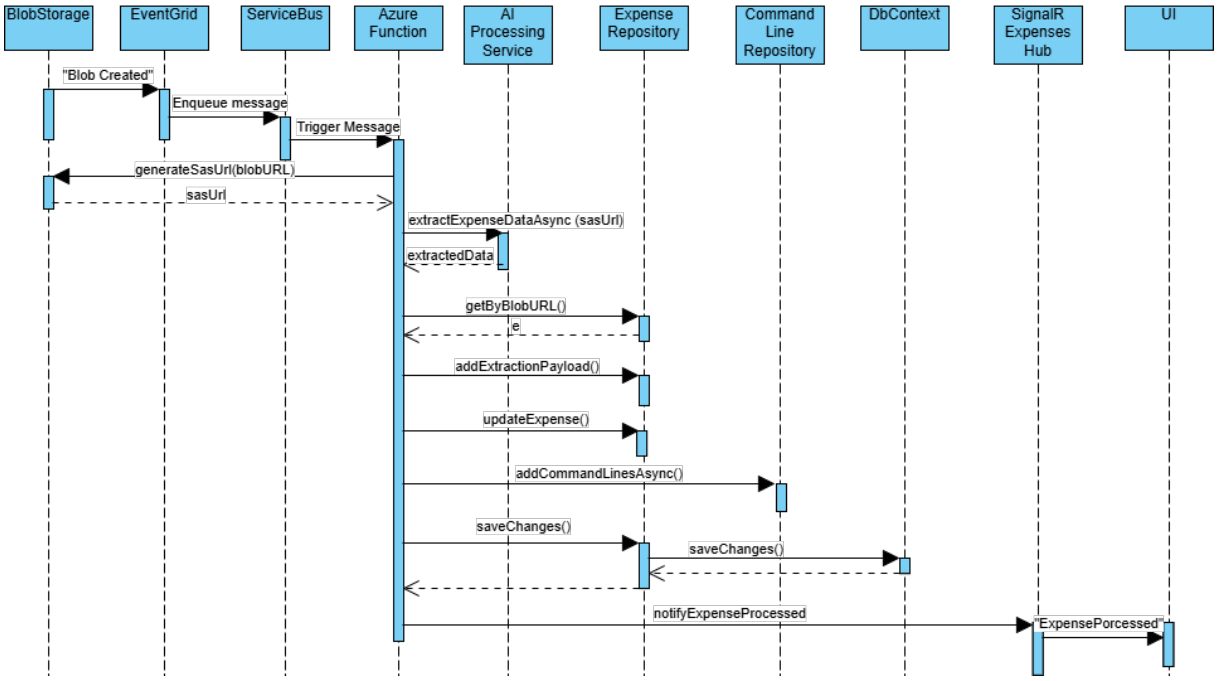

Figure 9.4.3: Sequence diagram for the asynchronous Expense processing workflow. [Own creation]

Synchronous Expense Upload Process

Figure 9.4.2 provides a comprehensive visualization of the synchronous expense upload process. When the user uploads a receipt, the client (UI) sends an HTTP request to the API. If the user is successfully authenticated, the `ExpensesController` receives the request and delegates the operation to the `ExpenseService`, which executes the expense creation business logic.

The `ExpenseService` first calls the `BlobStorageService` to upload the expense receipt file to Azure Blob Storage. After the upload is completed, the `BlobStorageService` returns the generated blob URL.

With this URL in hand, the `Expense- Service` creates a new `Expense` entity and passes it to the `ExpenseRepository`. The repository is responsible for persisting the entity by interacting with the `ApplicationDb- Context` through Entity Framework Core (EF Core), which ultimately stores the expense metadata in the SQL database.

Once the expense has been successfully persisted, the controller returns a 201 Created response to the client (UI) while the asynchronous expense-processing pipeline continues independently.

Asynchronous Background Processing

After a new expense file is uploaded to Blob Storage, the asynchronous flow represented in Figure 9.4.3 is performed in the background. This asynchronous flow was previously illustrated with software resources in the physical architecture diagram in Section 9.1 and corresponds to the Asynchronous AI Processing Flow explained in the same section.

Use Case 2: Approving a Report

This second use case shows how the system triggers asynchronous email notifications when a report status changes. This use case is also a hybrid approach composed of a synchronous report approval workflow, as shown in Figure 9.4.4, which triggers the subsequent asynchronous email notification and publishing flow depicted in Figure 9.4.5.

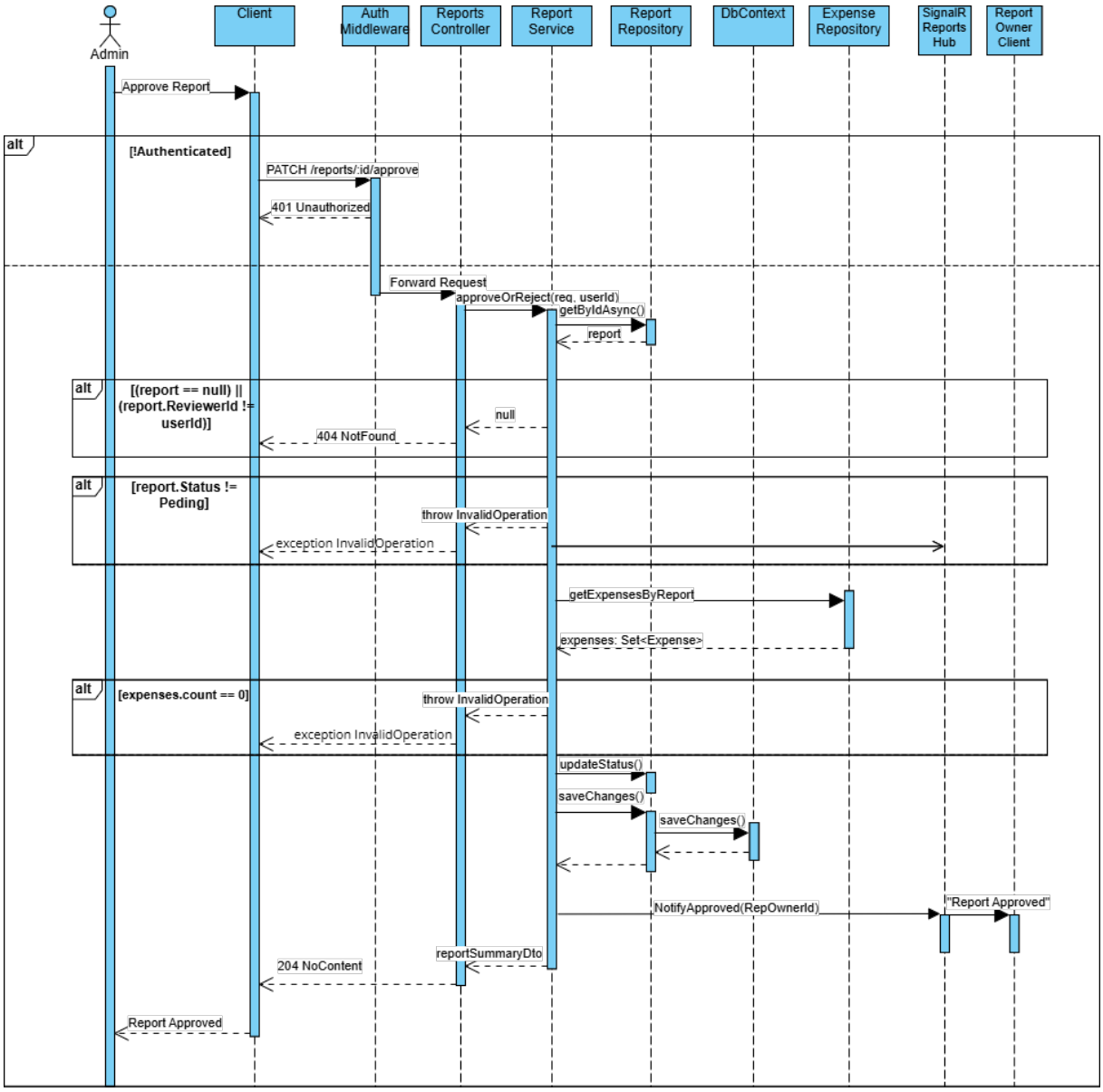

Figure 9.4.4: Sequence diagram for the synchronous Report approval workflow. [Own creation]

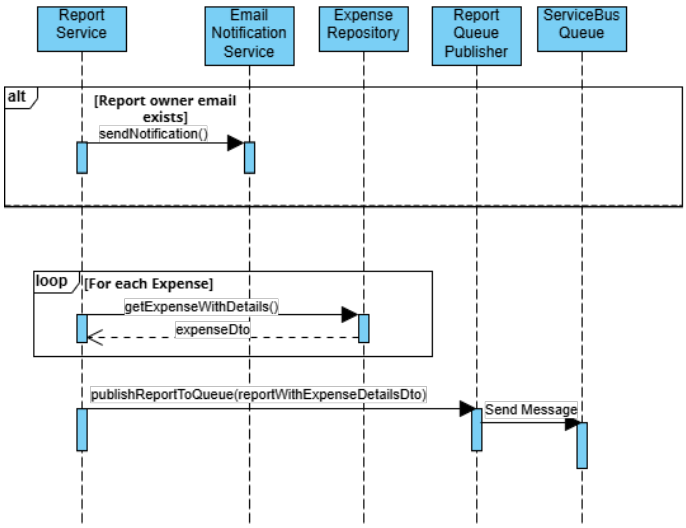

Figure 9.4.5: Sequence diagram for the asynchronous workflow after Report approval. [Own creation]

Report Approval Workflow

When an admin user approves a report submitted by an employee under their supervision, the client (UI) sends an HTTP request to the API. After the user is successfully authenticated and the `ReportsController` receives the request, the operation is delegated to the `ReportService`, which is responsible for applying the business rules.

The `ReportService` first verifies that the report exists and that the assigned reviewer matches the authenticated user. If either validation fails, the service returns null to the `ReportsController`, which subsequently returns a 404 Not Found response to the client.

Next, the `ReportService` validates that the report status is Pending and that at least one expense is attached to the report. If any of these conditions are not met, an `InvalidOperationException` is thrown and propagated to the controller, which returns the corresponding error response to the client (UI).

If all business rules are satisfied, the `ReportService` requests the `ReportRepository` to update the report status to Approved and persist the changes. The `ReportRepository` interacts with the `ApplicationDbContext` through Entity Framework Core (EF Core), which ultimately updates the report record in the SQL database.

Once the report has been successfully approved, the `ReportService` sends a message through the SignalR `ReportsHub` to notify the report owner. Finally, the controller returns a `204 No Content` response to the client (UI), indicating that the operation completed successfully.

Email Notification and Publishing after Approval

Once a report has been approved, the `ReportService` also performs several background tasks. First, it verifies that the report owner's email address exists to send an email notification informing them that their report has been approved.

Next, the service retrieves the detailed information for all expenses associated with the report and composes a complete `ReportWithExpenseDetailsDto`. This DTO is then published through the `ReportQueuePublisher`, which sends the message to an external service bus queue. The purpose of this final step is to provide data for training an internal AI model.

9.5 AI Approach Justification

Considering the client's current situation described in Section 1.3, a key problem that needed to be addressed was the tedious process of manually entering receipt data. To solve this problem, the client explicitly requested an application that integrates an external AI model. At the same time, the solution must require only minimal code changes, such as changing an endpoint, if the company later decides to use an internal model.

The proposed solution is a division between fast, synchronous operations for user interactions and a robust, asynchronous pipeline for background processing. Both flows serve distinct purposes and are completely separated in the Logical Architecture Diagram in Figure 9.2.1. This ensures that the user experience remains fast and responsive. The system uses the external AI model through an interface to satisfy the client's maintainability requirement, which is also reflected in the UML class diagram in Appendix D.

Regarding the origin, nature, and processing of data, this is already specified in Section 9.2, specifically in the Asynchronous AI Processing Flow.

9.6 Database Design

The application's data persistence strategy is designed to be secure, scalable, and efficient by separating data based on its structure. Structured, transactional data is managed by a relational database, while unstructured binary data is handled by a dedicated object storage service.

9.6.1 Database Model Justification

The nature of the application's data guided the choice of a relational data model. Core entities such as Users, Expenses, and Reports exhibit clear, well-defined relationships (e.g., a report contains many expenses; an expense belongs to one user). A relational model is particularly effective at enforcing these relationships and ensuring data integrity through mechanisms such as primary keys, foreign keys, and transaction support (ACID compliance).

- **Primary keys** define the unique identifier of an entity instance, enabling efficient retrieval of an instance at runtime.
- **Foreign keys** define relationships between entities, allowing efficient searches based on those relationships when required.
- **ACID support** guarantees that transactions behave correctly by ensuring the following properties:
  - **Atomicity**: Every transaction is treated as a single unit; either all operations succeed or all fail, preventing partial updates that would produce inconsistent data.

- **Consistency**: Transactions must adhere to the integrity constraints defined for storing valid data.
- **Isolation**: Each transaction is isolated; one transaction does not affect the result of another.
- **Durability**: Once a transaction is successfully committed, its changes are permanently saved, even if the system fails.

9.6.2 DBMS Selection

The chosen Database Management System (DBMS) is **Azure SQL Database**, which offers the following advantages:

- **Seamless Integration**: It can be natively integrated with the application's ASP.NET Core backend through Entity Framework Core, as well as with other Azure resources used throughout the application.
- **Managed Operations**: Development effort can focus on application features, as the Azure platform handles critical tasks such as backups, patching, and high availability.
- **Scalability**: The database's performance and storage can be scaled up or down, adapting to the application's growth and demand.
- **Robust Security**: It includes advanced, built-in security features that are fundamental to addressing the application's security requirements.

9.6.3 Logical Database Design

The logical database design is a direct translation of the entities defined in the application's Domain layer. The core entities are Users, Expenses, and Reports:

- **Users**: Store user identity information, including the user's role, which determines whether they have a supervisor or supervise others.
- **Expenses**: Contain details of each individual uploaded expense, including AI-extracted metadata, the command lines appearing on the receipt, and a foreign key linking to a User.
- **Reports**: Represent a collection of Expenses that are manually reviewed by the user and are intended to be submitted to obtain the corresponding reimbursement. This entity has a one-to-many relationship with the Expenses table, belongs to the user who created it, and can be reviewed by the creator's supervisor.

All these relationships are enforced through foreign key constraints, ensuring referential integrity across the data.

9.6.4 Physical Database Design

Figure 9.6.1: Database schema diagram. [Own creation]

The physical database design is shown in the schema above (Figure 9.6.1). While the logical design defines the structure, the physical design focuses on performance. To ensure fast query performance, indexes are critical. Entity Framework Core automatically creates indexes on all primary keys and foreign keys. These indexes cover the most common query patterns, such as retrieving all expenses belonging to a specific report.

10 Implementation

This section details a selection of the most relevant aspects of the application's implementation. It focuses on the technical decisions and tools used to build the system design described in the previous section. Rather than presenting the full codebase, only selected code snippets and configurations that illustrate key architectural challenges are highlighted.

10.1 Development Environment and Tooling

The set of tools mentioned below ensured efficient, high-quality development throughout the implementation process.

Integrated Development Environment (IDE)

Since the backend is developed on the .NET platform and the frontend uses React and TypeScript, and Visual Studio Code (VSC) perfectly supports both environments, VSC is an excellent IDE choice. Therefore, the entire development is carried out in VSC.

Source Code Management

The project's source code is managed using Git, the standard version control system in the industry. The Git repository is hosted on Azure DevOps, which facilitates version tracking.

Dependency Management

The NuGet package manager is integrated into the .NET ecosystem and handles backend dependencies and third-party libraries. At the same time, npm (Node Package Manager) is used to manage React libraries and development tools for building, running, and deploying the frontend application.

SDKs and Management Tools

The .NET SDK provides essential development tools and command-line utilities for building, running, and testing the application through PowerShell commands. The Azure Portal provides a web-based interface for managing Azure resources via an intuitive user interface.

10.2 Key Backend Implementation Aspects

This section specifies the transition from the most complex patterns and workflows explained in Section 9.4 into actual code implementation.

10.2.1 Security and Authentication

Regarding the security architecture design, the implementation is streamlined through `Microsoft.Identity.Web` library, which provides seamless integration with Azure Active Directory and simplifies JWT bearer token validation. The authentication middleware is configured in the application startup file (`Program.cs`) as shown in Figure 10.2.1.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApi(options =>
    {
        builder.Configuration.Bind("AzureAd", options);

        var clientId = builder.Configuration["AzureAd:ClientId"];

        options.TokenValidationParameters.ValidAudiences = new[]
        {
            $"api://{clientId}",
            clientId
        };

        options.TokenValidationParameters.ValidateAudience = true;
    },
    options =>
    {
        builder.Configuration.Bind("AzureAd", options);
    });
```

Figure 10.2.1: Authentication middleware configuration in `Program.cs`. [Own creation]

This configuration registers the JWT Bearer authentication scheme and integrates it with Azure AD through the `Microsoft.Identity.Web` library. The Azure AD settings are loaded from the application configuration, while the token validation parameters are customized to accept `api://clientId` and the Client ID as valid audiences. Audience validation ensures that only tokens intended for this API are accepted.

With this configuration in place, the connection with the required middleware is established to automatically validate the JWT bearer token included in the `Authorization` header of incoming requests. This ensures that every request is authenticated against Azure AD before reaching the controller actions.

To protect the API endpoints, it is sufficient to simply add an `[Authorize]` (see Figure 10.2.2) attribute at the top of the controller classes. This attribute restricts resource access to exclusively authenticated users.

```
You, last month | 1 author (You)
[ApiController]
[Route("api/[controller]")]
[Authorize] // Require authentication for all actions in this controller
3 references
public class ExpensesController : BaseController
{
```

Figure 10.2.2: Authorization attribute configuration in `ExpensesController.cs`. [Own creation]

10.2.2 Asynchronous Processing Pipeline

The asynchronous flow is hosted in an Azure Function. The `[ServiceBusTrigger]` attribute establishes the connection between the Azure Service Bus queue and the function's `ProcessExpenseFromQueue` implementation. This attribute instructs Azure Functions to execute the `Run` method whenever a new message appears in the queue named `expense-processing-queue`. This process is illustrated in Figure 10.2.3.

```
[Function("ProcessExpenseFromQueue")]
[SignalROutput(HubName = "expenseshub", ConnectionStringSetting =
"AzureSignalRConnectionString")]
9 references
public async Task<SignalRMessage?> Run([ServiceBusTrigger("expense-processing-queue"
Connection = "ServiceBusTriggerConnection")] string queueMessage)
```

Figure 10.2.3: `Run` method implementation in `ProcessExpenseFromQueue.cs`. [Own creation]

10.2.3 SAS URL Generation

The generation of a secure, temporary link (SAS URL) to the receipts stored in Azure Blob Storage is handled by the `BlobStorageService` class. This class uses the Azure Storage SDK to create a short-lived URL with read-only permissions. This URL is then sent to the external AI service, allowing secure access to the file and avoiding permanent public access without requiring a complex credential strategy.

At the same time, the client (frontend) also requests the URL to display the file associated with the expense to its owner. The code implementation for generating the SAS URL is presented in Figure 10.2.4.

```
var sasBuilder = new BlobSasBuilder
{
    BlobContainerName = containerName,
    BlobName = blobName,
    Resource = "b", // "b" = blob
    StartsOn = now.AddMinutes(-5),
    ExpiresOn = now.AddMinutes(expiryMinutes),
};
sasBuilder.SetPermissions(BlobSasPermissions.Read);

// Build the SAS URI using the user delegation key
Uri sasUri = blobClient.GenerateUserDelegationSasUri(sasBuilder, userDelegationKey);
return sasUri.ToString();
```

Figure 10.2.4: SAS URL generation in `BlobStorageService.cs`. [Own creation]

10.2.4 Data Persistence with EF Core Code-First

The database schema was implemented using Entity Framework Core's Code-First approach, where C# domain model classes (entities in the Domain layer) serve as the single source of truth to generate and migrate the database schema. The `ApplicationDbContext` class, as depicted in Figure 10.2.5, defines the tables (`DbSet<T>`) and their relationships.

```
14 references
public DbSet<User> Users { get; set; }
27 references
public DbSet<Expense> Expenses { get; set; }


0 references
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<User>().HasKey(u => u.UserId);
    modelBuilder.Entity<Expense>().HasKey(e => e.ExpenseId);


    modelBuilder.Entity<User>()
        .HasMany<Expense>()
        .WithOne(e => e.Employee)
        .HasForeignKey(e => e.EmployeeId)
        .OnDelete(DeleteBehavior.Restrict);
```

Figure 10.2.5: Entity Framework Core configuration in `ApplicationDbContext.cs`. [Own creation]

Every time the domain model changes (e.g., adding a new property to the `Expense` entity), a new database migration is created using the following .NET CLI command (included in the .NET SDK):

```
> dotnet ef migrations add <MigrationName>
```

The generated migration file contains the necessary SQL scripts to update the database schema without losing existing data or causing data inconsistency. The migration is applied using the following command:

```
> dotnet ef database update
```

By executing these two commands, the database stays in sync with the application's domain models.

10.3 Key Frontend Implementation Aspects

This section highlights the code-level solutions used to implement the frontend's most complex functionalities: authentication and real-time communication.

10.3.1 Authentication Flow with MSAL

User authentication uses the Microsoft Authentication Library (MSAL). This library is configured in a dedicated file (`authConfig.ts`) with the application's unique `clientId` and `tenantId` from Azure AD. Figure 10.3.1 holds the authConfig.ts file showing the MSAL configuration with clientId and tenantId.

```ts
import type { Configuration, PopupRequest } from "@azure/msal-browser";

const TENANT_ID = import.meta.env.VITE_API_TENANT_ID;
const UI_CLIENT_ID = import.meta.env.VITE_UI_CLIENT_ID;
const API_CLIENT_ID = import.meta.env.VITE_API_CLIENT_ID;

export const msalConfig: Configuration = {
  auth: {
    clientId: UI_CLIENT_ID,
    authority: `https://login.microsoftonline.com/${TENANT_ID}`,
    redirectUri: globalThis.location.origin,
  },
  cache: {
    cacheLocation: "sessionStorage", // Store session while using the app
    storeAuthStateInCookie: false,
  },
};
```

Figure 10.3.1: MSAL configuration in `authConfig.ts`. [Own creation]

To ensure the authentication is applied throughout the entire application, the root React component is wrapped with the `MsalProvider` in the main entry point file (`main.tsx`), as configured in Figure 10.3.2.

```tsx
const msalInstance = new PublicClientApplication(msalConfig);



  createRoot(rootElement).render(
    <StrictMode>
      <MsalProvider instance={msalInstance}>
        <BrowserRouter>
          <App />
        </BrowserRouter>
      </MsalProvider>
    </StrictMode>
  );
```

Figure 10.3.2: MSAL authentication setup in `main.tsx`. [Own creation]

An Axios interceptor is also implemented in the `apiClient.ts` file to automatically secure communication with the backend API. This small piece of code intercepts every outgoing API request, silently acquires a valid JWT access token from MSAL, and adds it to the `Authorization` header. In this way, every API request is authenticated while avoiding repetitive code in each component. The detailed code implementation is provided in Figure 10.3.3.

```
const msalInstance = new PublicClientApplication(msalConfig);

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
});

apiClient.interceptors.request.use(async (config) => {
    try {
      const response = await msalInstance.acquireTokenPopup(apiRequestConfig);

      if (response.accessToken) {
        config.headers.Authorization = `Bearer ${response.accessToken}`;
      }
```

Figure 10.3.3: Axios interceptor configuration in `apiClient.tsx`. [Own creation]

10.3.2 Real-time Communication with SignalR

The application's real-time updates are handled using SignalR. A dedicated service, `signalrService.ts`, encapsulates the connection logic. A key implementation detail is securing the SignalR connection through the use of an `accessTokenFactory`. This function is invoked by the SignalR client whenever an access token is required and retrieves a valid JWT access token from MSAL using the current user context.

Once the token is retrieved, it is included in the connection request, ensuring that the real-time communication channel is authenticated and authorized. Refer to Figure 10.3.4 for the `signalrService.ts` code snippet that demonstrates the `accessTokenFactory` implementation.

```
// Create MSAL instance to acquire tokens of this service
const msalInstance = new PublicClientApplication(msalConfig);

// Function to get access token for SignalR connection
const getAccessToken = async () => {
  await msalInstance.initialize();
  const accounts = msalInstance.getAllAccounts();
  if (accounts.length > 0) {
    const response = await msalInstance.acquireTokenSilent({
      ...apiRequestConfig,
      account: accounts[0],
    });
    return response.accessToken;
  }
  return "";
};

// Create SignalR connection for expenses updates
const expensesConnection = new signalR.HubConnectionBuilder()
  .withUrl(EXPENSES_HUB_URL, {
    //SignalR will call this function to get the access token before starting the connection
    accessTokenFactory: () => getAccessToken(),
  })
  .withAutomaticReconnect()
  .build();
```

Figure 10.3.4: `AccessTokenFactory` implementation in `signalrService.ts`.[Own creation]

Once the connection is established, the service subscribes to specific events broadcast by the server hub. The following code (Figure 10.3.5) shows how the frontend listens for the `"ExpenseProcessed"` and `"ExpenseProcessingFailed"` events and triggers a function to update the user interface accordingly.

```
export const addExpenseUpdateListener = (callback: (data: any) => void) => {
  expensesConnection.on("ExpenseProcessed", callback);
  expensesConnection.on("ExpenseProcessingFailed", callback);
  return () => {
    expensesConnection.off("ExpenseProcessed", callback);
    expensesConnection.off("ExpenseProcessingFailed", callback);
  };
};
```

Figure 10.3.5: SignalR event subscription implementation. [Own creation]

11 Validation Methods

The validation strategy ensures the application's quality and reliability. In this project, validation follows a multi-layered testing approach, combining static code analysis, automated tests (unit and integration), and manual acceptance testing. Together, these methods verify that the system functions as designed.

11.1 Static Code Analysis

ESLint and SonarQube, available as extensions in the Visual Studio Code editor, were installed before starting code implementation. These two tools helped detect potential bugs and code that is prone to bugs, known as "code smells", at an early stage. As a result, code quality is maintained, and a clean codebase is preserved throughout the project.

11.2 Unit and Integration Testing

A separate logical layer was created in the backend to execute automated tests. By running the `dotnet test` command, all test codes are executed, enabling continuous verification of the application. This test layer includes two types of automated tests: unit tests and integration tests.

Unit tests check the functionality of single, small, and isolated components. Due to the decoupled nature of Clean Architecture, component dependencies are defined through abstract interfaces. This characteristic significantly facilitates the development and execution of unit tests.

Integration tests verify that multiple components of the system work correctly when interacting with each other. These automated tests ensure that functionality spanning different architectural layers operates as expected and that communication between those layers is performed correctly.

To demonstrate the effectiveness of automated testing, a code coverage report was generated. As shown in Figure 11.2.1, a line coverage of 78% was achieved.

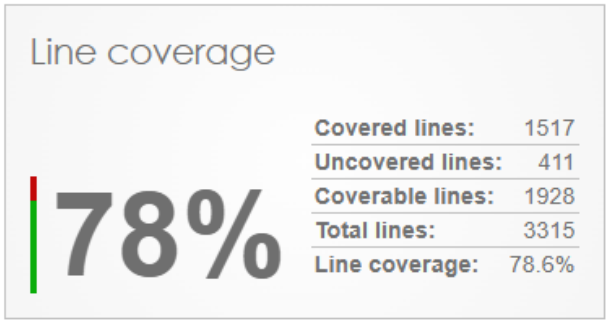

Figure 11.2.1: Code coverage report for automated tests. [Own creation]

11.3 Manual Acceptance Testing

To further demonstrate that both functional and non-functional requirements are satisfied, various testing strategies tailored to specific criteria are employed. The following subsections provide a more detailed description of these testing strategies.

11.3.1 Functional Requirements

The Product Owner (PO) was responsible for verifying the defined functional requirements. A demo is shown for every new functionality during the iterative meetings with the PO, who then approves it or requests additional implementation before considering the task completed.

Additionally, manual acceptance testing is performed by executing the primary user workflows defined in the use case diagrams (Section 7.1). This ensures that the application behaves as expected from an end-user's perspective. A summary table of the obtained results for each use case is provided.

Use Case	Result
Account Management	
Authenticate via Corporate Email	Accepted
Expense Management	
List Personal Expenses	Accepted
Register Expenses via Upload	Accepted
Monitor AI Processing Status	Accepted
Delete Individual Expense	Accepted
Delete Multiple Expenses	Accepted
Report Multiple Expenses	Accepted
Consult Expense Details	Accepted
Update Expense Information	Accepted
Delete Current Expense	Accepted
Search Expenses by Name	Accepted
Filter Expenses by Category	Accepted
Report Management	
List Personal Reports	Accepted
Create New Report	Accepted
Delete Individual Report	Accepted
Delete Multiple Reports	Accepted
Consult Report Details	Accepted
Update Report Information	Accepted
Delete Current Report	Accepted
Submit Report	Accepted
Attach Expenses to Report	Accepted
Detach Multiple Expenses from Report	Accepted
Detach Individual Expense from Report	Accepted
Consult Attached Expense Details	Accepted
Update Attached Expense Information	Accepted
Detach Current Expense from Report	Accepted

(Continued on next page)

(Continued from previous page)

Use Case	Result
Submitted Report Management	
List Personal Submitted Reports	Accepted
Monitor Submitted Report Processing Status	Accepted
Consult Submitted Report Details	Accepted
Consult the Attached Expense Details From Submitted Report	Accepted
Pending Review Report Management	
List Pending Review Reports	Accepted
Consult Pending Review Report Details	Accepted
Consult the Attached Expense Details From Pending Review Report	Accepted
Approve/Reject Report	Accepted
Reviewed Report Management	
List Reviewed Reports	Accepted
Consult Reviewed Report Details	Accepted
Consult Attached Expense Details From Reviewed Report	Accepted

Table 11.3.1: Summary of manual acceptance testing results for implemented use cases. [Own creation]

11.3.2 Non-Functional Requirements

For each non-functional requirement (NFR), the testing method is adapted according to the established criteria. The following table summarizes the previously defined acceptance criteria for each non-functional requirement before specifying the employed strategies.

NFR	Acceptance Criteria
NFR1	The UI must update expense statuses in less than **1 second** once the AI-driven data extraction is completed.
NFR2	The UI must be intuitive, allowing new users to learn how to use the application within **3 minutes**.
NFR3	The application must load previously uploaded expenses within **3 seconds**. Other basic user interactions (e.g., saving expense information, deleting expenses) must complete under **500 ms**. Additionally, the application must be available **24 hours a day, 365 days a year**.
NFR4	The application's architecture must employ a **hybrid approach** that separates **synchronous** and **asynchronous** workflows. The application's resources must be **deployed** on a **cloud computing platform** that **supports automatic horizontal scaling**.
NFR5	The acceptance criteria for this specific requirement are justified in **Section 14.1**.
NFR6	By using the application, the full expense report workflow is optimized from the current 20 minutes to **5 minutes** for both employees and finance staff.

(Continued on next page)

(Continued from previous page)

NFR	Acceptance Criteria
NFR7	The application's code implementation must use **interfaces** to interact with external services, place the concrete implementations in a separate layer, and adopt the **Dependency Inversion Principle**.
NFR8	The application's **business logic** code implementation must be **isolated** from the technical code implementation.

Table 11.3.2: Summary of non-functional requirements and their acceptance criteria. [Own creation]

Browser Development Tool

The satisfaction of NFR3 is verified using the performance metrics obtained from the browser development tools. The results are reported in the subsequent Table 11.3.3.

NFR	Performance Metric	Expected Time	Resulting Time
NFR3	Loading uploaded expenses in a new session	3 s	0.128 s
NFR3	Performing basic user interactions	500 ms	32 ms

Table 11.3.3: Performance evaluation of NFR3. [Own creation]

Manual Time Measurement

For the remaining non-functional requirements (NFRs) that rely on time-based acceptance criteria, measurements were conducted manually.

For NFR1, the elapsed time was calculated as the difference between timestamps recorded in the browser console. These timestamps correspond to the moment when the AI completes the expense processing and the moment when the expense status is updated in the user interface. They were generated through lightweight instrumentation code placed at the start and end of the status update process.

For NFR2 and NFR6, execution time was also measured manually. The reported values represent the average results obtained from 10 company employees who tested the application for the first time.

For NFR2, the measured time corresponds to the difference between the first and second execution of the complete expense report workflow by the same user, thereby capturing the time required to learn how to use the application. In contrast, NFR6 reflects the execution time recorded during the second attempt, excluding the initial learning phase. The results are summarized in Table 11.3.4.

During user testing, some cases exceeded the expected time thresholds due to a higher number of uploaded expenses or the execution of additional actions not previously performed. Uploading more expenses increased the time required to review and verify AI-extracted data. Similarly, users who explored additional functionalities during the second attempt took longer, reflecting higher engagement with the application.

Although these scenarios can be interpreted positively in terms of user adoption, they

introduced variability in the measurements and were therefore excluded from the final results.

NFR	Performance Metric	Expected Time	Resulting Time
NFR1	Updating expense processing status in real time	1 s	0. 478 s
NFR2	Learning the application usage curve	3 min	2 min 24 s
NFR6	Completing the full expense workflow	5 min	2 min 42 s

Table 11.3.4: Performance evaluation of NFR1, NFR2, and NFR6. [Own creation]

The measured time for the "Completion time of the full expense workflow" corresponds to an 86.5% efficiency gain compared to the 20 minutes previously required for manual expense data entry by employees. This result demonstrates that NFR6 has been successfully fulfilled, exceeding the initial target of a 75% efficiency gain.

Other NFR Acceptance Criteria Justification

Unlike the previous NFRs, which are demonstrated from an objective perspective, the remaining NFRs are attained indirectly. This is summarized in Table 11.3.5.

NFR	Fulfillment Justification
NFR2	During manual acceptance testing for this requirement, most users relied on the user manual for guidance. After uploading a receipt, the process proved ambiguous, as users were not aware that only manually reviewed AI-extracted expenses with the status "Completed" could be added to a report. In addition, most users did not know how to submit uploaded expenses for approval because the application requires them to create a report grouping those expenses before it can be sent to a reviewer. To address this issue, a detailed user guide pop-up was added to the user interface. As no further ambiguities were identified, this adjustment rendered the interface more intuitive for users.
NFR3	The application's resources are deployed through the Azure platform, therefore, they are cloud-based and are available 24 hours a day, 365 days a year. The deployment details are provided in the next section.
NFR4	The application's hybrid architecture is detailed in Section 9.1 and fulfills the defined acceptance criteria. Moreover, by deploying the application's resources on the Azure platform, these resources support automatic horizontal scaling.
NFR5	The fulfillment of this requirement is also justified in Section 14.1.
NFR7	The Dependency Inversion Principle and the "Programming to an Interface" pattern are applied throughout the application implementation and are stated in Section 9.2.2.
NFR8	The application's business logic code is implemented in a dedicated Domain layer following Domain-Driven Design; both concepts are explained in Section 9.2.2.

Table 11.3.5: Justification for other NFRs. [Own creation]

Fortunately, all specified requirements, both functional and non-functional, have been successfully satisfied, confirming that the application meets its intended goals and quality standards.

12 Deployment

This section describes the application's publishing process on the Microsoft Azure cloud platform. This process enables end-users to access the application. Azure's Platform-as-a-Service (PaaS) offerings simplify infrastructure management and provide a strong foundation for future automation. However, this initial version requires a manual deployment process.

12.1 Infrastructure Provisioning

The first step in the deployment process was to provision all necessary cloud infrastructure, as detailed in the Physical Architecture diagram (Figure 9.1.1). All Azure resources were created manually through the Azure Portal and include the following:

- **Azure App Service Plan and App Service**: To host the .NET backend API.
- **Azure SQL Database and Server**: For relational database persistence.
- **Azure Static Web App**: To host the frontend React application.
- **Supporting Services**: All other required services, such as Azure Storage Account (for Blob Storage), Azure Service Bus, Azure Event Grid, Azure Functions, and Azure SignalR, were also provisioned and configured.

By creating these resources, the application settings and secrets (e.g., database connection string and Azure AD application IDs) were securely configured. Instead of being stored in the source code, these values were added to the "Configuration" section of the Azure App Service and Azure Functions. This is a security best practice that allows sensitive information to be managed independently of the application code.

12.2 Backend Deployment

The .NET backend was deployed directly to the Azure App Service. The process consists of two steps:

1. **Publishing the Artifact**: The application is first published locally. This step is easily performed using Visual Studio's built-in "Publish" feature. This process involves compiling the C# code, resolving all dependencies, and gathering all necessary files into a single, optimized deployment package, a .zip file.
2. **Deploying to Azure**: The obtained package was then deployed to the App Service using the "Zip Deploy" method. This is a reliable and straightforward method to upload and deploy the application package and can be completed through various tools. In this case, Azure CLI was the tool used. Once the package is deployed, the App Service automatically unzips it and starts the application.

12.3 Frontend Deployment

The React frontend was deployed to the Azure Static Web Apps service. This service is specifically designed for modern web applications. The deployment process involved the following steps:

1. **Creating a Production Build**: The application was first built locally using the Node Package Manager (npm) command: `npm run build`. This command converts the TypeScript code into JavaScript, bundles all JavaScript modules, and optimizes assets such as CSS and images into an efficient build folder.

2. **Deploying to Azure**: Once the production-ready build folder was created, it was then deployed directly to the Azure Static Web Apps resource. This was performed using the Static Web Apps (SWA) Command-Line Interface (CLI) tool, which simplifies the deployment of static assets to the platform.

Although the first deployment was completed manually, the architecture's decoupled nature and the use of PaaS services are well-suited for seamless integration with Continuous Integration and Continuous Delivery (CI/CD) pipelines in future development cycles.

13 Project Management Results

Once the project execution is completed, the actual development progress is compared against the initially established timeline. For this purpose, an analysis is conducted to examine how the adopted methodology impacted project management. In addition, deviations from the initial Gantt diagram and budget, along with the subsequent adjustments and their consequences, are detailed.

13.1 Methodology

Moving away from traditional Scrum ceremonies to brief meetings every two days has been a beneficial decision for the project. This approach enables continuous project monitoring and eliminates traditional ceremonies that provide low value in a single-developer context.

Iterative meetings with the Product Owner (PO) allowed the project progress to stay on track and enabled the client's feedback to be received at an early stage. Possible delays were detected early, and new features were presented to the PO as soon as possible. At the same time, limiting meeting duration to 30 minutes helped focus on critical issues and prevented time overruns.

Regarding the WIP limit, it is not a restrictive constraint that blocks development progress. Quite the opposite, it has been a useful approach. Limiting the number of active tasks reduced the scope of development, allowing for a more dynamic project flow.

The adoption of ScrumBan for this project has been key to its successful execution so far. By combining selected characteristics of both Scrum and Kanban, this approach is well-suited to the project's needs. Thanks to this methodology, the project's actual execution has closely adhered to the scheduled timeline.

13.2 Project Timeline

For a better visualization of the comparison between the initial project timeline and the actual one, a comparative table is displayed in Table 13.2.1. Beyond that, a new Gantt diagram representing the actual execution progress is also created and corresponds to Figure 13.2.1.

ID	Name	Planned Effort (h)	Actual Effort (h)	Effort Variance (%)
PS	**Project Starting**	**50.00**	**50.00**	**0.00**
PS1	Context and Scope	20.00	20.00	0.00
PS2	Temporal Planning	10.00	10.00	0.00
PS3	Budget and Sustainability	10.00	10.00	0.00
PS4	Initial Documentation	10.00	10.00	0.00
PM	**Project Management**	**46.50**	**46.50**	**0.00**
PM1	Product Owner Meetings	30.50	30.50	0.00
PM2	Thesis Director Review Meetings	8.00	8.00	0.00
PM3	Product Backlog Update	8.00	8.00	0.00
FPD	**Final Product Delivery**	**56.50**	**106.00**	**87.61**
FPD1	User Manual Documentation	2.00	2.00	0.00
FPD2	Final Thesis Documentation	30.50	80.00	162.30
FPD3	Final Demonstration Video	4.00	4.00	0.00
FPD4	Bachelor's Thesis Defense	20.00	20.00	0.00
DT	**Design**	**16.00**	**16.00**	**0.00**
DT1	Use Case Diagram	2.00	2.00	0.00
DT2	Conceptual Model Diagram	4.00	4.00	0.00
DT3	Technology Stack Decision	2.00	2.00	0.00
DT4	Logic and Physic Architecture Design	4.00	4.00	0.00
DT5	UI Mockups Design	2.00	2.00	0.00
DT6	Security Design	2.00	2.00	0.00
Dev	**Development**	**86.00**	**78.00**	**-9.30**
Dev0	Self-learning and study of unfamiliar technologies	8.00	8.00	0.00
Dev1	Environment Configuration	2.00	2.00	0.00
Dev2	Product Deployment	8.00	20.00	150.00
Dev3	CI/CD Pipeline Implementation	20.00	28.00	40.00
Dev4	Logging and Monitoring	8.00	0.00	-100.00
Dev6	Git Repository Documentation	40.00	20.00	-50.00
Frontend and Backend Development		128.00	128.00	0.00
ER	**Expense and Report Development**	**64.00**	**64.00**	**0.00**
ER1	Expense Uploads	16.00	16.00	0.00
ER2	Uploaded Expenses Real-time Dashboard	4.00	4.00	0.00
ER3	AI-Extracted Expense Data Review and Correction	20.00	20.00	0.00
ER4	Uploaded Expenses Deletion	4.00	4.00	0.00
ER5	Expense Reports View, Creation, Edit, Submission, and Deletion	20.00	20.00	0.00
UA	**User Access Development**	**64.00**	**54.00**	**-15.63**
UA1	Company Email User Authentication	8.00	8.00	0.00
UA2	Role-based User Access	16.00	16.00	0.00
UA3	Submitted Report Approval and Rejection	8.00	16.00	100.00
UA4	Employee Spending Limit Configuration	8.00	0.00	-100.00
UA5	Report History Categorization by Status and User Role	4.00	4.00	0.00
UA6	Email Notification Service Integration	20.00	10.00	-50.00
BD	**Backend Development**	**120.00**	**120.00**	**0.00**
BD1	RESTful API Backend Logic Implementation	20.00	20.00	0.00
BD2	Azure SQL Database Integration and Manipulation	20.00	40.00	100.00
BD3	Azure Blob Storage Integration	8.00	8.00	0.00
BD4	Microsoft Document Intelligence AI Model Integration	20.00	10.00	-50.00
BD5	Automated AI-driven Data Extraction Pipeline	20.00	10.00	-50.00
BD6	Azure SignalR Real-time Notification Service Integration	16.00	16.00	0.00
BD7	Microsoft Entra ID User Authentication Integration	16.00	16.00	0.00
FD	**Frontend Development**	**10.00**	**8.00**	**-20.00**
FD1	Expenses Search and Filter	8.00	8.00	0.00
FD2	Reports Search	2.00	0.00	-100.00
Total		**513**	**542.5**	**5.75**

Table 13.2.1: Comparison of planned and actual effort per task, including effort variance. [Own creation]

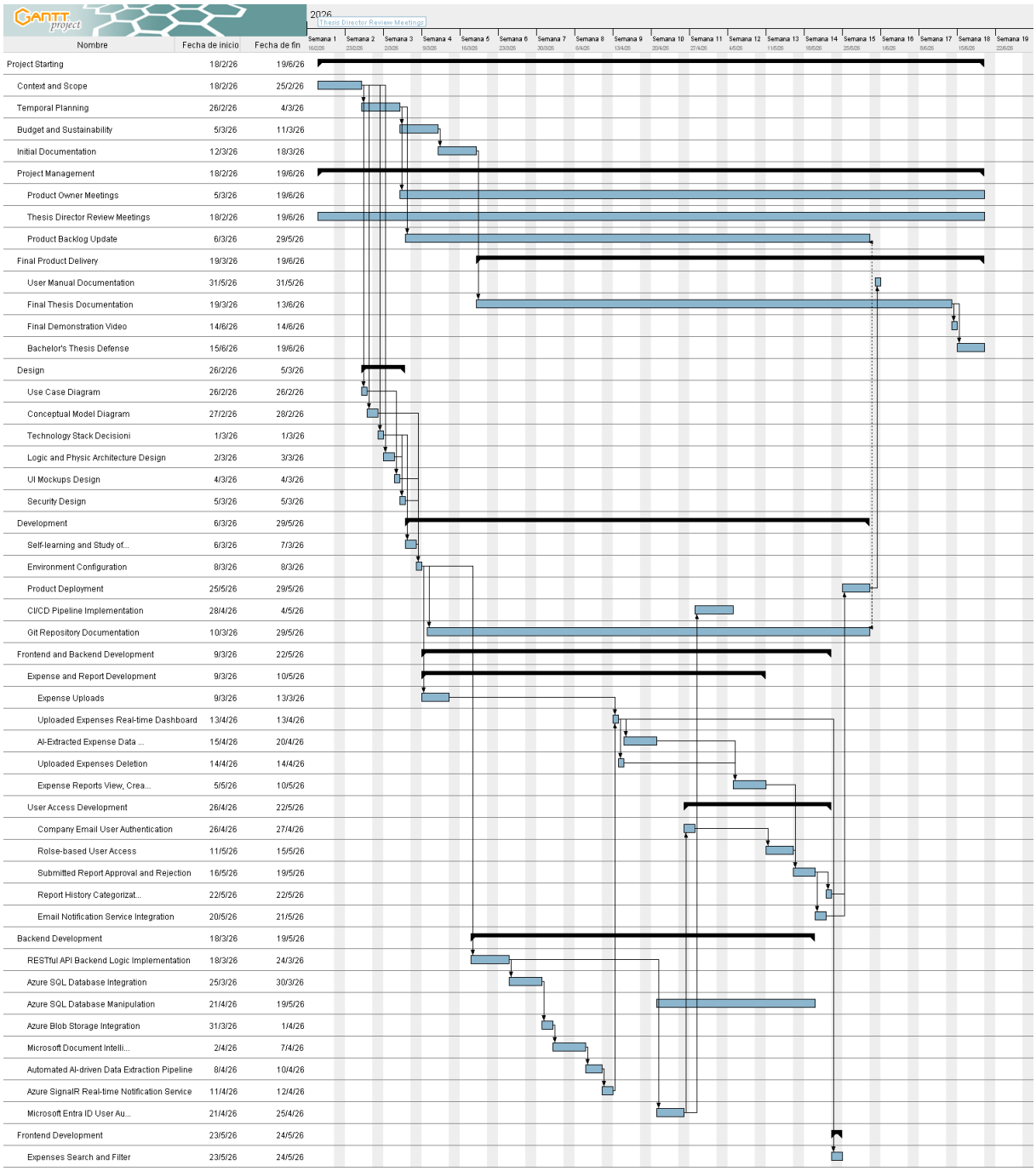

Figure 13.2.1: Actual project Gantt diagram. [Own creation]

Before diving into the details of the deviations between the planned and actual project execution, note that a minor adjustment was made to the **UA5** task. The original task name, "Submitted and Reviewed Expenses Records Visualization," was replaced with the more descriptive name "Report History Categorization by Status and User Role."

The **UA5** task was renamed because it was upgraded by the Product Owner during project execution. A more accurate description is that it involves developing a dashboard for each report status: for employees, dashboards showing "Not Sent" and "Submitted" reports, and for admins, dashboards showing "Pending Review" and "Reviewed" reports.

Looking at Table 13.2.1, the project clearly overspent its schedule: the initial plan was **513 hours**, but the actual effort reached **542.5 hours**. However, as shown in the Gantt diagram in Figure 13.2.1, the project was still executed within the planned **122 days**.

Comparing each task's planned time with its actual effort shows that some tasks exceeded their estimates while others required less time. The most significant underestimation occurred in task **FPD2** (final thesis documentation), which overran by 49.5 hours. The author acknowledges this clear underestimation of effort.

Other overrunning tasks were **Dev2** (product deployment), **UA3** (submitted report approval and rejection), and **BD2** (Azure SQL database integration and manipulation). These tasks required more effort than initially calculated. The root cause of this underestimation is the author's inexperience with database manipulation and product release.

Several tasks, however, were completed faster than expected: **Dev5**, **UA6**, **BD4**, and **BD5**. **Dev5** involved creating clear repository documentation to help future developers understand the project context. The originally estimated 40 hours were reduced by half.

UA6, **BD4**, and **BD5** tasks involved Azure resources. The initial approximation included a buffer because this was the author's first experience with cloud platforms. However, Azure Portal's intuitive interface and the straightforward handling of Azure resources through the .NET platform enabled efficient execution of these tasks.

Despite the time gained on some tasks, the overruns on others were larger, so the total execution time still exceeded the established effort. As explained in Section 5, this bachelor's thesis is organized for 540 total hours; the 27-hour buffer is insufficient, and actual execution has already surpassed the planned workload. To keep the project within budget, some tasks were discarded following the MoSCoW approach.

Tasks **UA4** and **FD2** are *Could have* functionalities with low priority. Omitting them does not significantly impact the final product value, so they are not included in the actual Gantt diagram. Another missing task in the Gantt diagram is **Dev4**, which involves logging and monitoring the application. It is also low-priority, and omitting it does not affect the product's final value.

During project execution, dedicating one week to **Dev3** task (CI/CD Pipeline Implementation) revealed a shortfall compared to the original approximation of five days due to the author's lack of expertise. The execution was already exceeding the planned allocation, the task is not high-priority and has no strong dependencies on other tasks.

Given these circumstances, **Dev3** was rescheduled to the final development stage and ultimately discarded. By the project's final stage, no buffer time remained for its completion, leaving the possibility to implement this task in future iterations.

Despite all the challenges encountered and the incomplete execution of the project, the final product fulfills all high-priority and medium-priority requirements (*Must have* and *Should have*) thanks to the MoSCoW strategy. As a result, an application that delivers optimal value within the timeline, as well as within the author's capabilities and expertise, has been delivered.

13.3 Budget

As shown in Table 13.2.1, some tasks differed from the initially scheduled effort. Consequently, the actual personnel cost may also diverge from the planned cost; the recalculated values are provided in Table 13.3.1.

			Hours per role						
ID	**Name**	**Total hours**	PM	D	T	SA	TW	**Cost (€)**
PS	**Project Starting**	**50**	**80**	**0**	**0**	**0**	**0**	**1,433.33**
PS1	Context and Scope	20	20	0	0	0	0	577.33
PS2	Temporal Planning	10	10	0	0	0	0	288.67
PS3	Budget and Sustainability	10	10	0	0	0	0	288.67
PS4	Initial Documentation	10	10	0	0	0	0	288.67
PM	**Project Management**	**46.5**	**38.5**	**46.5**	**0**	**0**	**0**	**2,035.17**
PM1	Product Owner Meetings	30.5	30.5	30.5	0	0	0	1486.37
PM2	Thesis Director Review Meetings	8	8	8	0	0	0	389.87
PM3	Product Backlog Update	8	0	8	0	0	0	158.93
DT	**Design**	**16**	**0**	**0**	**0**	**16**	**0**	**445.87**
DT1	Use Case Diagram	2	0	0	0	2	0	55.73
DT2	Conceptual Model Diagram	4	0	0	0	4	0	111.47
DT3	Technology Stack Decision	2	0	0	0	2	0	55.73
DT4	Logical and Physical Architecture Design	4	0	0	0	4	0	111.47
DT5	UI mockups Design	2	0	0	0	2	0	55.73
DT6	Security Design	2	0	0	0	2	0	55.73
FPD	**Final Product Delivery**	**106**	**0**	**5**	**0**	**0**	**101**	**2,206.87**
FPD1	User Manual Documentation	2	0	1	0	0	1	40.73
FPD2	Final Thesis Documentation	80	0	0	0	0	80	1,669.33
FPD3	Final Demonstration Video	4	0	4	0	0	0	79.47
FPD4	Bachelor's Thesis Defense	20	0	0	0	0	20	417.33
Dev	**Development**	**78**	**0**	**78**	**28**	**0**	**0**	**2,051.51**
Dev0	Self-learning and study of unfamiliar technologies	8	0	8	0	0	0	158.93
Dev1	Environment Configuration	2	0	2	0	0	0	39.73
Dev2	Product Deployment	20	0	20	0	0	0	397.33
Dev3	CI/CD Pipeline Implementation	28	0	28	28	0	0	1,058.18
Dev4	Logging and Monitoring	0	0	0	0	0	0	0.00
Dev5	Git Repository Documentation	20	0	20	0	0	0	397.33
Frontend and Backend Development		**128**	**0**	**128**	**0**	**0**	**0**	**2,542.93**
ER	**Expense and Report Development**	**64**	**0**	**64**	**0**	**0**	**0**	**1,271.47**
ER1	Expense Uploads	16	0	16	0	0	0	317.87
ER2	Uploaded Expenses Real-time Dashboard	4	0	4	0	0	0	79.47
ER3	AI-Extracted Expense Data Review and Correction	20	0	20	0	0	0	397.33
ER4	Uploaded Expenses Deletion	4	0	4	0	0	0	79.47
ER5	Expense Reports View, Creation, Edit, Submission, and Deletion	20	0	20	0	0	0	397.33
UA	**User Access Development**	**54**	**0**	**54**	**0**	**0**	**0**	**1,072.80**
UA1	Company Email User Authentication	8	0	8	0	0	0	158.93
UA2	Role-based User Access	16	0	16	0	0	0	317.87
UA3	Submitted Report Approval and Rejection	16	0	16	0	0	0	317.87
UA4	Employee Spending Limit Configuration	0	0	0	0	0	0	0.00
UA5	Report History Categorization by Status and User Role	4	0	4	0	0	0	79.47
UA6	Email Notification Service Integration	10	0	10	0	0	0	198.67
BD	**Backend Development**	**120**	**0**	**120**	**0**	**0**	**0**	**2,384.00**
BD1	RESTful API Backend Logic Implementation	20	0	20	0	0	0	397.33
BD2	Azure SQL Database Integration and Manipulation	40	0	40	0	0	0	794.67
BD3	Azure Blob Storage Integration	8	0	8	0	0	0	158.93
BD4	Microsoft Document Intelligence AI Model Integration	10	0	10	0	0	0	198.67
BD5	Automated AI-driven Data Extraction Pipeline	10	0	10	0	0	0	198.67
BD6	Azure SignalR Real-time Notification Service Integration	16	0	16	0	0	0	317.87
BD7	Microsoft Entra ID User Authentication Integration	16	0	16	0	0	0	317.87
FD	**Frontend Development**	**8**	**0**	**8**	**0**	**0**	**0**	**158.93**
FD1	Expenses Search and Filter	8	0	8	0	0	0	158.93
FD2	Reports Search	0	0	0	0	0	0	0.00

(Continued on next page)

(Continued from previous page)

ID	Name	Total hours	PM	D	T	SA	TW	Cost (€)
Total		**542.5**	**88.5**	**375.5**	**28**	**16**	**101**	**13,069.94**

Table 13.3.1: Actual personnel cost breakdown by task, role, and total personnel cost. [Own creation]

By exceeding the initial total project hours, the initial estimation of generic costs is adjusted to reflect the actual project cost presented in Table 13.3.2. Only costs derived from execution hours are recalculated, namely, amortization and electricity, but their deviation is minimal. The remaining costs are based on the total project days, which coincide with the planned days; consequently, no deviation is shown from the initial calculation.

Concept	Cost (€)
Amortization	108.00
Electricity	8.45
Internet	89.26
Office Rental	976.00
Software	57.25
Generic Cost (GC)	**1,239.06**

Table 13.3.2: Actual total generic cost. [Own creation]

Based on the actual personnel and generic costs, the total project cost is derived in Table 13.3.3. Contingency and incidental costs functioned as a buffer and are excluded from the final project cost calculation.

Concept	Cost (€)
Personnel Cost (PC)	13,069.94
Generic Cost (GC)	1,239.06
Total Budget	**14,309.00**

Table 13.3.3: Actual total cost of the project. [Own creation]

13.4 Project Deviations

With the actual project cost calculated, the deviation formulas defined earlier in Section 6.5 are applied to compute the project deviations across different areas. Table 13.4.1 below summarizes the cost deviations for human resources, amortization, and total cost.

Cost Category	Planned Cost (€)	Actual Cost (€)	Cost Deviation (€)
Human Resources	11,913.05	13,069.94	-1,156.89
Amortization	102.22	108.09	-5.87
Total Cost	14,592.78	14,309.00	283.78

Table 13.4.1: Project cost deviation analysis. [Own creation]

Although personnel and amortization costs exceeded the initial estimate, contingency and incidental costs covered the budget shortfall. As a result, the overall project cost remained within the allocated budget, thereby demonstrating that the original cost estimation was well-founded.

14 Regulatory and Legal Identification

Aside from satisfying the client's interests, the application's legal framework must also be taken into account. To this end, all regulations that the application must comply with are listed, along with a justification for compliance or non-compliance.

14.1 General Data Protection Regulation (GDPR)

This application is targeted at FSP's Barcelona office team. As part of European citizenship, this group is legally protected by the GDPR. Therefore, the application must comply with all applicable GDPR provisions in this project.

According to the data minimization principle (**Article 5(1)(c)**), the application must gather and store strictly essential information for the stated purpose. Any information that falls outside this scope must not be collected.

In this case, the application aims to manage employees' expenses. Hence, the application only stores relevant data to approve or reject the reported expenses. In addition, the collected user personal information is limited to the user's name and email, which are used exclusively for the email notification service.

Pursuant to **Article 6 of the GDPR**, any data processing must rest on a valid legal basis and pursue a lawful purpose. In this project, the data processing is based on contractual necessity, since the company expense reimbursement is a condition for fulfilling the employee's employment contract.

Following **Article 32 of the GDPR**, technical and organizational measures are required to guarantee data security. This is adequately addressed by applying several techniques. Secure communication between the client and the server is achieved using HTTPS, which encrypts data in transit. Data at rest is stored in Azure SQL Database and encrypted through Transparent Data Encryption, and receipt files are stored in Azure Blob Storage with private access.

Finally, to fully comply with **Article 32**, access to the application is protected through the authentication and authorization mechanisms provided by Azure Active Directory (Azure AD). User authorization is enforced based on role assignments contained in the issued security tokens, restricting each user to access only their own expenses and each administrator to access the expenses corresponding to their team. This approach ensures compliance with the principle of least privilege.

14.2 Intellectual Property (LPI)

This final degree project is also governed by the Intellectual Property Law (LPI) and the UPC regulations. As stated in **Article 10.1 of LPI**, the software application developed in this project, including source code, diagrams, and documentation, is considered a work protected by copyright. [13]

The LPI distinguishes between moral rights and economic rights. Moral rights, such as the right to be credited, are inalienable. Economic rights, on the other hand, are proprietary and can be assigned to third parties.

Given that this bachelor's thesis is under B modality (in collaboration with FSP), the UPC regulations approved in 2022 (**Acord CG/2022/05/23**) are also applied. As a consequence, the student retains moral rights, primarily the right to be credited, while the ownership of the intellectual property and the application's economic rights are assigned to FSP. This has been previously agreed upon by both parties, UPC and the company.

Moreover, the application's use of third-party software complies with their respective licenses. Permissive licenses such as the MIT license are used for React, .NET, and many other libraries. In addition, Azure services are used under Microsoft's terms of service. These licenses are compatible within a commercial application context.

15 Sustainability and Social Responsibility

The technological revolution has accelerated dramatically in recent decades. A clear example is the launch of the first telephone (1876), the first computer (1937), the first laptop (1974), the first smartphone (2007), and the first artificial intelligence (2017) [14].

This acceleration pace reveals that the time it takes for a technological invention to emerge and significantly impact society continues to shrink. Consequently, this exponential growth underscores the increasing importance of evaluating the economic, environmental, and social impacts of technological projects. These three dimensions together constitute sustainability, the topic discussed in this section.

Building on this trend, technology will continue shaping our future in the coming years. The primary mission of engineers is to make people's daily lives easier. However, developing a solution is not enough; a study of its sustainability effects is also a must.

Therefore, this section addresses sustainability questions, enabling a better evaluation of the project's impact across the economy, environment, and society. This analysis raises awareness of the weight of sustainability in project development.

15.1 Self-Assessment

Personally, the author acknowledges a limited level of expertise in sustainability, primarily focused on environmental footprint and social impact. The economic dimension, along with broader sustainability indicators, requires further study. The following subsection addresses a series of key sustainability questions within the scope of the author's current knowledge.

By engaging with the following sustainability assessment questions, the author's understanding of sustainability has been strengthened. Although several sustainability considerations were incorporated during the development phase, additional opportunities remain to further enhance the application's sustainability profile. Adopting more comprehensive and sustainable approaches would increase the overall value of the solution.

In the context of rapid technological advancement, sustainability is becoming increasingly significant. As a future software engineer, integrating sustainability considerations into system design and development contributes additional value to society. Technology should not only address everyday challenges but also support the transition toward a more sustainable future.

15.2 Economic Impact

15.2.1 Initial Milestone

Regarding PPP: Have you estimated the project execution cost?

The project budget is €14,592.78, as detailed in Section 6. This corresponds to the full "bill" for the application, including all personnel, hardware amortization, software licensing, and contingency costs.

Regarding Exploitation: How are currently solved economic issues (costs...) related to the problem that you want to address (state of the art)?

According to the problems stated in Section 1.3, the current time-consuming, low-efficiency expense management process delivers minimal value while misallocating company personnel costs for both employees and finance staff.

Regarding Exploitation: How will your solution improve economic issues (costs, etc.) with respect to other existing solutions?

The primary benefit of the proposed solution is the operational efficiency optimization. The current manual process requires approximately 225 hours per year, as estimated in Section 1.3. The application is designed to reduce the time required for this activity by at least 75%, in accordance with the non-functional requirement NFR6 from Section 3.2.2.

Based on this reduction and the actual personnel wages at FSP, the solution generates annual cost savings of approximately €5,062.5. Given the estimated development cost of €14,592.78, the return on investment (ROI) is projected to be achieved within 3 years. This demonstrates that the proposed solution is more economically viable than investing in external solutions, supporting its adoption as a sustainable long-term approach.

However, an initial budget misestimation could extend the ROI period. This is mitigated with the 27-hour built-in buffer (calculated in Section 5) and the MoSCoW prioritization strategy (employed in Section 3.2.1). As a result, a high-value product is most likely to be delivered, since *Could have* requirements can be postponed in case of budget shortfalls.

15.2.2 Final Milestone

Regarding PPP: Have you quantified the cost (human and material resources) of undertaking the project? What decisions have you taken to reduce the cost? Have you quantified these savings?

The final project cost has been quantified and amounts to €14,309.00, as detailed in Section 13.3. This figure includes actual personnel costs derived from the effort allocated to each task, as well as generic costs adjusted to the total of 542.5 hours of work.

During project execution, several critical decisions were made to allocate resources within the available budget while maintaining a high-value product despite time constraints. The MoSCoW prioritization technique was adopted to achieve this objective. As the project exceeded its planned effort, low-priority tasks, specifically those categorized as *Could have* requirements, were excluded.

The omitted tasks were UA4 (Employee Spending Limit Configuration), FD2 (Reports Search), and Dev3 (CI/CD Pipeline Implementation). This exclusion generated cost savings of €356.59 (extracted from the costs indicated in Table 13.3.1), which proved essential in keeping the project within the initial budget.

Regarding PPP: Is the expected cost similar to the final cost? Have you justified any differences (lessons learnt)?

The final project cost remains close to the initially estimated cost. The estimated cost was €14,592.78, while the final actual cost was €14,309.00, resulting in a deviation of €283.78 below the initial estimate.

An analysis of the deviation values presented in Section 13.4 indicates that the primary discrepancy occurred in personnel costs. An overrun of 29.5 hours led to an additional cost of €1,156.89. A key lesson learned is the underestimation of tasks involving extensive documentation and the author's limited proficiency in database manipulation, particularly in tasks FPD2 and BD2.

Conversely, tasks related to the integration of cloud platform resources were completed more efficiently than expected, largely due to the platform's intuitive interface and straightforward integration processes. Ultimately, the overall budget remained balanced through the inclusion of contingency and incidental costs, which offset the personnel cost overrun.

Regarding Exploitation: What cost do you estimate the project will have during its useful life? Could this cost be reduced to increase viability?

The operational cost during the application's useful life primarily consists of Azure's monthly service fees, as the system relies entirely on cloud-based resources. No server maintenance or hardware replacement costs are required. The monthly Azure cost depends on resource usage; however, Azure provides a range of pricing tiers that allow scaling according to application demand.

At present, the application operates under the most basic subscription tier. Consequently, further cost reduction is not feasible. Nonetheless, the current strategy leverages several cloud-native advantages.

Currently, the application's resources meet performance and demand requirements while remaining within the most cost-effective pricing tier. Additionally, the asynchronous processing pipeline described in the physical architecture (Section 9.1) is serverless and incurs costs only when executed. This characteristic of Azure Functions converts it particularly suitable for asynchronous workflows.

Regarding Exploitation: Have you considered the cost of adaptations, updates, and repairs during the project's useful life?

Future maintenance, including adaptations, updates, and repairs, was a primary consideration during the design of the application's logical architecture. The adoption of Clean Architecture and SOLID principles directly addresses maintainability concerns.

The application's core business logic resides in a dedicated Domain layer, isolated from technical implementation details. Interactions with external systems are defined through abstract interfaces, with their implementations located in the Infrastructure layer.

A representative scenario illustrating the benefits of this approach is a change in the email notification service or database system. In such cases, only the Infrastructure layer components implementing the relevant interfaces need to be modified or replaced, while the remaining layers remain unaffected. Additional design patterns that further support maintainability are discussed in Section 9.2.2.

Regarding Risks: Could situations occur that are detrimental to the project's viability?

Several risks may affect the project's long-term economic viability, which depends on achieving the projected annual cost savings of €5,838.75, based on the actual 86.5% efficiency gain derived from the validation of NFR6 in Section 11.3.2.

- **Low user adoption**: Resistance from employees to adopting the new system may prevent the realization of expected efficiency gains, thereby extending the ROI period. This risk is mitigated through the development of a user manual (Task FPD1) and the design of an intuitive user interface (NFR2).
- **Uncontrolled cloud costs**: Unexpected increases in application usage without proper monitoring could lead to escalating Azure costs, negatively affecting the projected ROI. This risk is addressed by implementing cost alerts and regularly reviewing service tiers, particularly during periods of increased activity.
- **Increased maintenance costs**: The omission of Task Dev4 (Logging and Monitoring) introduces technical debt. The absence of a robust monitoring solution may result in time-consuming and costly debugging processes in production environments, potentially leading to system downtime and reduced user productivity.

15.3 Environmental Impact

15.3.1 Initial Milestone

Regarding PPP: Have you estimated the project's environmental impact?

The only resource directly affecting the environmental footprint is energy consumption. As shown in Table 6.2.2, the total energy consumption is 45.73 kWh. Using a conversion factor of 0.283 kg CO_2/kWh [15], the project's carbon footprint is around 12.94 kg CO_2.

Regarding PPP: Have you considered minimizing its impact, for example, by reusing resources?

The project's environmental impact is minimized by conducting the full project development in FSP's shared office, reusing existing lighting and preventing unnecessary energy consumption.

Regarding Useful Life: How is the problem that you want to address currently solved (state of the art)?

The current employees' annual effort spent on manual expense management is 225 hours, corresponding to an estimated energy consumption of 12.63 kWh (assuming a PC power consumption of 65 W). As the proposed solution reduces the required effort by 75%, the associated energy consumption is expected to decrease by around 3.16 kWh. In addition, digitalizing the process reduces the environmental impact associated with printing, transportation, and the physical handling of expense records.

Moreover, the application uses Azure for software resources. The Azure resources required for application development cost less than traditional physical resources. This cost optimization results from a significant reduction in resource consumption. At the same time, cloud resources do not require physical transportation or hardware replacement, further reducing electronic waste.

Nevertheless, suboptimal code may lead to excessive CPU and memory consumption, resulting in a negative environmental impact on the project (as defined in SO5 from Section 3.1). Therefore, adherence to high-quality coding practices is emphasized throughout the project development. As a result, a more sustainable application is delivered, using only essential resources and minimizing overall consumption throughout its lifecycle.

Regarding Useful Life: How will your solution improve the environment with respect to other existing solutions?
Developing an internal application tailored to the company's scale is inherently more resource-efficient than relying on external parties, since only useful functionalities are implemented.

15.3.2 Final Milestone

Regarding PPP: Have you quantified the environmental impact of undertaking the project? What measures have you taken to reduce the impact? Have you quantified this reduction?

The final environmental impact of the project's development phase has been adjusted based on the actual execution time. The total effort of 542.5 hours, as detailed in Section 13.2, corresponds to an energy consumption of 48.36 kWh. Using a conversion factor of 0.283 kg CO_2/kWh, the resulting carbon footprint for the development phase (PPP) is approximately 13.69 kg CO_2.

Both implicit and explicit measures were implemented to minimize environmental impact. First, existing resources were reused by developing the application at the FSP Barcelona office, where infrastructure such as lighting, heating, and networking is shared among employees. Second, the use of cloud-hosted resources (Microsoft Azure) enabled access to energy-efficient computing infrastructure, resulting in minimal baseline power consumption.

Although the exact reduction is difficult to quantify, there is clear evidence that operating within a shared environment significantly reduces environmental impact compared to deploying a dedicated, single-purpose infrastructure.

Regarding PPP: If you carried out the project again, could you use fewer resources?

A retrospective analysis indicates that fewer human hours, and therefore less energy, could have been consumed. Several tasks, particularly BD2 (Database Integration) and FPD2 (Documentation), exceeded initial estimates due to the author's limited experience.

If the project were undertaken again, the experience gained would likely reduce the time required for these tasks, leading to lower energy consumption and a reduced carbon footprint. Furthermore, improved estimation accuracy would enhance project planning and potentially prevent the omission of tasks such as Dev3 (CI/CD Pipeline Implementation). The inclusion of such automation could further streamline development processes, particularly during Dev2 (Deployment), thereby improving overall efficiency.

Regarding Exploitation: What resources do you estimate will be used during the useful life of the project? What will be the environmental impact of these resources?

During the application's operational phase, the primary resource consumption will be the electrical energy required to run Microsoft Azure services. Unlike on-premises solutions, this approach eliminates the need for direct hardware maintenance, cooling systems, and physical infrastructure. Therefore, the environmental impact is primarily determined by the energy consumption of Azure data centers.

Microsoft has committed to sustainability goals, including the use of 100% renewable energy for its data centers since 2025. [16] As the application is hosted on Azure, it indirectly benefits from these initiatives, resulting in a significantly lower carbon footprint compared to traditional on-premises hosting.

The application's environmental impact is directly proportional to its usage and represents only a negligible fraction of total data center consumption. The most computationally intensive process, AI-based data extraction from uploaded receipts, is executed through serverless Azure Functions, which consume energy only when triggered. This event-driven model further minimizes unnecessary energy usage.

Regarding Exploitation: Will the project enable a reduction in the use of other resources? Overall, does the use of the project improve or worsen the ecological footprint?

The project reduces resource consumption by automating expense management. As shown by the fulfillment of NFR6 (Section 11.3.2), the application reduces manual effort by approximately 86.5%, which also lowers the energy consumed in processing tasks and improves the overall ecological footprint.

Regarding Risks: Could situations occur that could increase the project's ecological footprint?

There are many risks that may negatively impact the project's environmental performance during its operational life:

- **Inefficient code and queries**: Suboptimal implementation or poorly designed database queries may result in excessive CPU and memory usage within Azure services. This increases both operational costs and energy consumption. This risk is mitigated by adhering to high-quality coding practices, as defined in the project's objectives (SO5).

- **Uncontrolled scalability**: Improper scaling strategies may lead to over-provisioning of resources, such as selecting unnecessarily large database tiers. This results in wasted computational resources and increased energy usage. This risk is managed through continuous monitoring of resource utilization using Azure's built-in tools and adjusting service tiers accordingly.
- **Data growth**: Uncontrolled growth in stored data, particularly large receipt files in Azure Blob Storage, may increase storage requirements and associated energy consumption. Although cloud storage is highly optimized, implementing a data retention policy (e.g., archiving or deleting outdated data after a defined period) would mitigate this risk.

15.4 Social Impact

15.4.1 Initial Milestone

Regarding PPP: What do you think you will achieve, in terms of personal growth, from doing this project?

This project enables the author to apply prior academic knowledge and software development skills in a real-world context. As a result, it enriches their working experience as future software engineer. Furthermore, it is the author's first experience managing and leading an entire project independently, strengthening self-learning, self-management skills and the ability to build software projects from scratch.

Regarding Useful Life: How is the problem that you want to address currently solved (state of the art)?

The current expense management process is handled manually through traditional methods. This involves manually entering data into Excel spreadsheets, emailing them to the finance department, and reviewing them line by line.

Regarding Useful Life: How will your solution improve the quality of life (social dimension) with respect to other existing solutions?

The proposed solution optimizes time spent on expense management while reducing employees' frustration and fatigue. This frees employees from this tedious task, allowing additional time to focus on more meaningful work. As a result, it maximizes employees' productivity.

However, negative effects are also encountered with the proposed application. An initial resistance to change and a steep learning curve are expected. In addition, a non-intuitive UI would exacerbate the adoption process. To mitigate the last risk, the Product Owner is actively involved in task DT5, a user manual guide is required as part of task PM4 (both defined in Section 5.1), and compliance with NFR2 from Section 3.2.2 is emphasized.

Another potential risk is excessive reliance on AI extraction, leading employees to poorly review the AI-extracted data, increasing the risk of errors. To mitigate this, the workflow is redesigned so that reports cannot be submitted until employees have explicitly reviewed the extracted information.

Regarding Useful Life: Is there a real need for the project?
Regarding all project sustainability impacts (social, environmental, and economic), there is a clear need to build a robust solution. This solution overcomes challenges from all involved parties while introducing a sustainability-focused approach.

15.4.2 Final Milestone

Regarding PPP: Has undertaking this project led to meaningful reflections at the personal, professional, or ethical level among the people involved?

At a personal level, managing the project from conception to delivery significantly strengthened self-discipline, time management, and problem-solving skills. Overcoming unforeseen technical challenges and adapting the project plan to deviations fostered resilience and encouraged a more pragmatic approach to achieving objectives.

At a professional level, the project provided practical experience within the software engineering domain. The author gained hands-on experience with modern cloud technologies (Microsoft Azure), enterprise architectural patterns (Clean Architecture), and agile prioritization techniques (MoSCoW). A key learning outcome was the importance of accurate time estimation and the need to balance technical ambition with delivery constraints, as illustrated by the decision to exclude the CI/CD implementation.

At an ethical level, the project involved handling sensitive financial data, requiring a strong focus on security. Measures such as robust authentication via Azure Active Directory and data encryption at rest were implemented. This experience highlighted the ethical responsibility of protecting user data and privacy, and reinforced regulatory considerations such as GDPR compliance as concrete design constraints.

Regarding Exploitation: Who will benefit from the use of the project? Could any group be adversely affected by the project? To what extent?

The primary beneficiaries are company employees and finance or administrative staff. Employees benefit from a substantial reduction in the time and effort required to manage and submit expense reports, allowing greater focus on core responsibilities and improving productivity.

Finance staff benefit from a centralized and digitalized approval workflow. The system enforces data consistency, reduces manual errors, and provides a clear audit trail, thereby improving the reliability and efficiency of the reimbursement process.

No group is expected to be adversely affected in the long term. However, during the adoption phase, some users may experience difficulties adapting to the new system. Resistance to change or initial learning barriers may temporarily reduce productivity. These risks are mitigated through the provision of a user manual (Task FPD1) and the design of an intuitive user interface (NFR2), both validated during the testing phase.

Regarding Exploitation: To what extent does the project solve the initially defined problem?

The project successfully addresses the initial problem of a time-consuming, inefficient, and error-prone manual expense management process.

The delivered solution achieves this through several key improvements:

- **Automated data entry**: Integration of an AI model enables automatic extraction of relevant information from receipts, fulfilling requirement FR3.
- **Centralized workflows**: The application provides a unified platform for expense submission, report creation, and approval management, replacing fragmented tools such as spreadsheets and email.
- **Improved efficiency**: The digitalization of the process establishes the foundation for achieving the actual 86.5% efficiency improvement (NFR6).

Although some lower-priority features, such as advanced report search (FD2) and spending limits (UA4), were not implemented due to time constraints, all *Must have* and *Should have* requirements were delivered. Consequently, the solution provides significant value and effectively optimizes the company's expense management process.

Regarding Risks: Could situations occur in which the project adversely affects a specific population segment?

While the application is designed to benefit most users, certain groups may experience temporary challenges. Employees less comfortable with digital tools, despite working in a technical environment, may find it difficult to adapt, particularly if they are accustomed to manual processes. Insufficient onboarding or support could lead to short-term frustration and reduced productivity.

Additionally, the system's reliance on an active internet connection may disadvantage users operating in environments with limited or unstable connectivity. Unlike manual processes that allow offline work, this constraint may disrupt workflows and require users to relocate to areas with better network access.

Regarding Risks: Could the project create any kind of dependency that puts users in a weak position?

The project introduces a dependency on Microsoft Azure and the Azure AI Document Intelligence service, leading to two primary risks:

- **Service downtime**: If critical Azure services experience outages, the application may become temporarily unavailable. While this risk is mitigated by Azure's high availability guarantees, it cannot be entirely eliminated. Currently, the impact is limited due to the relatively small user base (approximately 25 employees in the Barcelona office and 4 finance staff). However, this risk will become more significant if the system is scaled to additional locations.
- **Vendor lock-in**: The application relies heavily on Azure services, making the company dependent on Microsoft's technology and pricing model. Potential changes in pricing or service availability could create challenges. This risk is mitigated through the adoption of Clean Architecture, which isolates core business logic from infrastructure concerns. For example, the dependency on the AI service is abstracted through the IDocumentAnalysisService interface, allowing alternative implementations to be introduced with minimal impact on the overall system.

16 Conclusions

This final chapter synthesizes the outcomes of the project, evaluating the extent to which the initial objectives have been achieved and reflecting on the knowledge acquired throughout the development lifecycle. It includes an assessment of the technical competencies developed, presents overall conclusions regarding the project's success, and outlines recommendations for future work.

16.1 Integration of Technical Knowledge

Since this bachelor's thesis belongs to the software engineering specialization, the knowledge acquired in various specialization subjects is applied throughout the project lifecycle. Below is a list of each of these subjects, along with their specific contributions to this project

Web Services and Applications (ASW)

Provided that the project is centered on a web application, this is undoubtedly the first subject to address. The key outcome of this subject is a solid understanding of RESTful API design. At the same time, hands-on experience with Ruby on Rails has strengthened the author's skills in database management and migrations. Finally, the introduction of the Swagger tool has also improved API testing and documentation. As a result, these concepts have been actively applied throughout the project.

Software Engineering Projects (PES)

PES is also a key component of this project. This subject gave the author the first opportunity to develop a software application from the ground up, covering all necessary phases of the software lifecycle. Through this experience, the subject highlights that software engineering goes beyond coding, incorporating essential planning and management activities to ensure project success.

On top of that, it enabled the practical application of Agile methodologies, which fostered pragmatic habits such as continuous documentation and regular updates to the product backlog. Additionally, it provided the opportunity to build a decoupled frontend–backend application and work with unfamiliar frameworks, significantly strengthening the author's expertise in React.

The introduction of new tools has also been beneficial for the author and has therefore enabled their application in this project. For example, editor extensions that support coding best practices and Git version control.

Software Project Management (GPS)

GPS is the first subject that introduced the author to agile methodology concepts, one of the most relevant approaches in the current software market. It also taught how to create an initial timeline and budget estimate. Furthermore, it covered other common software project concepts, such as risk analysis and the Gantt diagram.

Requirements Engineer (ER)

The project's context study, requirements analysis, and use case diagrams are all documented in accordance with the concepts introduced in this specific subject.

Software Engineering Introduction (IES) and Software Architecture (AS)

These two subjects introduced the basics for building UML diagrams and designing layered architecture.

Database (BD) and Database Design (DBD)

As the name suggests, these subjects helped the author design a secure and optimized database, write efficient queries, and become familiar with SQL databases.

Cybersecurity Management (GCS)

The theory presented in this subject raised the author's awareness of the wide variety of cyberattack strategies and underscored the importance of preventing any foreseeable cybersecurity risks.

Informatics' Social and Environmental Aspects (ASMI)

While the previous subjects provided more technical knowledge, this one focuses on non-technical aspects. Specifically, it has significantly enriched the project's sustainability analysis.

16.2 Objective Achievement

This section evaluates the project's success by assessing the degree to which each of the initial sub-objectives, defined at the outset, has been fulfilled.

SO1. Design the application's technology stack and the database structure.

Status: Fully achieved

Evidence: A complete technology stack was defined and justified in Section 9.1, including .NET 8 for the backend, React with TypeScript for the frontend, and Microsoft Azure as the cloud platform. The database structure was thoroughly designed and documented in Section 9.6, and implemented using Azure SQL Database with Entity Framework Core following a Code-First approach.

SO2. Develop a full-stack web application.

Status: Fully achieved

Evidence: A complete end-to-end application was successfully developed. It includes a fully functional frontend interface for managing expenses and reports, as well as a robust backend API responsible for business logic, data persistence, and security. The delivered system satisfies all *Must have* and *Should have* functional requirements.

SO3. Design a scalable pipeline to automate AI-driven data extraction from expense uploads.

Status: Fully achieved

Evidence: The asynchronous processing pipeline, described in the Physical Architecture (Section 9.1) and illustrated in the sequence diagrams (Figure 9.4.3), represents a key contribution of the project. The event-driven architecture, leveraging Azure Blob Storage, Event Grid, Service Bus, and Azure Functions, decouples AI processing from user requests, ensuring scalability and responsiveness.

SO4. Use a cloud infrastructure platform to manage external application resources.

Status: Fully achieved

Evidence: The application is fully built and deployed on the Microsoft Azure cloud platform. As detailed in the Physical Architecture and Implementation sections, the solution makes extensive use of Platform-as-a-Service (PaaS) resources. This approach reduces infrastructure management overhead while providing scalability, security, and reliability, supporting the development of a cloud-native solution.

SO5. Maintain a clean, reusable, maintainable, and integration-friendly application codebase.

Status: Fully achieved

Evidence: The project consistently applies Clean Architecture principles, along with SOLID and other design patterns, as described in Section 9.2.2. The resulting codebase is highly decoupled, with clear separation between the Domain, Application, and Infrastructure layers. This structure facilitates maintainability and extensibility, fulfilling non-functional requirements related to integration (NFR7) and maintainability (NFR8). Code quality was further supported by a multi-layered testing strategy, achieving 78% code coverage.

16.3 Technical Competency Achievement

This project has served as a practical means for the development and consolidation of several key technical competencies within the field of Software Engineering. This section evaluates the extent to which each targeted competency has been achieved, supported by evidence from the project lifecycle.

CES2.1: Define and manage the requirements of a software system. [In-depth]

This competency was achieved at an in-depth level. A structured process was conducted to gather and analyze system requirements in collaboration with the Product Owner, resulting in a comprehensive set of functional and non-functional requirements (Section 3.2). These requirements were continuously prioritized using the MoSCoW methodology, which proved essential for scope management and for delivering a high-value product despite time constraints.

CES1.1: Develop, maintain, and evaluate complex and/or critical software systems and services. [Substantially]

This competency was substantially achieved through the end-to-end development of the full-stack application. The design and implementation of an asynchronous, event-driven

processing pipeline (Section 9.1, Physical Architecture) exemplify the development of a complex system. Additionally, the adoption of Clean Architecture and SOLID principles (Section 9.2, Logical Architecture) supports maintainability and long-term system evolution.

CES1.7: Control quality and design tests in software production. [Substantially]

Quality assurance was maintained throughout the project lifecycle. Static code analysis tools were initially integrated to monitor code quality (Section 11.1). Subsequently, unit and integration tests (Section 11.2) were implemented to validate functionality and component interactions. Test coverage was quantitatively assessed, achieving 78% code coverage. Finally, manual acceptance testing ensured compliance with defined requirements, reflecting a comprehensive approach to quality control.

CES1.3: Identify, evaluate, and manage potential risks associated with software construction. [Substantially]

This competency was substantially achieved, as demonstrated in the Risk Management section (Section 5.5). Key risks, including tight deadlines and limited experience with certain technologies, were identified early. Mitigation strategies, such as incorporating time buffers and allocating incidental costs within the project budget (Section 6.3), were defined at the project outset. The application of MoSCoW prioritization further supported effective risk management, enabling successful project completion.

CES1.9: Demonstrate an understanding of the management and governance of software systems. [Substantially]

This competency is evidenced by comprehensive project planning and monitoring activities. These include the development of a work breakdown structure (Table 5.3.1), time planning through a Gantt chart (Figure 5.4.1), and detailed budget estimation (Section 6). Governance was ensured through regular review meetings with the Product Owner and Thesis Director, maintaining alignment with project objectives. The comparative analysis carried out on the Project Management Results (Section 13) between planned and actual outcomes further reflects a mature understanding of project management.

CES2.2: Design appropriate solutions in one or more application domains, using software engineering methods that integrate ethical, social, legal, and economic aspects. [Partially]

This competency was partially achieved. The sustainability analysis (Section 15) evaluates the project's economic, environmental, and social dimensions, demonstrating consideration of these factors in the solution design. For instance, economic viability (ROI) and social impact (reduction of manual effort) were key drivers. However, certain aspects, particularly deeper legal and ethical considerations such as comprehensive data retention policies, were beyond the project scope, resulting in partial fulfillment of this competency.

16.4 Project Conclusions

By this stage, this project constitutes the most significant challenge the author has undertaken. Prior to its initiation, certain phases addressed in this work had been previously experienced within academic coursework in collaborative team settings. However, this project marks the author's first comprehensive engagement with a complete end-to-end development process, encompassing requirements elicitation, project scheduling, budget estimation, system implementation, execution analysis, and final validation.

Throughout the project lifecycle, the author has acknowledged substantial personal and professional growth developed over four academic years at UPC. The continuous adherence to the UPC's rigorous academic standards has enabled the author to acquire the competencies required to carry out the project independently while effectively overcoming ongoing challenges.

Beyond demonstrating fundamental engineering skills and learning how software projects are managed in a professional context, it has offered a valuable opportunity to integrate and consolidate key capabilities developed throughout the author's academic career. These include self-learning, adaptability, critical thinking, problem-solving, and continuous personal development.

In conclusion, this project rises above the scope of a conventional academic assignment. It has strengthened the author's confidence in addressing challenges beyond the academic environment, fostering a greater sense of independence in a professional context and providing a clearer vision of the engineer the author aspires to become. It also reflects the level of preparedness and proficiency expected from a UPC graduate when delivering high-quality results under pressure and within constrained time and resource conditions.

16.5 Future Work and Recommendations

Although the current version of the application fulfills all high-priority requirements and delivers substantial value, the development process has revealed several opportunities for further enhancement. This section outlines recommendations for future iterations aimed at improving functionality, robustness, and operational efficiency.

1. **Implement a Full CI/CD Pipeline (High Priority)**

   A key next step is the implementation of a complete Continuous Integration and Continuous Delivery (CI/CD) pipeline, as initially defined in Task Dev3. Automating build, testing, and deployment processes, using tools such as GitHub Actions, would provide multiple benefits:

   - **Increased Reliability**: Automated execution of unit and integration tests for each code change would significantly reduce the risk of regressions.
   - **Faster Delivery**: Automated deployment to Azure would eliminate manual and error-prone steps, enabling more frequent and reliable releases for new feature implementations and bug fixes.

- **Improved Code Quality**: Integration of static code analysis within the pipeline would enforce code quality standards consistently.

2. **Enhance Monitoring and Logging (High Priority)**

   The implementation of Task Dev4 (Logging and Monitoring) is another high-priority recommendation. While Azure provides basic monitoring metrics, integrating Azure Application Insights into the application code would enable deeper visibility into system performance and user behavior. This includes:

   - **Structured logging**: Enabling advanced diagnostics for identifying errors and performance bottlenecks through queryable logs.
   - **Creating Custom Dashboards and Alerts**: Supporting proactive monitoring of application health and early detection of issues, such as high failure rates in AI processing tasks.

3. **Develop *Could Have* Features**

   With core functionalities completed, future work may focus on implementing deferred features categorized as *Could have* requirements. The remaining tasks include FD2 (Advanced Report Search) and UA4 (Employee Spending Limits), both of which would enhance usability and control.

4. **Improve User Onboarding and Support**

   To facilitate user adoption and reduce support requirements, the existing user manual (Task FPD1) could be extended into an interactive, in-application guidance system. Features such as guided tours and context-sensitive help would reduce the learning curve and improve overall user experience.

5. **Explore Data Archiving and Retention Policies**

   As the application evolves, data volume in Azure SQL Database and Blob Storage will increase. However, long-term retention of receipt data may not always be necessary, leading to inefficient storage usage and increased costs. Implementing data retention and archiving policies would allow older data to be migrated to lower-cost storage tiers or removed when no longer required, thereby optimizing resource utilization.

References

[1] Jun Yin Zhou. *Functional and Non-Functional Requirements*. Concordia University, 2004. URL: https://spectrum.library.concordia.ca/id/eprint/7991/1/MQ91163.pdf.

[2] M. Alqudah and R. Razali. "An Empirical Study of Scrumban Formation based on the Selection of Scrum and Kanban Practices". In: *Int. J. Adv. Sci. Eng. Inf. Technol.* 8.6 (Dec. 2018), pp. 2315–2322.

[3] *Salary in Spain*. Talent.com. 2026. URL: https://es.talent.com/salary (visited on 03/05/2026).

[4] *Dell Pro 16 Laptop*. Dell. 2026. URL: https://www.dell.com/es-es/shop/port%C3%A1tiles-de-dell/port%C3%A1til-dell-pro-16/spd/dell-pro-pc16250-laptop (visited on 03/05/2026).

[5] *Logitech M171 Wireless Mouse*. Logitech. 2026. URL: https://www.logitech.com/es-es/shop/p/m171-wireless-mouse.910-006867 (visited on 03/05/2026).

[6] *Dell Pro 24 Plus P2425H Monitor*. Dell. 2026. URL: https://www.dell.com/es-es/shop/monitor-dell-pro-24-plus-p2425h/apd/210-bmff/monitores-y-accesorios (visited on 03/05/2026).

[7] *Logitech K120 USB Keyboard*. Logitech. 2026. URL: https://www.logitech.com/es-es/shop/p/k120-usb-standard-computer (visited on 03/05/2026).

[8] *Electricity Price in Spain*. Tarifa Luz Hora. 2026. URL: https://tarifaluzhora.es/ (visited on 03/05/2026).

[9] *Orange Fiber Offer*. Orange. 2026. URL: https://www.orange.es/ofertas-tarifas/clientes/oferta-fibra-adicional/ (visited on 03/05/2026).

[10] *Azure Pricing*. Microsoft. 2026. URL: https://azure.microsoft.com/en-us/pricing/ (visited on 03/05/2026).

[11] *Microsoft 365 Pricing*. Microsoft. 2026. URL: https://www.microsoft.com/es-es/microsoft-365/buy/compare-all-microsoft-365-products (visited on 03/05/2026).

[12] Craig Larman. *Applying UML and Patterns. An Introduction to Object-Oriented Analysis and Design*. Prentice Hall, 2005.

[13] *Modificació de la normativa sobre els drets de la propietat industrial i intel · lectual de la UPC*. Universitat Politècnica de Catalunya (UPC) - RDI. 2022. URL: https://rdi.upc.edu/ca/area/estructura-i-persones/10-modificacio-de-la-normativa-sobre-els-drets-de-la-propietat-industrial-i-intel-lectual-de-la-upc-1.pdf (visited on 06/13/2026).

[14] *History of Technology Timeline*. Encyclopaedia Britannica. 2026. URL: https://www.britannica.com/story/history-of-technology-timeline (visited on 03/05/2026).

[15] *Nuevos factores de emision*. ECODES. 2024. URL: https://ecodes.org/hacemos/cambio-climatico/mitigacion/ceroco2/nuevos-factores-de-emision-2024-lo-que-necesitas-saber-para-calcular-tu-huella-con-rigor (visited on 03/17/2026).

[16] Microsoft News Center. *6 Projects That Helped Microsoft Meet Its Renewable Energy Goal*. Microsoft. 2022. URL: https://news.microsoft.com/source/features/sustainability/6-projects-that-helped-microsoft-meet-its-renewable-energy-goal/ (visited on 06/15/2026).

Appendices

A Detailed Use Case Specifications

This appendix provides the complete and exhaustive technical specifications for all the use cases identified in the system. For clarity and narrative focus, the detailed descriptions for the three most critical use cases are presented in the main body of the document, within Section 7.1.

A.1 Account Management

The primary use case for this functional area, "Authenticate via Corporate Email", is described in detail in Section 7.1.1.

A.2 Expense Management

The use case "Register Expenses via Upload" is described in detail in Section 7.1.2. The specifications for the remaining use cases in this functional area are provided below.

A.2.1 Use Case: List Personal Expenses

Primary Actor: User

Trigger: The user navigates to the Expenses page.

Preconditions: The user is successfully authenticated.

Postconditions: The system displays a complete list of the user's expenses that are not attached to any report.

Main Success Scenario:

1. The user selects the "Expenses" option in the navigation bar or the user has recently authenticated successfully.
2. The system redirects the user to the Expenses page.
3. The system retrieves and displays all expenses associated with the authenticated user that are not attached to any report.

Extensions:

3a. No Expenses Found:

3a1. The user has no registered expenses or all existing expense records are already associated with other reports.

3a2. The system displays a drag-and-drop zone and an upload button that enable the user to upload new expenses directly.

3b. Database Connection Error:

3b1. The system notifies the user of the error.

3b2. The system prompts the user to refresh the page.

A.2.2 Use Case: Monitor AI Processing Status

Primary Actor: User

Trigger: The AI model succeeds or fails to process a receipt while the user is on the Expenses page.

Preconditions: The user is successfully authenticated and viewing the Expenses page, and at least one expense is in the processing queue, awaiting AI extraction.

Postconditions: The expense AI processing status is updated in real-time, eliminating the need for the user to refresh the page manually.

Main Success Scenario:

1. The system processes a new receipt from the queue when the AI model is available.
2. The system successfully extracts the data, triggers a real-time expense status update, and makes the AI-extracted data available for user review.
3. The user visualizes the updated "Pending Review" status without manually refreshing the page.

Extensions:

2a. AI Processing Failure (Technical Error or Unrecognized Document):
2a1. The system fails to extract the expense data due to either:
Scenario A (Invalid Content): If the AI determines the image or file is not a valid receipt or invoice.
Scenario B (Technical Issue): If a timeout or connection loss occurs, the system re-queues the record for a retry.
2a2. The system updates the expense status to "Failed".
2a3. The system enables the user to enter the expense data manually.

A.2.3 Use Case: Delete Individual Expense

Primary Actor: User

Trigger: The user deletes a single expense from the list.

Preconditions: The user is successfully authenticated and viewing the Expenses page, and at least one expense is displayed in the list.

Postconditions: The expense is deleted.

Main Success Scenario:

1. The user initiates the deletion of a single expense from the list.
2. The system prompts the user to confirm the deletion.
3. The user confirms the deletion.
4. The system deletes the expense.
5. The system restores the standard Expenses page view with the updated list (without displaying the recently deleted expense).

Extensions:

3a. Deletion Confirmation Rejected:
3a1. The user declines or closes the confirmation dialog.
3a2. The system closes the confirmation dialog and restores the standard Expenses page view.

4a. Expense Deletion Error:
4a1. An error occurs during the expense deletion due to a database connection error or an internal error.
4a2. The system notifies the user of the error and keeps them on the same page without deleting the expense.

A.2.4 Use Case: Delete Multiple Expenses

Primary Actor: User

Trigger: The user activates the bulk expense deletion mode.

Preconditions: The user is successfully authenticated and viewing the Expenses page, and at least one expense is displayed in the list.

Postconditions: The selected expenses are deleted.

Main Success Scenario:

1. The user activates bulk expense deletion mode.
2. The user selects one or more expense records from the list.
3. The user initiates the deletion of the selected expenses.
4. The system prompts the user to confirm the bulk deletion.
5. The user confirms the deletion.
6. The system deletes the selected expense records.
7. The system returns the user to the standard Expenses page view.

Extensions:

1-3a. Cancel Deletion or Deactivate Mode:
1-3a1. The user cancels the operation or deactivates bulk deletion mode.
1-3a2. The system clears all active selections, deactivates bulk deletion mode without deleting any records, and restores the standard Expenses page view.

5a. Deletion Confirmation Rejected:
5a1. The user declines or closes the confirmation dialog.
5a2. The system closes the confirmation dialog and preserves the current selection without deleting any records.

6a. Expense Deletion Error:
6a1. An error occurs during the bulk deletion due to a database connection error or an internal error.
6a2. The system notifies the user of the error and restores the standard Expenses page view without deleting any expense.

A.2.5 Use Case: Report Multiple Expenses

Primary Actor: User

Trigger: The user activates the bulk reporting mode.

Preconditions: The user is successfully authenticated and viewing the Expenses page, and at least one expense is displayed in the list.

Postconditions: The selected expenses are included in a report.

Main Success Scenario:

1. The user activates bulk reporting mode.
2. The system filters the list of expenses with a "Reviewed" status (indicating that the information has been entered manually or AI-extracted and reviewed by the user).
3. The user selects one or more expense records from the list.
4. The system presents two options: "Create New Report" and "Assign to Existing Report".
5. The user selects "Create New Report".
6. The system prompts the user to provide the required information to create a new report.
7. The user enters the required information.
8. The system validates that all required fields have been provided.
9. The system enables the report creation action.
10. The user confirms the new report creation.
11. The system generates the report using the provided information and associates the selected expenses with it.
12. The system returns the user to the standard Expenses page view and updates the expenses list to exclude the assigned expenses.

Alternative Flows:

5a. Assign to Existing Report:

5a1. The user selects "Assign to Existing Report".

5a2. The system retrieves and displays a list of existing unsubmitted reports.

5a3. The user selects the target report and confirms the assignment.

5a4. The system associates the expenses with the selected report.

5a5. The system returns the user to the standard Expenses page view and updates the expenses list to exclude the assigned expenses.

Extensions:

7a. Missing Required Fields:

7a1. The user has not provided all required fields.

7a2. The system detects missing required fields, disables the report creation action, and restricts the interaction to either canceling the operation or continuing to enter the required information.

Scenario A (Action Cancelled): The system returns the user to the standard Expenses page view without preserving the entered information.

Scenario B (Continue with the Process): The flow returns to step 6 of the main success scenario.

5a2a. No Reports Available:

1. The system detects that no reports meet the criteria for assignment.
2. The system notifies the user that no reports are available, disables the assignment functionality, and restricts the interaction to either canceling the operation or exiting the current view.
3. The user follows the **G1. User Cancels During Creation or Assignment** flow to exit.

11a. Report Creation Error:

11a1. An error occurs during the report creation due to a database connection error or an internal error.

11a2. The system notifies the user of the error and returns the user to the standard Expenses page view without creating the report.

G1. User Cancels or Resets:

During Creation or Assignment:

1. The user cancels the operation or exits the current view.
2. The system ends the process without saving any changes while preserving the selected expenses.
3. The flow returns to step 3 of the main success scenario.

Before Workflow Selection:

1. The user deactivates the bulk reporting mode or cancels the operation before selecting a workflow.
2. The system clears all active selections and restores the standard Expenses page view.

A.2.6 Use Case: Consult Expense Details

Primary Actor: User

Trigger: The user requests the detailed information for a specific expense record.

Preconditions: The user is authenticated, viewing the Expenses page, and at least one expense record is displayed in the list.

Postconditions: The system displays detailed expense information in an editable form, along with the original receipt image or file.

Main Success Scenario:

1. The user selects an expense record from the list.
2. The system retrieves the expense record and its full metadata from the database.
3. The system retrieves the associated receipt image or file from blob storage.
4. The system displays a form containing the expense metadata with the original receipt image or file.

Extensions:

2a. Expense Not Found:
2a1. The system cannot find the expense record with the provided ID.
2a2. The system displays a 404 "Expense Not Found" error message and prompts the user to return to the previous page.

3a. File or Image Loading Error:
3a1. The system retrieves the expense metadata but fails to load the receipt image or file.
3a2. The system displays a warning while keeping the metadata form visible.

A.2.7 Use Case: Update Expense Information

Primary Actor: User

Trigger: The user wants to update the data of a specific expense.

Preconditions: The user is authenticated, and the system displays a form containing the expense metadata and the original receipt image or file.

Postconditions: The system saves the updated expense data.

Main Success Scenario:

1. The user edits the necessary fields.
2. The user submits the changes.
3. The system validates and saves the updated information.
4. The system notifies the user of the success and keeps the user on the current expense information view.

Extensions:

3a. Missing Required Information or Invalid Input:
3a1. The system detects missing required information or invalid input.
3a2. The system notifies the user that the information cannot be saved and outlines the specific fields that are missing or invalid.
3a3. The flow returns to step 1 of the main success scenario.

3b. Expense Update Error:
3b1. An error occurs during the expense update due to a database connection error or an internal error.
3b2. The system notifies the user of the error and keeps the user on the current expense information view without saving the updated information.

1-2a. User Cancels / Navigates Back:
1-2a1. The user navigates back to the previous page or cancels the current editing session.
1-2a2. The system checks whether there are any unsaved changes:
Scenario A (No Changes Detected): The system terminates the session and returns the user to the Expenses page.

Scenario B (Unsaved Changes Detected): The system prompts the user to confirm before proceeding:

B1 (Discard): The user confirms the exit. The system discards changes and returns the user to the Expenses page.

B2 (Stay): The user declines the confirmation. The system preserves the current form state, and the user remains on the current form page.

A.2.8 Use Case: Delete Current Expense

Primary Actor: User

Trigger: The user wants to delete the currently displayed expense.

Preconditions: The user is authenticated, and the system displays a form containing the expense metadata and the original receipt image or file.

Postconditions: The system deletes the currently displayed expense.

Main Success Scenario:

1. The user initiates the deletion of the currently displayed expense.
2. The system prompts the user to confirm the deletion.
3. The user confirms the deletion.
4. The system deletes the currently displayed expense and returns the user to the Expenses page.

Extensions:

3a. Deletion Confirmation Rejected:

3a1. The user declines or closes the confirmation dialog.

3a2. The system closes the confirmation dialog and preserves the current expense without deleting it.

4a. Expense Deletion Error:

4a1. An error occurs during the expense deletion due to a database connection error or an internal error.

4a2. The system notifies the user of the error and keeps the user on the current expense information view.

A.2.9 Use Case: Search Expenses by Name

Primary Actor: User

Trigger: The user wants to search for a specific expense by name.

Preconditions: The user is successfully authenticated and viewing the Expenses page.

Postconditions: The system filters a list of expenses whose names match the search string.

Main Success Scenario:

1. The user enters a keyword into the search field.
2. The system filters and displays a list of expenses whose names match the search string.

Extensions:

2a. No Matches Found:

2a1. The system cannot find any match for the searched string.

2a2. The system displays an empty list and informs the user that no expenses were found.

2a3. The user decides how to proceed:

Scenario A (Reset Filter): The user clears the search input; the system restores the standard Expenses page view.

Scenario B (New Search): The user modifies the keyword; the flow returns to step 2.

A.2.10 Use Case: Filter Expenses by Category

Primary Actor: User

Trigger: The user wants to filter expenses by category.

Preconditions: The user is successfully authenticated and viewing the Expenses page.

Postconditions: The system displays a filtered list of expenses that belong to the selected category.

Main Success Scenario:

1. The user selects a category from the filter sidebar.
2. The system filters and displays all expenses that belong to the selected category.

Extensions:

2a. No Matches Found:

2a1. The system cannot find any expense that belongs to the selected category.

2a2. The system displays an empty list and informs the user that no expenses were found.

2a3. The user decides how to proceed:

Scenario A (Reset Filter): The user clears the filter; the system restores the standard Expenses page view.

Scenario B (New Filter): The user modifies the filter; the flow returns to step 2.

A.3 Report Management

A.3.1 Use Case: List Personal Reports

Primary Actor: User

Trigger: The user navigates to the Reports page.

Preconditions: The user is successfully authenticated.

Postconditions: The system displays a list of the user's pending reports.

Main Success Scenario:

1. The user selects the "Reports" option in the navigation bar.
2. The system redirects the user to the Reports page.
3. The system retrieves and displays all reports previously created by the user that are pending submission.

Extensions:

3a. No Reports Found:
3a1. The system cannot find any report that meets the criteria and informs the user.

3b. Database Connection Error:
3b1. The system notifies the user of the error.
3b2. The system prompts the user to refresh the page.

A.3.2 Use Case: Create New Report

Primary Actor: User

Trigger: The user wants to create a new report.

Preconditions: The user is successfully authenticated and viewing the Reports page.

Postconditions: The system creates a new report.

Main Success Scenario:

1. The user initiates the creation of a new report.
2. The system prompts the user to provide the required information.
3. The user enters the required information.
4. The system validates that all required fields have been provided.
5. The system enables the report creation action.
6. The user confirms the report creation.
7. The system generates the report using the provided information.
8. The system restores the standard view of the Report page (including the recently created report).

Extensions:

3a. Associate Expenses to the New Report:

3a1. The user enters the required information and activates the Expense Assignment mode to include expenses in the new report.

3a2. The system displays a list of available reportable expenses.

3a3. The user selects the expenses to include in the new report.

3a4. The system validates that all mandatory information has been provided.

3a5. The system enables the report creation action.

3a6. The user confirms the report creation.

3a7. The system creates a new report using the provided information and includes the selected expenses.

3a8. The system restores the standard view of the Report page.

3a2a. No Expenses Available:

1. The system detects that no expenses are available for reporting.

2. The system displays an empty list of expenses and notifies the user that no expenses are available.

3. The flow returns to step 4 of the main success scenario.

3a2-3a5a. Deactivate Mode:

1. The user deactivates the Expense Assignment Mode.

2. The system clears the selection; the flow returns to step 4 of the main success scenario.

3b. Missing Required Fields:

3b1. The user has not provided all required fields.

3b2. The system detects missing required fields, disables the report creation action, and restricts the interaction to either canceling the operation or continuing to enter the required information.

Scenario A (Action Cancelled): The system returns the user to the standard Reports page view without preserving the entered information.

Scenario B (Continue with the Process): The flow returns to step 2 of the main success scenario.

G1. Report Creation Error:

G1.1. An error occurs during the report creation due to a database connection error or an internal error.

G1.2. The system notifies the user of the error and returns the user to the standard Reports page view without creating the report.

A.3.3 Use Case: Delete Individual Report

Primary Actor: User

Trigger: The user deletes a single report from the list.

Preconditions: The user is successfully authenticated and viewing the Reports page, and at least one report is displayed in the list.

Postconditions: The report is deleted.

Main Success Scenario:

1. The user initiates the deletion of a single report in the list.
2. The system prompts the user to confirm the deletion.
3. The user confirms the deletion.
4. The system unassigns all expenses associated with the report, if applicable, and marks the report as deleted in the database.
5. The system restores the standard Reports page view with the updated list (without displaying the recently deleted report).

Extensions:

3a. Deletion Confirmation Rejected:
3a1. The user declines or closes the confirmation dialog.
3a2. The system closes the confirmation dialog and restores the standard Reports page view.

4a. Report Deletion Error:
4a1. An error occurs during the report deletion due to a database connection error or an internal error.
4a2. The system notifies the user of the error and keeps the user on the report's detailed information view.

A.3.4 Use Case: Delete Multiple Reports

Primary Actor: User

Trigger: The user activates the bulk report deletion mode.

Preconditions: The user is successfully authenticated and viewing the Reports page, and at least one report is displayed in the list.

Postconditions: The selected reports are deleted.

Main Success Scenario:

1. The user activates bulk report deletion mode.
2. The user selects one or more reports from the list.
3. The user initiates the deletion of the selected reports.
4. The system prompts the user to confirm the bulk deletion.
5. The user confirms the deletion.
6. The system unassigns all expenses associated with the selected reports, if applicable, and marks the reports as deleted in the database.
7. The system restores the standard Reports page view with the updated list (without displaying the recently deleted reports).

Extensions:

1-3a. Cancel Deletion or Deactivate Mode:
1-3a1. The user cancels the operation or deactivates bulk deletion mode.

1-3a2. The system clears all active selections, deactivates bulk deletion mode without deleting any records, and restores the standard Reports page view.

5a. Deletion Confirmation Rejected:

5a1. The user declines or closes the confirmation dialog.

5a2. The system closes the confirmation dialog and preserves the current selection without deleting any records.

6a. Report Deletion Error:

6a1. An error occurs during the bulk deletion due to a database connection error or an internal error.

6a2. The system notifies the user of the error and restores the standard Reports page view without deleting any report.

A.3.5 Use Case: Consult Report Details

Primary Actor: User

Trigger: The user requests the detailed information for a specific report.

Preconditions: The user is authenticated, is on the Reports page, and at least one report record is displayed in the list.

Postconditions: The system displays detailed report information in an editable form, along with any associated expenses.

Main Success Scenario:

1. The user selects a report record from the list.
2. The system retrieves the record and its full metadata from the database using the unique Report ID.
3. The system retrieves the associated expenses.
4. The system displays a form containing the report metadata and a list of associated expenses.

Extensions:

2a. Report Not Found:

2a1. The system cannot find the report record with the provided ID.

2a2. The system displays a 404 “Report Not Found” error message and prompts the user to return to the previous page.

3a. No Expenses Found:

3a1. The system detects that no expenses are associated with the report.

3a2. The system displays a form containing the report metadata and an empty expenses list while informing the user that no expenses are attached to this report.

A.3.6 Use Case: Update Report Information

Primary Actor: User

Trigger: The user wants to update the data of a specific report.

Preconditions: The user is authenticated, and the system displays a form containing the report metadata and any associated expenses.

Postconditions: The system saves the updated report data.

Main Success Scenario:

1. The user edits the necessary fields.
2. The user submits the changes.
3. The system validates and saves the updated information.
4. The system notifies the user of the success and keeps the user on the current report details view.

Extensions:

3a. Missing Required Information or Invalid Input:
3a1. The system detects missing required information or invalid input.
3a2. The system notifies the user that the information cannot be saved and outlines the specific fields that are missing or invalid.
3a3. The flow returns to step 2 of the main success scenario.

3b. Report Update Error:
3b1. An error occurs during the report update due to a database connection error or an internal error.
3b2. The system notifies the user of the error and keeps the user on the current report details view.

1-2a. User Cancels / Navigates Back:
1-2a1. The user navigates back to the previous page or cancels the current editing session.
1-2a2. The system checks whether there are any unsaved changes:
Scenario A (No Changes Detected): The system terminates the session and returns the user to the Reports page.
Scenario B (Unsaved Changes Detected): The system prompts the user to confirm before proceeding:
B1 (Discard): The user confirms the exit. The system discards changes and returns the user to the Reports page.
B2 (Stay): The user declines the confirmation. The system preserves the current form state, and the user remains on the current form page.

A.3.7 Use Case: Delete Current Report

Primary Actor: User

Trigger: The user wants to delete the currently displayed report.

Preconditions: The user is authenticated, and the system displays a form containing the report metadata and any associated expenses.

Postconditions: The system deletes the currently displayed report.

Main Success Scenario:

1. The user initiates the deletion of the currently displayed report.
2. The system prompts the user to confirm the deletion.
3. The user confirms the deletion.
4. The system unassigns all expenses associated with the report, if applicable, and marks the report as deleted in the database.
5. The system restores the standard Reports page view with the updated list (without displaying the recently deleted report).

Extensions:

3a. Deletion Confirmation Rejected:
3a1. The user declines or closes the confirmation dialog.
3a2. The system closes the confirmation dialog and preserves the current report without deleting it.

4a. Report Deletion Error:
4a1. An error occurs during the report deletion due to a database connection error or an internal error.
4a2. The system notifies the user of the error and keeps them on the same page without deleting the report.

A.3.8 Use Case: Submit Report

Primary Actor: User

Trigger: The user wants to submit a specific report.

Preconditions: The user is authenticated, the system displays a form containing the report metadata, and at least one expense is included in the report.

Postconditions: The system submits the currently displayed report to the corresponding supervisor for review.

Main Success Scenario:

1. The user submits the currently displayed report.
2. The system submits the report and notifies the user's supervisor that a new report is pending review.
3. The report and its associated expenses become read-only and remain available for consultation.

Extensions:

2a. Report Submission Error:
2a1. An error occurs during the report submission due to a database connection error or an internal error.
2a2. The system notifies the user of the error and keeps the user on the current report details view.

A.3.9 Use Case: Attach Expenses to Report

Primary Actor: User

Trigger: The user wants to attach new expenses to a report.

Preconditions: The user is authenticated, and the system displays a form containing the report metadata along with a list of any associated expenses.

Postconditions: The selected expenses are included in the report.

Main Success Scenario:

1. The user initiates the attachment of new expenses.
2. The system displays a list of available reportable expenses for selection (selectable expenses must have been manually reviewed and corrected by the user, and must not belong to any report).
3. The user selects one or more expense records from the list.
4. The user asks the system to attach the selected expenses to the current report.
5. The system includes the selected expenses in the current report and updates the report's information accordingly, such as the total amount or expense count.
6. The system updates the detailed information view of the report to reflect the current changes.

Extensions:

2a. No Expenses Available:

2a1. The system detects that no reportable expenses are available.

2a2. The system notifies the user that no expenses are available.

2a3. Once the user cancels the operation or terminates the process, the system restores the detailed information view of the report.

5a. Expense Attachment Error:

5a1. An error occurs during the expense attachment due to a database connection error or an internal error.

5a2. The system notifies the user of the error and restores the detailed information view of the report.

A.3.10 Use Case: Detach Multiple Expenses from Report

Primary Actor: User

Trigger: The user wants to remove new expenses from a report.

Preconditions: The user is authenticated, and the system displays a form containing the report metadata.

Postconditions: The selected expenses are excluded from the report.

Main Success Scenario:

1. The user initiates the removal of expenses from the report.
2. The system enables bulk expense selection.
3. The user selects one or more expense records from the list.
4. The user requests to exclude the selected expenses from the current report.
5. The system prompts the user to confirm the action.
6. The user confirms the detachment.
7. The system removes the selected expenses from the current report and updates the report's information accordingly, such as the total amount or expense count.
8. The system updates the detailed information view of the report to reflect the current changes.

Extensions:

3a. No Expenses Attached:
3a1. No expenses are attached to the report; the system displays an empty list.
3a2. The user cannot select any expense to detach.

6a. Detachment Confirmation Rejected:
6a1. The user declines or closes the confirmation dialog.
6a2. The system closes the confirmation dialog and preserves the current expense selection without detaching any expenses.

7a. Expense Detachment Error:
7a1. An error occurs during the expense detachment due to a database connection error or an internal error.
7a2. The system notifies the user of the error and restores the detailed information view of the report.

A.3.11 Use Case: Detach Individual Expense from Report

Primary Actor: User

Trigger: The user removes a single expense from the report's expenses list.

Preconditions: The user is authenticated, and the system displays a form containing the report metadata.

Postconditions: The expense is excluded from the report.

Main Success Scenario:

1. The user initiates the detachment of a single expense from the report's expenses list.
2. The system prompts the user to confirm the detachment.
3. The user confirms the detachment.
4. The system removes the selected expense from the current report and updates the report's information accordingly, such as the total amount or expense count.
5. The system updates the detailed information view of the report to reflect the current changes.

Extensions:

3a. Detachment Confirmation Rejected:
3a1. The user declines or closes the confirmation dialog.
3a2. The system closes the confirmation dialog and preserves the current Report Information page view.

4a. Expense Detachment Error:
4a1. An error occurs during the expense detachment due to a database connection error or an internal error.
4a2. The system notifies the user of the error and restores the detailed information view of the report.

A.3.12 Use Case: Consult Attached Expense Details

Primary Actor: User

Trigger: The user requests the detailed information for an expense attached to the report.

Preconditions: The user is authenticated, the system displays a form containing the report metadata, and at least one expense is included in the report.

Postconditions: The system displays detailed expense information in an editable form, along with the original receipt image or file.

Main Success Scenario:

1. The user selects an expense record from the list.
2. The system retrieves the record and its full metadata from the database using the unique Expense ID.
3. The system retrieves the associated receipt image or file from blob storage.
4. The system displays a form containing the expense metadata and the original receipt image or file.

Extensions:

2a. Expense Not Found:
2a1. The system cannot find the expense record with the provided ID.
2a2. The system displays a 404 “Expense Not Found” error message and prompts the user to return to the previous page.

3a. File or Image Loading Error:
3a1. The system retrieves the expense metadata but fails to load the receipt image or file.
3a2. The system displays a warning while keeping the metadata form visible.

A.3.13 Use Case: Update Attached Expense Information

Primary Actor: User

Trigger: The user wants to update the data of the current expense.

Preconditions: The user is authenticated, and the system displays a form containing the expense metadata along with the original receipt image or file.

Postconditions: The system saves the updated expense data.

Main Success Scenario:

1. The user edits the necessary fields.
2. The user submits the changes.
3. The system validates and saves the updated information.
4. The system notifies the user of the success and keeps the user on the current expense information view.

Extensions:

3a. Missing Required Information or Invalid Input:

3a1. The system detects missing required information or invalid input.

3a2. The system notifies the user that the information cannot be saved and outlines the specific fields that are missing or invalid.

3a3. The flow returns to step 2 of the main success scenario.

3b. Expense Update Error:

3b1. An error occurs during the expense update due to a database connection error or an internal error.

3b2. The system notifies the user of the error and keeps the user on the expense's detailed information view.

1-2a. User Cancels / Navigates Back:

1-2a1. The user navigates back to the previous page or cancels the current editing session.

1-2a2. The system checks whether there are any unsaved changes:

Scenario A (No Unsaved Changes): The system terminates the session and returns the user to the previous page.

Scenario B (Unsaved Changes Detected): The system prompts the user to confirm before proceeding:

B1 (Discard): The user confirms the exit. The system discards changes and returns the user to the previous page.

B2 (Stay): The user declines the confirmation. The system preserves the current form state, and the user remains on the current form page.

A.3.14 Use Case: Detach Current Expense from Report

Primary Actor: User

Trigger: The user wants to remove the current expense from its report.

Preconditions: The user is authenticated, and the system displays a form containing the expense metadata along with the original receipt image or file.

Postconditions: The system detaches the expense from the belonging report.

Main Success Scenario:

1. The user requests to exclude the current expense from the belonging report.
2. The system prompts the user to confirm the detachment.
3. The user confirms the detachment.
4. The system detaches the currently displayed expense from its report and updates the report's information accordingly, such as the total amount or expense count.
5. The system returns the user to the report information page, displaying the updated information.

Extensions:

3a. Detachment Confirmation Rejected:
3a1. The user declines or closes the confirmation dialog.
3a2. The system closes the confirmation dialog and preserves the current expense information view without detaching it from its report.

4a. Expense Detachment Error:
4a1. An error occurs during the expense detachment due to a database connection error or an internal error.
4a2. The system notifies the user of the error and keeps the user on the current expense information view.

A.4 Submitted Report Management

A.4.1 Use Case: List Personal Submitted Reports

Primary Actor: User

Trigger: The user navigates to the Submitted Reports page.

Preconditions: The user is successfully authenticated.

Postconditions: The system displays a list of the user's submitted reports.

Main Success Scenario:

1. The user selects the "Submitted Reports" option in the navigation bar.
2. The system redirects the user to the Submitted Reports page.
3. The system retrieves and displays all reports previously created by the user that have been submitted.

Extensions:

3a. No Reports Found:
3a1. The system cannot find any reports that meet the criteria, displays an empty list, and informs the user.

3b. Database Connection Error:
3b1. The system notifies the user of the error.
3b2. The system prompts the user to refresh the page.

A.4.2 Use Case: Monitor Submitted Report Processing Status

Primary Actor: User

Trigger: A submitted report is reviewed by the supervisor while the user is on the Submitted Reports page.

Preconditions: The user is successfully authenticated, is viewing the Submitted Reports page, and at least one report in the list is pending review.

Postconditions: The report review status is updated in real-time, eliminating the need for the user to refresh the page manually.

Main Success Scenario:

1. The supervisor approves or rejects a report submitted by the user.
2. The system triggers a real-time report status update and notifies the user that their report has been reviewed.
3. The user visualizes the updated report status without manually refreshing the page.

A.4.3 Use Case: Consult Submitted Report Details

Primary Actor: User

Trigger: The user requests detailed information for a specific report.

Preconditions: The user is authenticated and is on the Submitted Reports page.

Postconditions: The system displays the detailed report information along with its associated expenses.

Main Success Scenario:

1. The user selects a report record from the list.
2. The system retrieves the record and its full metadata from the database using the unique Report ID.
3. The system retrieves the associated expenses.
4. The system displays the report metadata along with a list of associated expenses.

Extensions:

2a. Report Not Found:
2a1. The system cannot find the report record with the provided ID.
2a2. The system displays a 404 "Report Not Found" error message and prompts the user to return to the previous page.

A.4.4 Use Case: Consult the Attached Expense Details From Submitted Report

Primary Actor: User

Trigger: The user requests detailed information for an expense attached to a report.

Preconditions: The user is authenticated, and the system displays the report metadata along with a list of associated expenses.

Postconditions: The system displays detailed expense information along with the original receipt image or file.

Main Success Scenario:

1. The user selects an expense record from the list.
2. The system retrieves the record and its full metadata from the database using the unique Expense ID.
3. The system retrieves the associated receipt image or file from blob storage.
4. The system displays the expense metadata along with the original receipt image or file.

Extensions:

2a. Expense Not Found:
2a1. The system cannot find the expense record with the provided ID.
2a2. The system displays a 404 "Expense Not Found" error message and prompts the user to return to the previous page.

3a. File or Image Loading Error:
3a1. The system retrieves the expense metadata but fails to load the receipt image or file.
3a2. The system displays a warning while keeping the metadata visible.

A.5 Pending Review Report Management

The use case "Approve/Reject Report" is described in detail in Section 7.1.5. The specifications for the remaining use cases in this functional area are provided below.

A.5.1 Use Case: List Pending Review Reports

Primary Actor: Administrator

Trigger: The administrator navigates to the Pending Review Reports page.

Preconditions: The administrator is successfully authenticated.

Postconditions: The system displays a list of reports submitted by employees under their supervision.

Main Success Scenario:

1. The administrator selects the "Pending Review Reports" option in the navigation bar.
2. The system redirects the administrator to the Pending Review Reports page.
3. The system retrieves and displays all reports submitted by employees under their supervision that are pending review.

Extensions:

3a. No Reports Found:
3a1. The system cannot find any reports that meet the criteria and informs the administrator.

3b. Database Connection Error:
3b1. The system notifies the administrator of the error.
3b2. The system prompts the administrator to refresh the page.

A.5.2 Use Case: Consult Pending Review Report Details

Primary Actor: Administrator

Trigger: The administrator requests detailed information for a specific pending review report.

Preconditions: The administrator is authenticated and is on the Pending Review Reports page.

Postconditions: The system displays the detailed report information along with its associated expenses.

Main Success Scenario:

1. The administrator selects a report record from the list.
2. The system retrieves the record and its full metadata from the database using the unique Report ID.
3. The system retrieves the associated expenses.
4. The system displays the report metadata along with a list of associated expenses.

Extensions:

2a. Report Not Found:
2a1. The system cannot find the report record with the provided ID.
2a2. The system displays a 404 "Report Not Found" error message and prompts the administrator to return to the previous page.

A.5.3 Use Case: Consult the Attached Expense Details From Pending Review Report

Primary Actor: Administrator

Trigger: The administrator requests detailed information for an expense attached to the pending review report.

Preconditions: The administrator is authenticated, and the system displays the report metadata along with a list of associated expenses.

Postconditions: The system displays detailed expense information along with the original receipt image or file.

Main Success Scenario:

1. The administrator selects an expense record from the list.
2. The system retrieves the record and its full metadata from the database using the unique Expense ID.
3. The system retrieves the associated receipt image or file from blob storage.
4. The system displays the expense metadata along with the original receipt image or file.

Extensions:

2a. Expense Not Found:
2a1. The system cannot find the expense record with the provided ID.
2a2. The system displays a 404 "Expense Not Found" error message and prompts the administrator to return to the previous page.

3a. File or Image Loading Error:
3a1. The system retrieves the expense metadata but fails to load the receipt image or file.
3a2. The system displays a warning while keeping the metadata visible.

A.6 Reviewed Report Management

A.6.1 Use Case: List Reviewed Reports

Primary Actor: Administrator

Trigger: The administrator navigates to the Reviewed Reports page.

Preconditions: The administrator is successfully authenticated.

Postconditions: The system displays a list of reports that the administrator has reviewed.

Main Success Scenario:

1. The administrator selects the "Reviewed Reports" option in the navigation bar.
2. The system redirects the administrator to the Reviewed Reports page.

3. The system retrieves and displays all reports reviewed by the administrator.

Extensions:

3a. No Reports Found:
3a1. The system cannot find any reports that meet the criteria and informs the administrator.

3b. Database Connection Error:
3b1. The system notifies the administrator of the error.
3b2. The system prompts the administrator to refresh the page.

A.6.2 Use Case: Consult Reviewed Report Details

Primary Actor: Administrator

Trigger: The administrator requests detailed information for a specific report they have reviewed.

Preconditions: The administrator is authenticated and is on the Reviewed Reports page.

Postconditions: The system displays the detailed report information along with its associated expenses.

Main Success Scenario:

1. The administrator selects a report record from the list.
2. The system retrieves the record and its full metadata from the database using the unique Report ID.
3. The system retrieves the associated expenses.
4. The system displays the report metadata along with a list of associated expenses.

Extensions:

2a. Report Not Found:
2a1. The system cannot find the report record with the provided ID.
2a2. The system displays a 404 “Report Not Found” error message and prompts the administrator to return to the previous page.

A.6.3 Use Case: Consult Attached Expense Details From Reviewed Report

Primary Actor: Administrator

Trigger: The administrator requests detailed information for an expense attached to the report they have reviewed.

Preconditions: The administrator is authenticated, and the system displays the report metadata along with a list of associated expenses.

Postconditions: The system displays detailed expense information along with the original receipt image or file.

Main Success Scenario:

1. The administrator selects an expense record from the list.
2. The system retrieves the record and its full metadata from the database using the unique Expense ID.
3. The system retrieves the associated receipt image or file from blob storage.
4. The system displays the expense metadata along with the original receipt image or file.

Extensions:

2a. Expense Not Found:

2a1. The system cannot find the expense record with the provided ID.

2a2. The system displays a 404 "Expense Not Found" error message and prompts the administrator to return to the previous page.

3a. File or Image Loading Error:

3a1. The system retrieves the expense metadata but fails to load the receipt image or file.

3a2. The system displays a warning while keeping the metadata visible.

B User Interface Gallery

This appendix provides a complete visual reference of the application's user interface. It includes all primary screens, key interaction dialogs, and illustrates the different states a view can have (e.g., editable vs. read-only).

B.1 LoginPage View

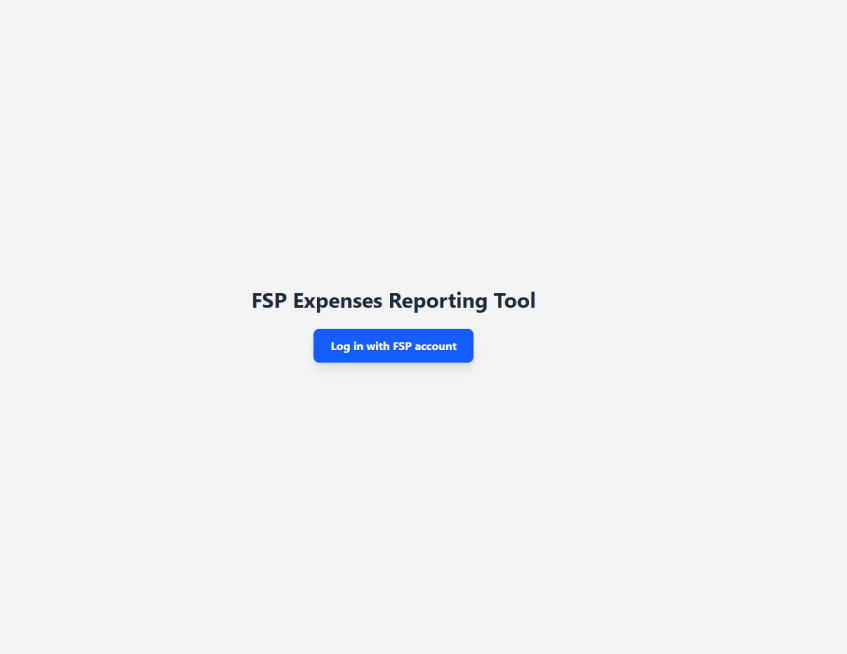

Figure B.1.1: LoginPage user interface view. [Own creation]

B.2 ExpensesPage View

B.2.1 Main User Interface

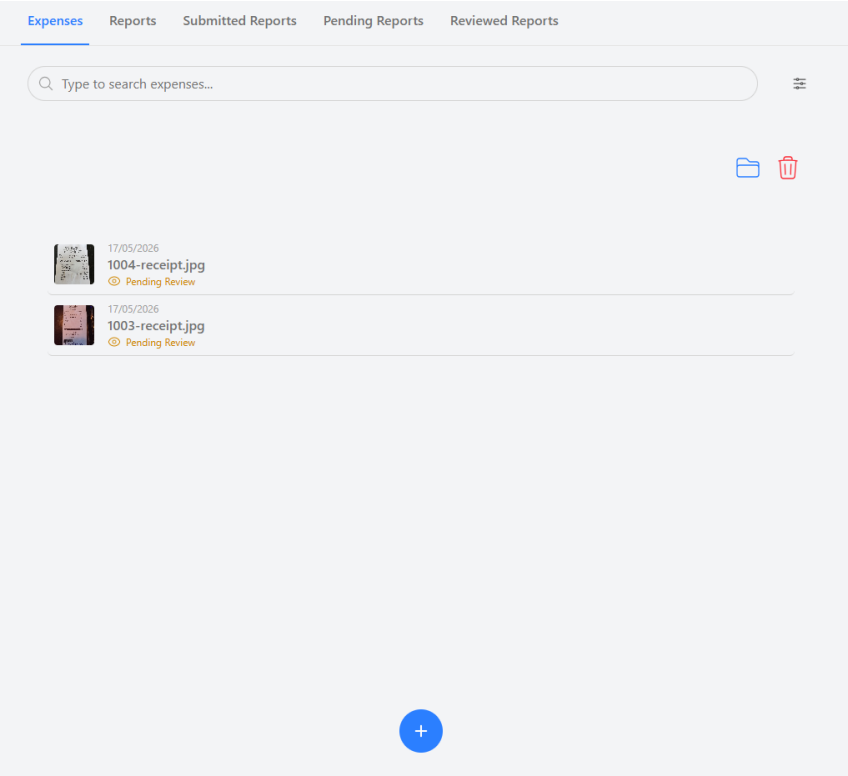

Figure B.2.1: ExpensesPage user interface view. [Own creation]

B.2.2 Filtering by Category and Name

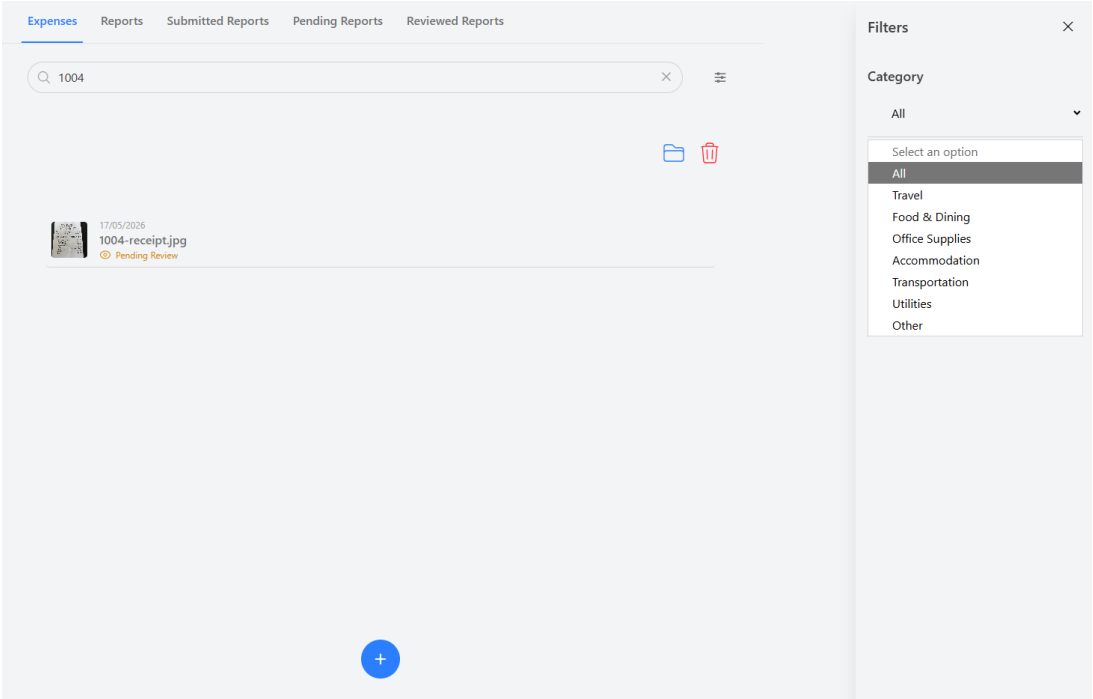

Figure B.2.2: ExpensesPage view illustrating filtering by category and name. [Own creation]

B.2.3 Attaching Expenses to a Report

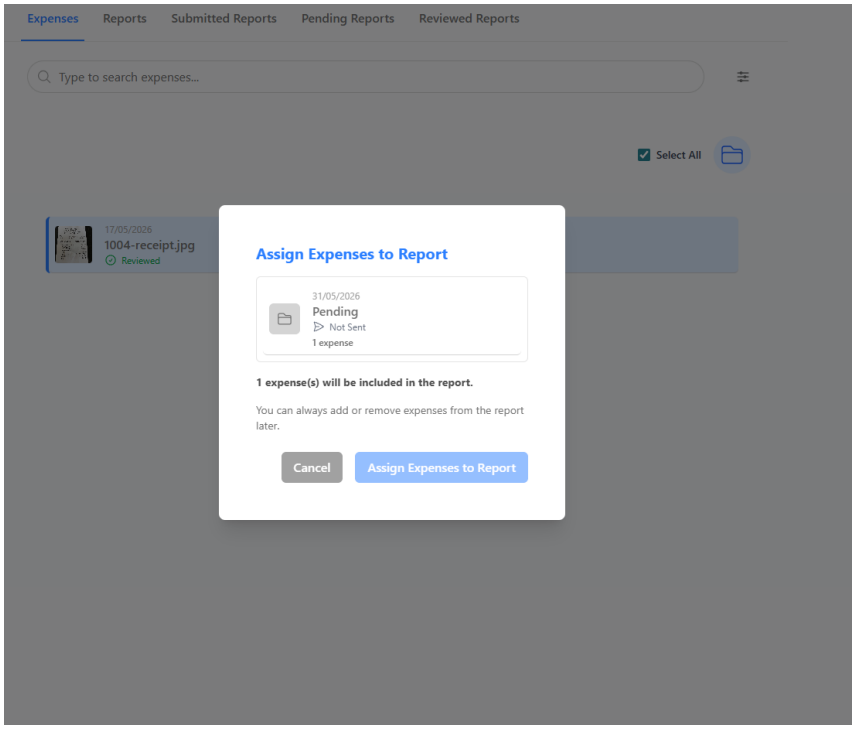

Figure B.2.3: ExpensesPage view illustrating attachment of expenses to a report. [Own creation]

B.3 ReportsPage View

B.3.1 Main User Interface

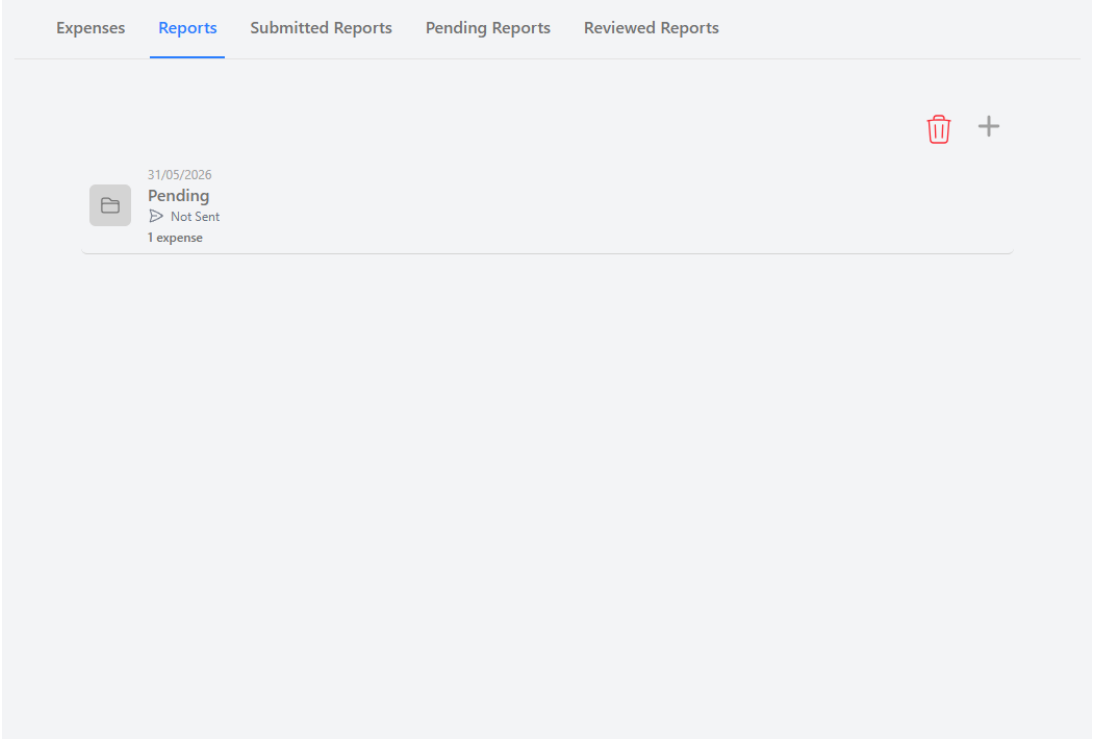

Figure B.3.1: ReportsPage user interface view. [Own creation]

B.3.2 Creating a New Report

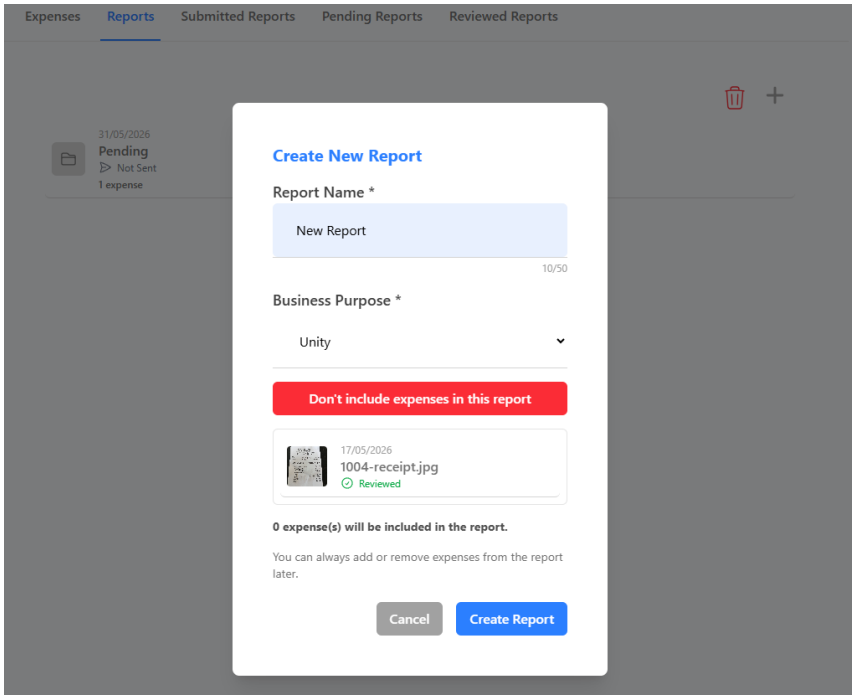

Figure B.3.2: ReportsPage view illustrating the creation of a new report. [Own creation]

B.4 Other Report Page Views

B.4.1 SubmittedReportsPage View

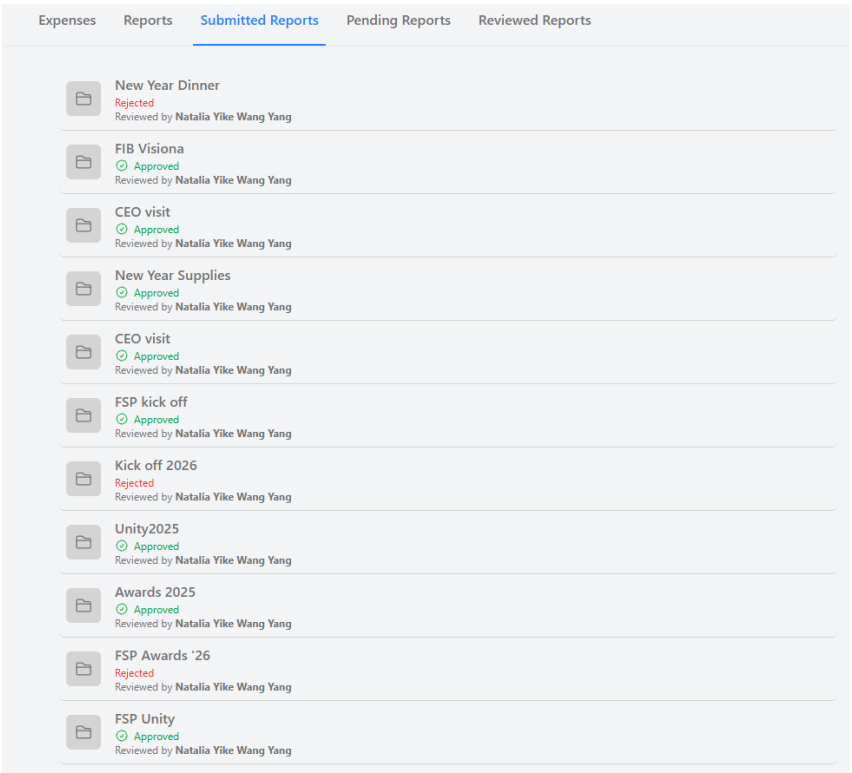

Figure B.4.1: SubmittedReportsPage user interface view. [Own creation]

B.4.2 PendingReportsPage View

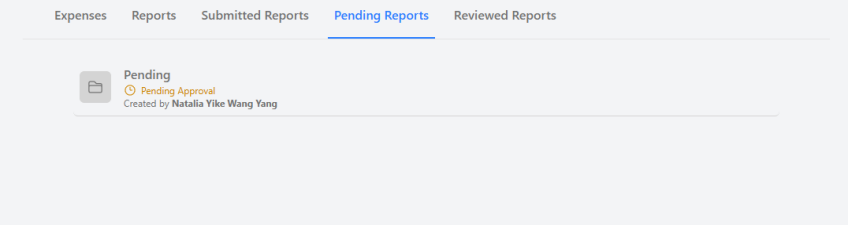

Figure B.4.2: PendingReportsPage user interface view. [Own creation]

B.4.3 ReviewedReportsPage View

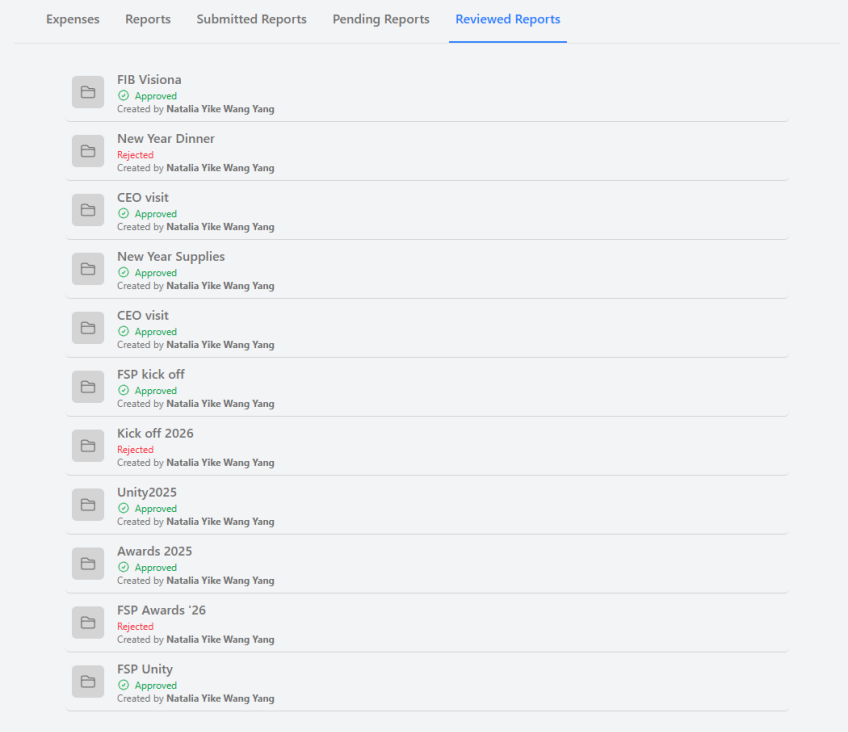

Figure B.4.3: ReviewedReportsPage user interface view. [Own creation]

B.5 ExpenseInfoPage

B.5.1 ExpenseInfoPage View

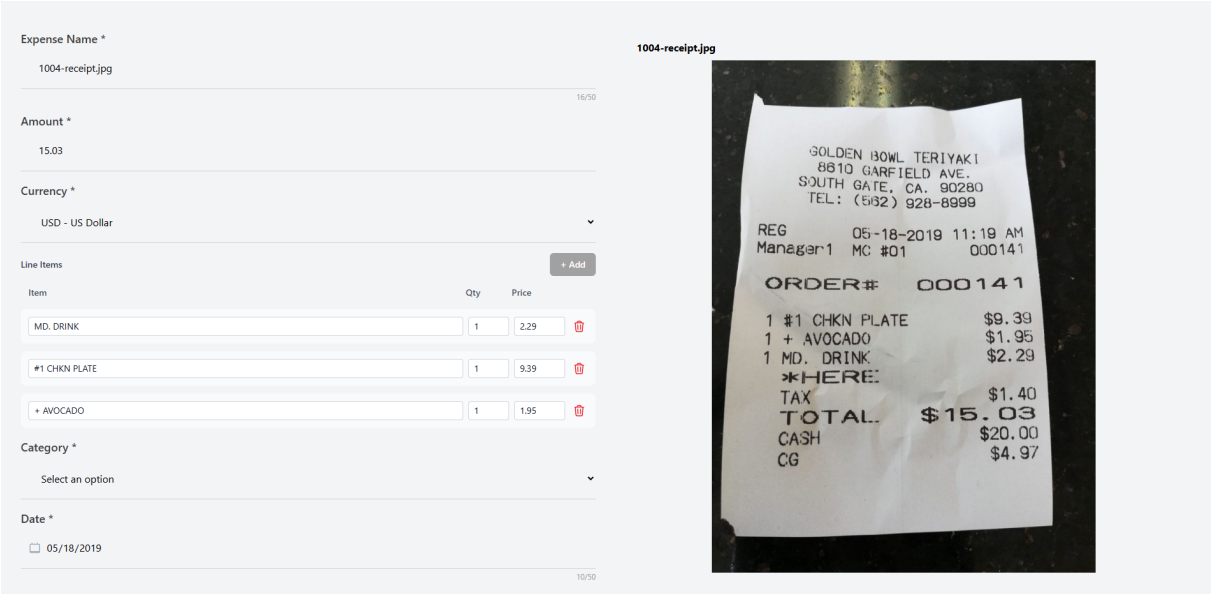

Figure B.5.1: ExpenseInfoPage user interface view. [Own creation]

B.5.2 Editable View

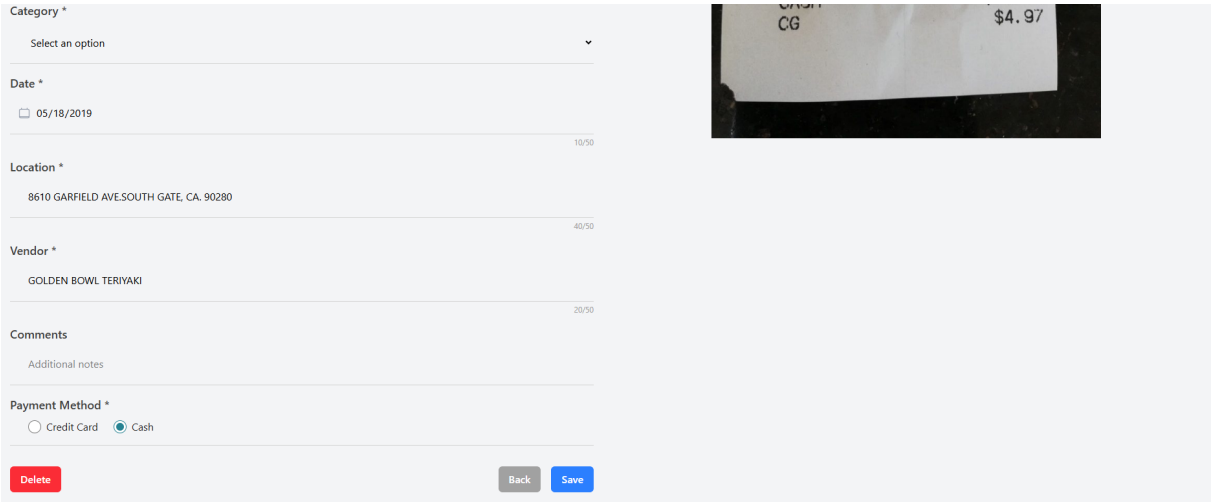

Figure B.5.2: ExpenseInfoPage editable view. [Own creation]

B.5.3 Read-Only View

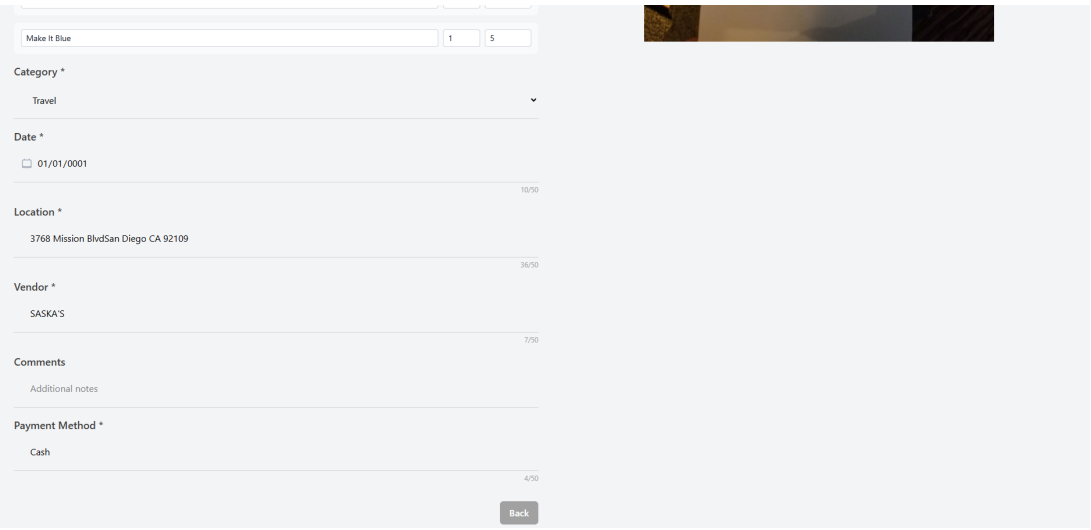

Figure B.5.3: ExpenseInfoPage read-only view. [Own creation]

B.6 ReportInfoPage

B.6.1 Editable View

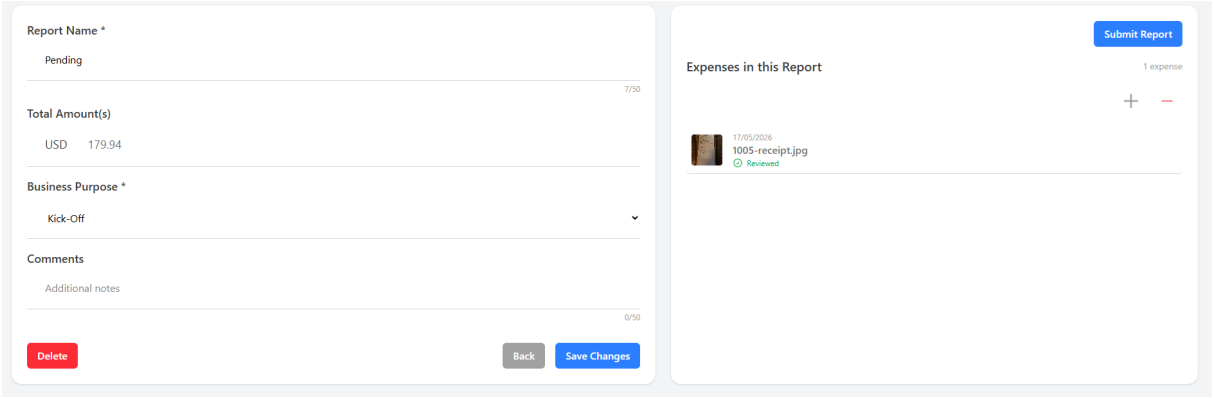

Figure B.6.1: ReportInfoPage editable view. [Own creation]

B.6.2 Read-Only View

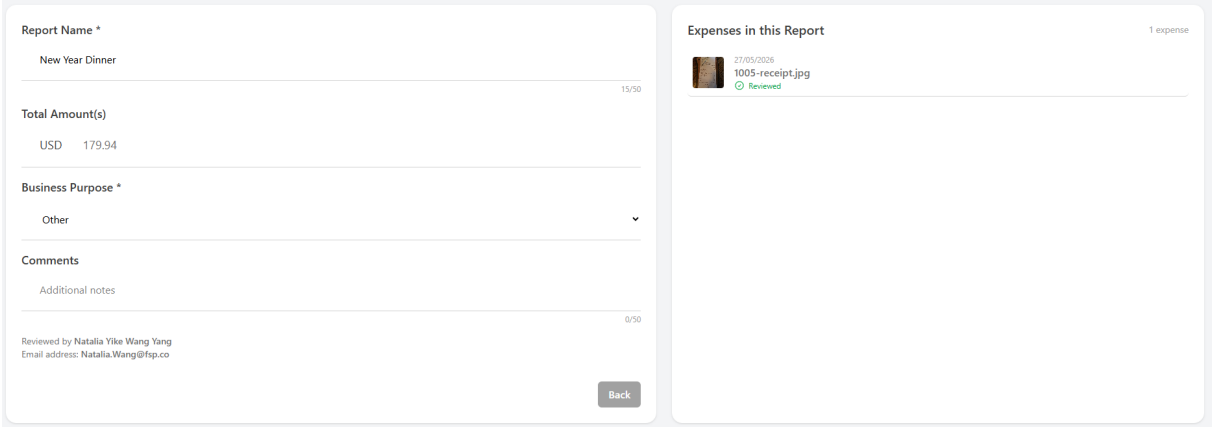

Figure B.6.2: ReportInfoPage read-only view. [Own creation]

C API Endpoint Specification

This appendix provides the technical specification for the API. It includes a visual overview of all available endpoints and detailed examples of the contracts for key operations.

C.1 API Endpoints Overview

The following Figure C.1.1, generated by the Swagger UI tool, provides a complete list of all endpoints supported by the backend API.

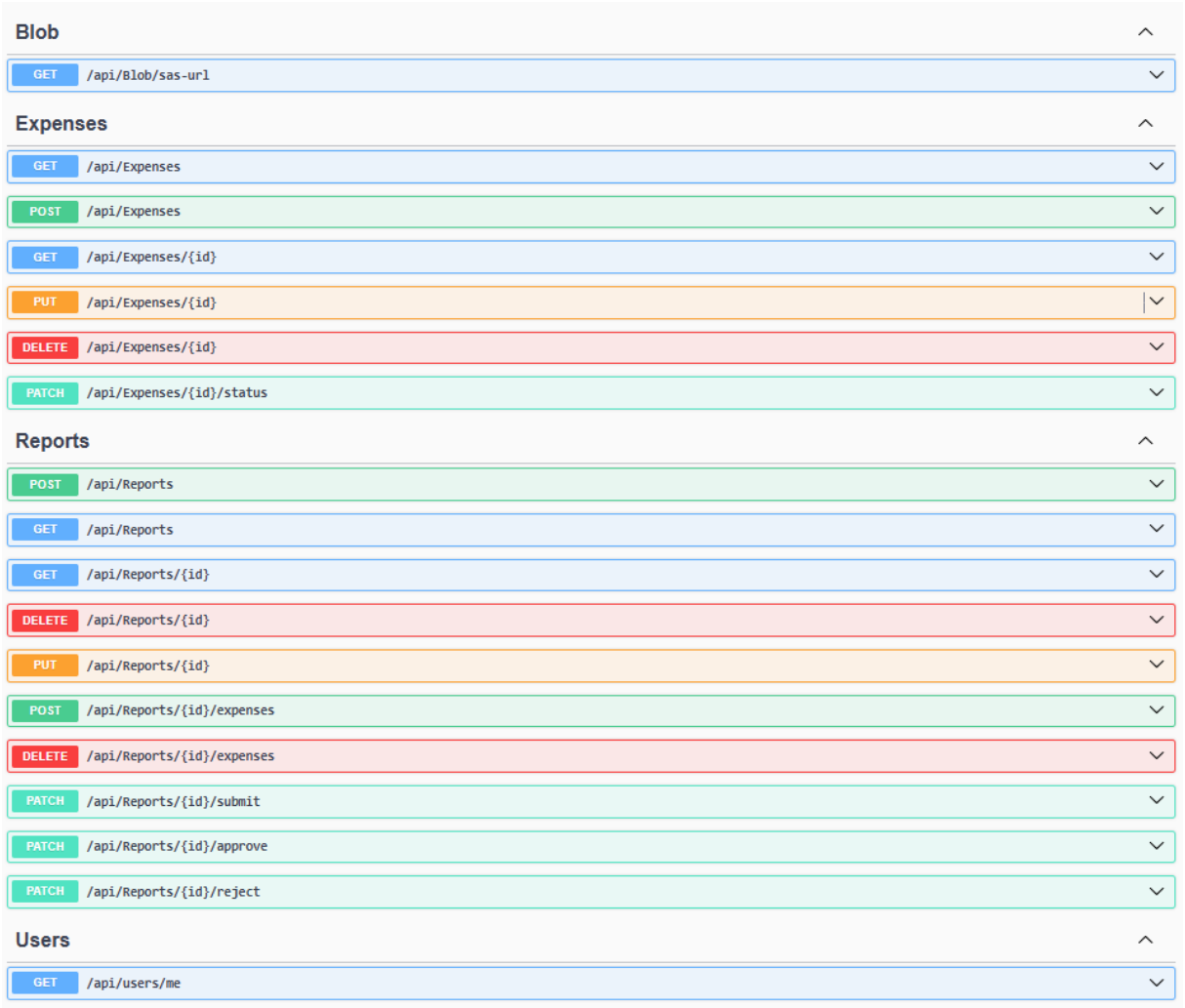

Figure C.1.1: Complete endpoints list generated by Swagger UI. [Own creation]

C.2 Detailed Endpoint Examples

To illustrate the structure of the API contracts, this section provides the full specification for two critical endpoints.

C.2.1 Upload a New Expense

This endpoint is used to upload a new expense receipt and create the initial record in the system. It is the entry point for the asynchronous AI processing flow. Figure C.2.1 shows the request body specification, while Figure C.2.2 depicts the response schemas for this concrete endpoint.

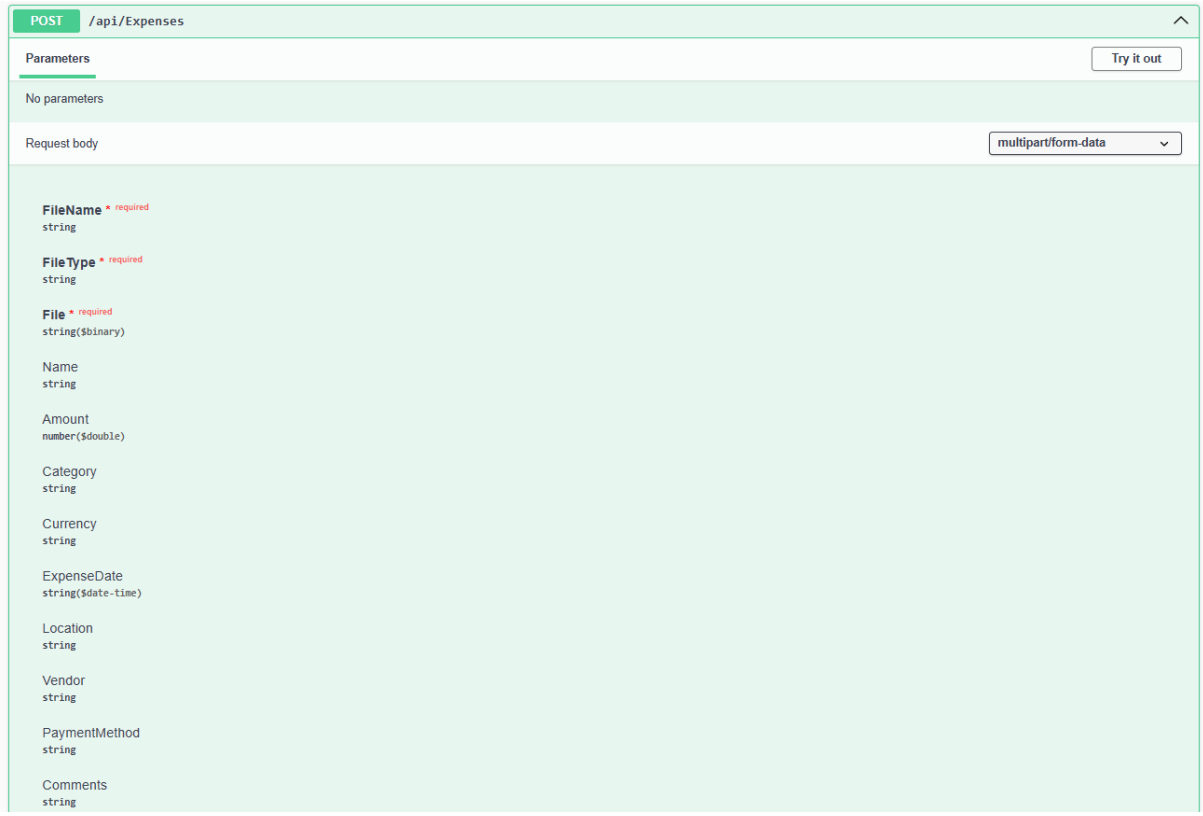

Figure C.2.1: Request body specification for the Upload Expense endpoint. [Own creation]

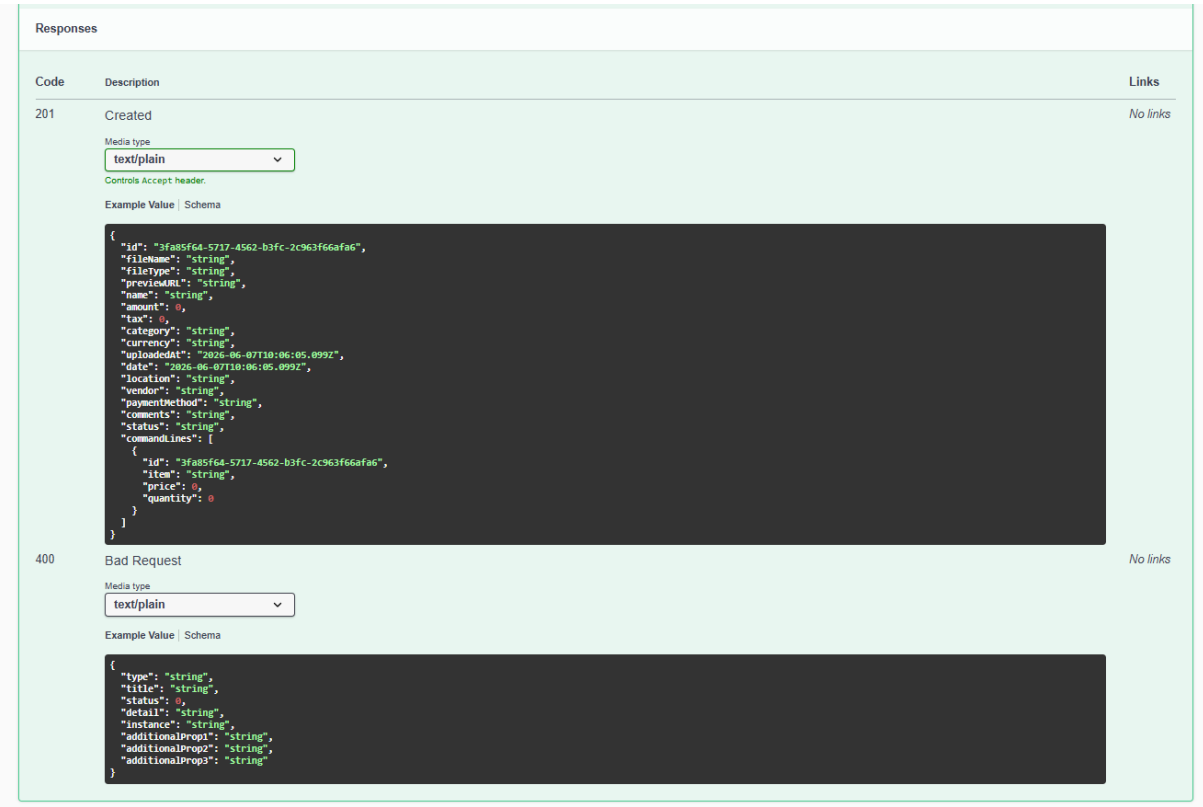

Figure C.2.2: Response schemas for the Upload Expense endpoint. [Own creation]

C.2.2 Approve a Report

This endpoint manages a key business workflow, allowing a user with administrative privileges to approve a submitted expense report. Both request body and response specifications are depicted in Figure C.2.3.

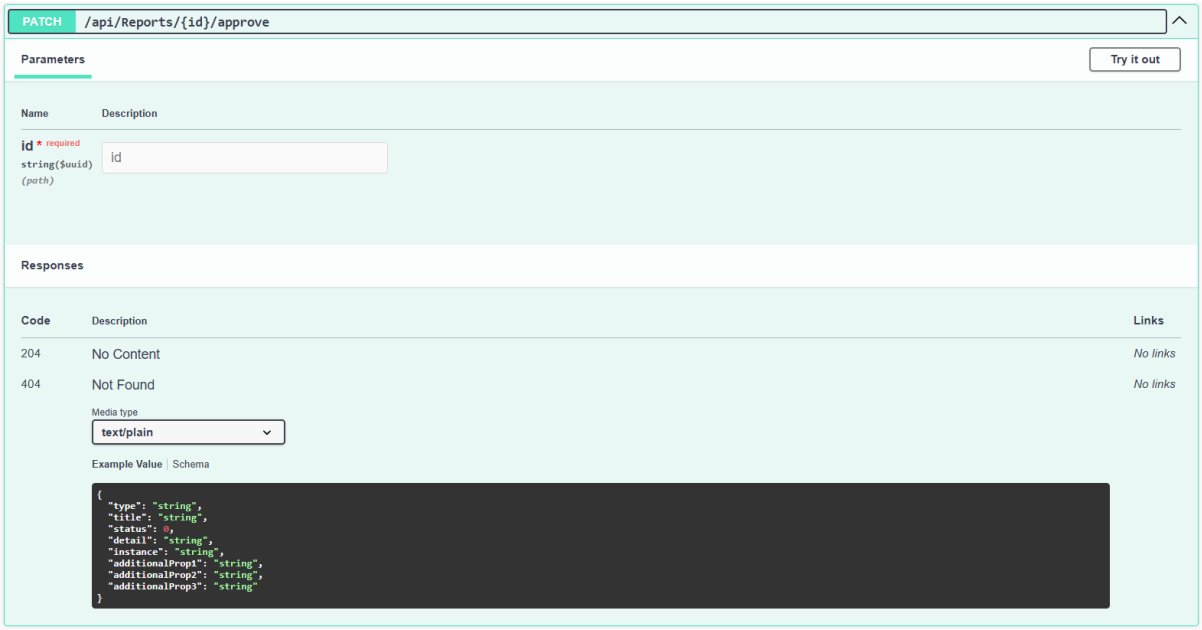

Figure C.2.3: Complete specification for the Approve Report endpoint. [Own creation]

D Complete Backend Class Diagram

This appendix provides the complete and detailed UML class diagram for the backend application. It illustrates the static structure of all major classes across the API, Application, and Infrastructure logic layers, showcasing their properties, methods, and relationships.

Due to its comprehensive nature and to ensure readability, the diagram is presented in two parts. The first part focuses on the core `Expense` and `User` management workflows, while the second part details the `Report` management and real-time notification workflows. Together, they form a single, cohesive model of the system's codebase.

D.1 Expense, User, and Storage Management Classes

Figure D.1.1 below details the classes involved in the expense and user management lifecycle. It highlights key components such as the `ExpenseService`, the `UserService`, the `BlobStorageService`, and their corresponding interfaces and implementations. It also shows their direct dependencies on the persistence layer through repositories.

The `ProcessExpenseFromQueue` class contains the implementation executed when the Azure Functions Worker is triggered. Although hosted in an Azure Functions Worker (mentioned in the Physical Architecture Diagram in Figure 9.1.1), its code is logically part of the aPI Layer.

Azure Functions act as an adapter for asynchronous system events, just as a Controller acts as an adapter for HTTP requests. The asynchronous flow has already been detailed in Section 9.1, specifically in the Asynchronous AI Processing Flow section.

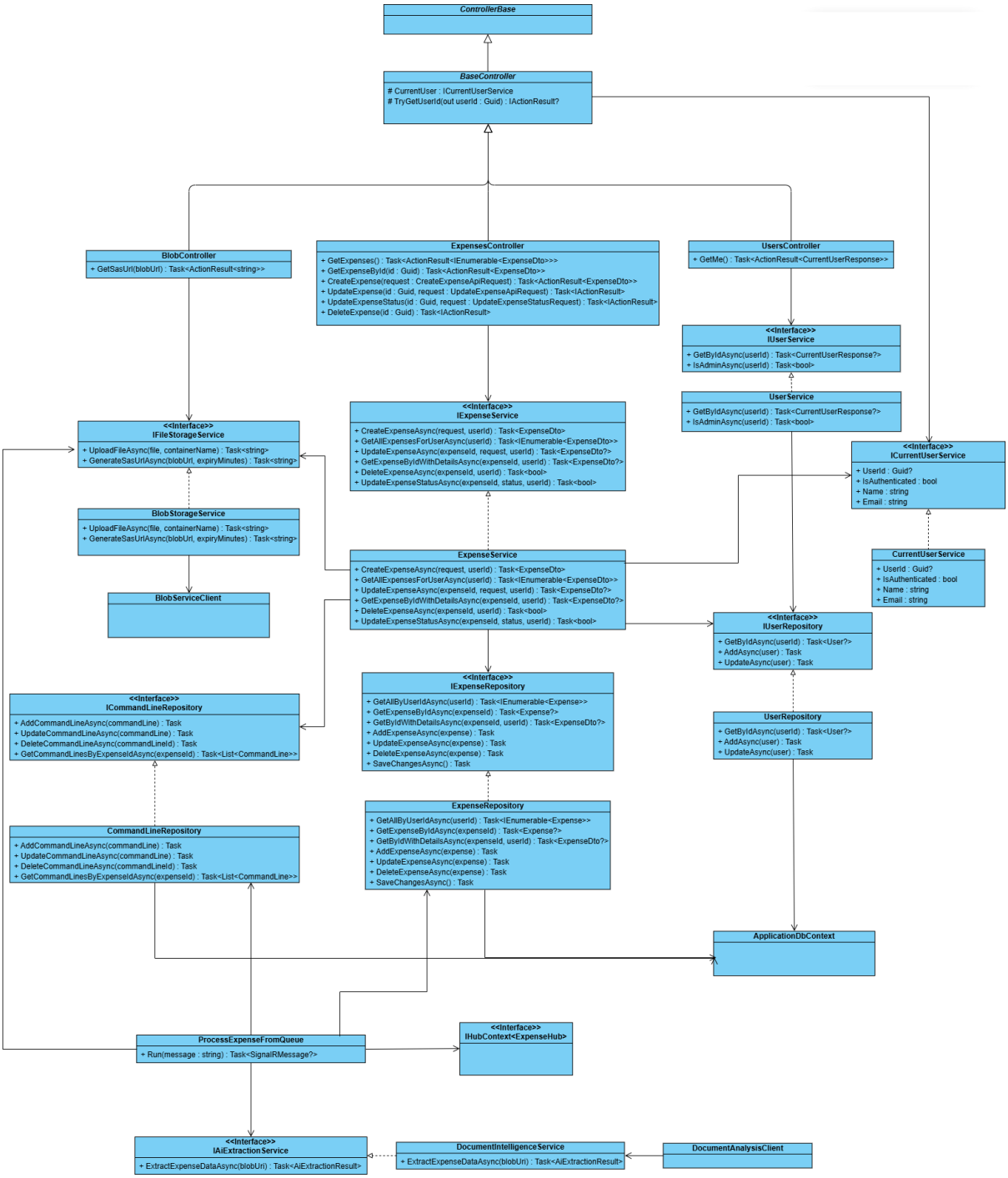

Figure D.1.1: UML Class Diagram - Part 1: Expense, User, and Blob Storage Workflows. [Own creation]

D.2 Report Management and Notification Classes

Figure D.2.1 below illustrates the classes responsible for the report management lifecycle and real-time communications. It shows the `ReportsController`, the `ReportService`, and its dependencies on both `IReportRepository` and `IExpenseRepository`, connecting it to the classes shown in Figure D.1.1. This diagram also details the components responsible for notifications, including the `EmailNotificationService` and the `SignalR ReportHub`.

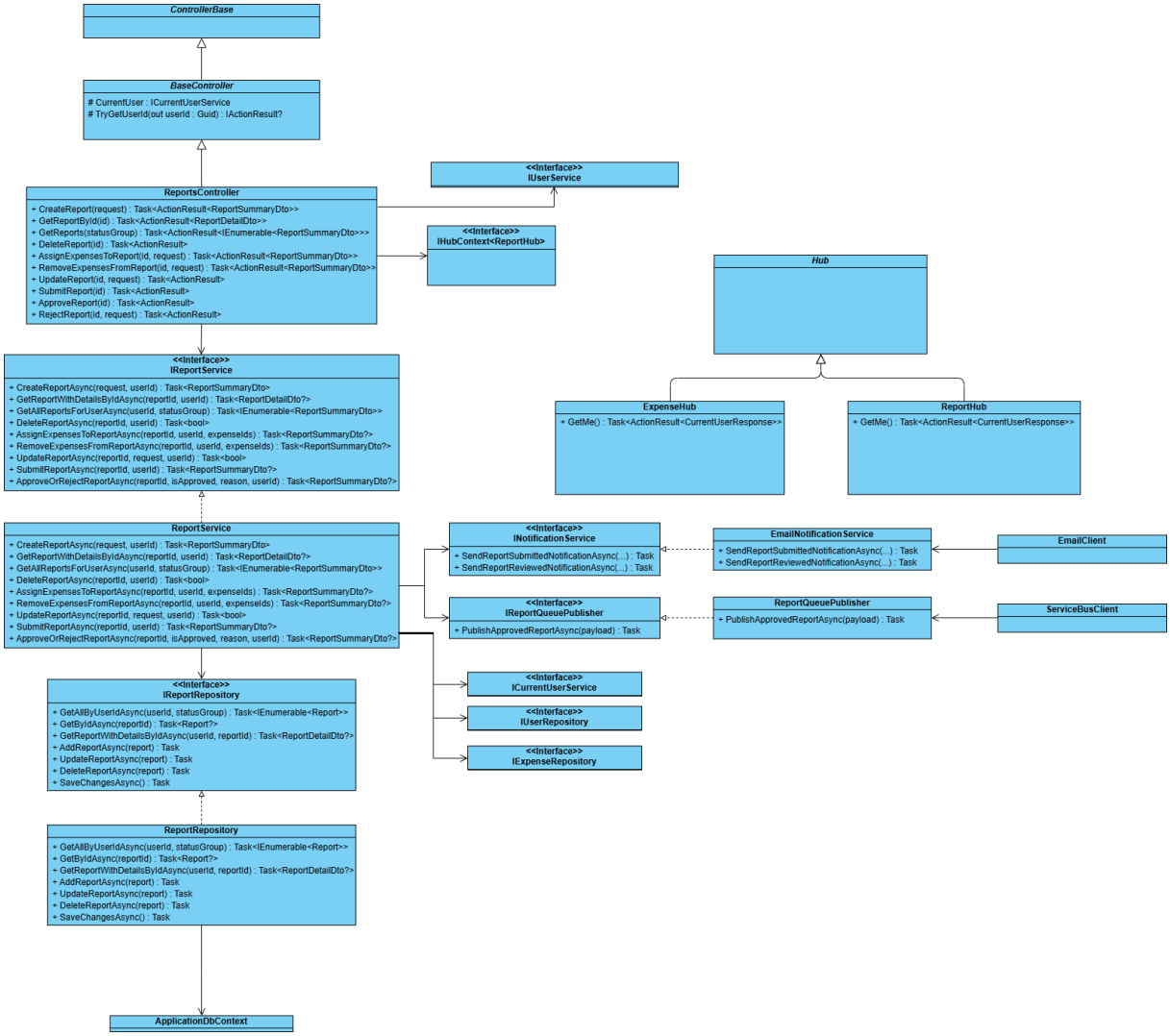

Figure D.2.1: UML Class Diagram - Part 2: Report, Notification, and Real-time Hub Workflows. [Own creation]